

ICPC Team Reference Document

Tom w Jerry — Monofia

Contents

1	Clean	3
1.1	clean	3
1.2	geo	3
2	Combinatorics	5
2.1	nCr	5
2.2	stars_and_bars	5
3	Data Structures	6
3.1	2D BIT	6
3.2	4D prefix sum	6
3.3	BIT	7
3.4	Coordinate Compression	8
3.5	DSU with array sz	8
3.6	DSU	8
3.7	Dynamic Segment Tree	8
3.8	Lazy Segment Tree	9
3.9	Max Sum in Subarray With update	10
3.10	Merge Sort Tree With update	11
3.11	Merge sort tree	12
3.12	MO	12
3.13	Monotonic Queue	13
3.14	Monotonic Stack	13
3.15	Order Set	14
3.16	Palindromic Tree Persistent	14
3.17	Persistent Trie	15
3.18	Segment Tree Beats	15
3.19	Segment Tree	18
3.20	Segtree Tree Iterative	18
3.21	Sparse Segment Tree	19
3.22	Sparse_Table	19
3.23	SQRT	20
3.24	Treap persistent	20
4	DP Optimizations	21

4.1	Convex Hull Trick	21
5	Fast	22
5.1	dp bitset	22
6	Geometry	22
6.1	Angle Between Vectors	22
6.2	Area of Polygon	23
6.3	Area of Triangle	23
6.4	Bisector	23
6.5	Center Circle Pass 3 points	24
6.6	Check if polygon is rectangle and Square	24
6.7	Dist Between 2 Segment	25
6.8	Dist from point to line	26
6.9	Dist from point to seg	26
6.10	Distance between 2 polygons	27
6.11	Distance between rays	29
6.12	Equation Line by point and vector	29
6.13	Equation Line passing through 2 points	29
6.14	Finding common tangents to two circles	30
6.15	Intersec Between 2 line	31
6.16	Intersection between 2 Circle	31
6.17	Intersection Between Circle and Line	32
6.18	Intersection between line and segment	32
6.19	Intersection between two segments	33
6.20	Is 2 points on same side of line	34
6.21	Is 2 Segment Intersect	34
6.22	Is point on segment	35
6.23	Is the point on line	35
6.24	Is the point on ray	35
6.25	Lattice Points	36
6.26	Length of the union of segments	36
6.27	Location of Point in line	36
6.28	MEC	37
6.29	Minkowski sum of convex polygons	37
6.30	Point Location Relative to Polygon	38
6.31	Polar Angle	39
6.32	Polar Sort	39
6.33	struct line	39
6.34	type of intersection between Circle and Line	40

6.35	vector length	41	11.1	ahoCorasick	54
6.36	winding Number	41	11.2	compute_automaton	55
6.37	WindingNumber for Integers	42	11.3	Hashing	56
7	Graph Algorithms	42	11.4	Hashing2D	56
7.1	articulation_points	42	11.5	KMP for integer	57
7.2	BCC	43	11.6	KMP	57
7.3	Bellman-Ford	43	11.7	SuffixArrayWithSparseTable	58
7.4	Bridges	44	11.8	SuffixArrayWithVectorInt	59
7.5	Bron-Kerbosch	44	11.9	Suffix_Array	60
7.6	Dinics Algorithm	45	11.10	Trie	61
7.7	Floyed	46	11.11	Trie01	62
7.8	Hungarian Algorithm	46	12	Trees	62
7.9	SCC_Kosaraju	47	12.1	Centroid	62
7.10	SCC_Tarjan_s	47	12.2	diameter	63
7.11	small to large	48	12.3	eular_tour	63
8	Math	49	12.4	HLD	64
8.1	bezout_sIdentity	49	12.5	LCA with Sparse Tables	65
8.2	Fast_Power	49	12.6	lca	65
8.3	MatrixExpo	49	12.7	Max Query by LCA	66
8.4	Mod INV by phi	50	12.8	MO on Tree	66
8.5	mod inverse for not prime	50			
8.6	NTT	50			
8.7	XOR Basis	51			
9	Number Theory	52			
9.1	CRT	52			
9.2	divisors	52			
9.3	Euler_s_totient	52			
9.4	is_prime	52			
9.5	linearSieve	53			
9.6	Miller-Rabin	53			
9.7	Pollard Rho	53			
9.8	PrimeFactorization	54			
10	Random	54			
10.1	random_range	54			
11	String Algorithms	54			

1 Clean

1.1 clean

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     return 0;
9 }
10
11 // 8 neighbors
12
13 int Dx[] = {-1, -1, -1, 0, 0, 1, 1, 1};
14 int Dy[] = {-1, 0, 1, -1, 1, -1, 0, 1};
15
16 // Knight Movement
17
18 int Kx[] = {2, 1, -1, -2, -2, -1, 1, 2};
19 int Ky[] = {1, 2, 2, 1, -1, -2, -2, -1};
20
21 // 4 neighbors
22
23 int dx[] = {-1, 1, 0, 0};
24 int dy[] = {0, 0, -1, 1};
25 char dir[] = {'U', 'D', 'L', 'R'};
26
27 /*
28 freopen("hps.in", "r", stdin);
29 freopen("hps.out", "w", stdout);
30 */

```

1.2 geo

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     // positive number
13     if (val > EPS) return 1;
14     // negative number
15     if (val < -EPS) return -1;
16     // 0
17     return 0;
18 }
19
20 // TODO if (0, 0) not ORIGIN
21 // will be positive if the rotation from a to b is performed counter-clockwise and negative
22 // otherwise
23 // so for two vectors that are "close" in angle (within 180 deg) the sign of the cross product
24 // correctly tells which has the larger angle.
25 T cross(pt v, pt w, pt origin = {0, 0}) {
26     return (v.x - origin.x) * (w.y - origin.y) - (v.y - origin.y) * (w.x - origin.x);
27 }
28
29 // the dot product is positive if the angle between them is acute, negative if it is obtuse and
30 // it equals zero if they are orthogonal.

```

```

28 T dot(pt v, pt w) {
29     return v.x * w.x + v.y * w.y;
30 }
31
32 void sort_points(vector<pt> &v) {
33     sort(v.begin(), v.end(), [](pt &a, pt &b) {
34         return sgn(fabs(a.x - b.x)) ? a.x < b.x : a.y < b.y;
35     });
36 }
37
38 // erase(unique) with double
39 void unique(vector<pt> &v) {
40     auto cmp_eq = [](pt &a, pt &b) {
41         return abs(a.x - b.x) < EPS && abs(a.y - b.y) < EPS;
42     };
43     v.erase(unique(v.begin(), v.end(), cmp_eq), v.end());
44 }
45
46 bool is_int(T factor) {
47     return (fabsl(factor - roundl(factor)) < 1e-5);
48 }
49
50 const double PI = acos(-1.0);
51
52 // Convert radians to degrees
53 double radToDeg(double rad) {
54     return rad * 180.0 / PI;
55 }
56
57 // Convert degrees to radians
58 double degToRad(double deg) {
59     return deg * PI / 180.0;
60 }
61
62 // returns the angle (in radians) between the positive x-axis and the point (x, y)
63 double angle(pt p) {
64     return atan2l(p.y, p.x);
65 }
66
67 bool equal(T a, T b) {
68     return (abs(a - b) < EPS);
69 }
70
71 T dist(pt a, pt b) {
72     return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
73 }
74
75 T sq(pt p) {
76     return p.x * p.x + p.y * p.y;
77 }
78
79 pt translate(pt v, pt p) {
80     return p + v;
81 }
82
83 pt scale(pt c, T factor, pt p) {
84     return c + (p - c) * factor;
85 }
86
87 // rotate vector v by angle c by radian
88 pt rotate(pt v, T c) {
89     return {v.x * cos(c) - v.y * sin(c), v.x * sin(c) + v.y * cos(c)};
90 }
91
92 pt rot(pt p, T a) {
93     return {p.x * cos(a) - p.y * sin(a), p.x * sin(a) + p.y * cos(a)};
94 }
95

```

```

96 pt perp(pt p) {
97     return {-p.y, p.x};
98 }
99
100 pt linearTransfo(pt p, pt q, pt r, pt fp, pt fq) {
101     return fp + (r - p) * (fq - fp) / (q - p);
102 }
103
104 bool isPerp(pt v, pt w) {
105     return dot(v, w) == 0;
106 }
107
108 double angle(pt v, pt w) {
109     double cosTheta = dot(v, w) / abs(v) / abs(w);
110     return acos(max(-1.0, min(1.0, cosTheta)));
111 }
112
113 // determining the direction of the points (right -1, left 1, or on a line 0).
114 T orient(pt a, pt b, pt c) {
115     return cross(b - a, c - a);
116 }
117
118 bool inAngle(pt a, pt b, pt c, pt p) {
119     assert(orient(a,b,c) != 0);
120     if (orient(a, b, c) < 0) swap(b, c);
121     return orient(a, b, p) >= 0 && orient(a, c, p) <= 0;
122 }
123
124 double orientedAngle(pt a, pt b, pt c) {
125     return (orient(a, b, c) >= 0 ? angle(b - a, c - a) : 2 * M_PI - angle(b - a, c - a));
126 }
127
128 bool half(pt p) {
129     // true if in blue half
130     assert(p.x != 0 || p.y != 0);
131     return p.y > 0 || (p.y == 0 && p.x < 0);
132 }
133
134 // around origin
135 void polarSort(vector<pt> &v) {
136     sort(v.begin(), v.end(), [](pt v, pt w) {
137         return make_tuple(half(v), 0, sq(v)) < make_tuple(half(w), cross(v, w), sq(w));
138     });
139 }
140
141 // around point o
142 void polarSortAround(pt o, vector<pt> &v) {
143     sort(v.begin(), v.end(), [o](pt v, pt w) {
144         return make_tuple(half(v - o), 0) < make_tuple(half(w - o), cross(v - o, w - o));
145     });
146 }
147
148 struct line {
149     pt v;
150     T c;
151
152     // From direction vector v and offset c
153     line(pt v, T c) : v(v), c(c) {
154     }
155
156     // From equation ax + by = c
157     line(T a, T b, T c) : v(b, -a), c(c) {
158     }
159
160     // From points P and Q
161     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
162     }
163 }

```

```

164 // Determine the direction of a point relative to a straight line
165 T side(pt p) {
166     return cross(v, p) - c;
167 }
168
169 T dist(pt p) {
170     return abs(side(p)) / abs(v);
171 }
172
173 T sqDist(pt p) {
174     return side(p) * side(p) / (double) sq(v);
175 }
176
177 line perpThrough(pt p) {
178     return {p, p + perp(v)};
179 }
180
181 // Sorting along a line
182 bool cmpProj(pt p, pt q) {
183     return dot(v, p) < dot(v, q);
184 }
185
186 // translate a line l by vector t
187 line translate(pt t) {
188     return {v, c + cross(v, t)};
189 }
190
191 // s shifting line l to the left by a certain distance
192 line shiftLeft(T dist) {
193     return {v, c + dist * abs(v)};
194 }
195
196 pt proj(pt p) {
197     return p - perp(v) * side(p) / sq(v);
198 }
199
200 pt refl(pt p) {
201     return p - perp(v) * (T) 2.0 * side(p) / sq(v);
202 }
203 };
204
205 // intersection point between two lines l1 and l2
206 bool inter(line l1, line l2, pt &out) {
207     T d = cross(l1.v, l2.v);
208     if (d == 0) return false;
209     out = (l2.v * l1.c - l1.v * l2.c) / d;
210     return true;
211 }
212
213 // An angle bisector of two (non-parallel) lines l1 and l2 is a line that forms equal angles
    with l1 and l2.
214 line bisector(line l1, line l2, bool interior) {
215     assert(cross(l1.v, l2.v) != 0);
216     T sign = interior ? 1 : -1;
217     return {
218         l2.v / abs(l2.v) + l1.v / abs(l1.v) * sign,
219         l2.c / abs(l2.v) + l1.c / abs(l1.v) * sign
220     };
221 }
222
223 // checks if a point P lies on the disk of diameter [AB].
224 bool inDisk(pt a, pt b, pt p) {
225     return dot(a - p, b - p) <= 0;
226 }
227
228 // check if a point P lies on segment [AB].
229 bool onSegment(pt a, pt b, pt p) {
230     return orient(a, b, p) == 0 && inDisk(a, b, p);

```

```

231 }
232
233 // intersection between two segments
234 bool properInter(pt a, pt b, pt c, pt d, pt &out) {
235     T oa = orient(c, d, a), ob = orient(c, d, b), oc = orient(a, b, c), od = orient(a, b, d);
236     // Proper intersection exists if opposite signs
237     if (oa * ob < 0 && oc * od < 0) {
238         out = (a * ob - b * oa) / (ob - oa);
239         return true;
240     }
241     return false;
242 }
243
244 // To create sets of points we need a comparison function
245 struct cmpX {
246     bool operator()(pt a, pt b) {
247         return make_pair(a.x, a.y) < make_pair(b.x, b.y);
248     }
249 };
250
251 set<pt, cmpX> inters(pt a, pt b, pt c, pt d) {
252     pt out;
253     if (properInter(a, b, c, d, out)) return {out};
254     set<pt, cmpX> s;
255     if (onSegment(c, d, a)) s.insert(a);
256     if (onSegment(c, d, b)) s.insert(b);
257     if (onSegment(a, b, c)) s.insert(c);
258     if (onSegment(a, b, d)) s.insert(d);
259     return s;
260 }
261
262 double areaTriangle(pt a, pt b, pt c) {
263     return abs(cross(b - a, c - a)) / 2.0;
264 }
265
266 double areaPolygon(vector<pt> p) {
267     double area = 0.0;
268     for (int i = 0, n = p.size(); i < n; i++) {
269         area += cross(p[i], p[(i + 1) % n]); // wrap back to 0 if i == n-1
270     }
271     return abs(area) / 2.0;
272 }
273
274 // true if P at least as high as A (blue part)
275 bool above(pt a, pt p) {
276     return p.y >= a.y;
277 }
278
279 // check if [PQ] crosses ray from A
280 bool crossesRay(pt a, pt p, pt q) {
281     return (above(a, q) - above(a, p)) * orient(a, p, q) > 0;
282 }
283
284 // if strict true, returns false when A is on the boundary
285 bool inPolygon(vector<pt> p, pt a, bool strict = true) {
286     int numCrossings = 0;
287     for (int i = 0, n = p.size(); i < n; i++) {
288         if (onSegment(p[i], p[(i + 1) % n], a))
289             return !strict;
290         numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
291     }
292     return numCrossings & 1;
293 }
294
295 int main() {
296     return 0;
297 }

```

2 Combinatorics

2.1 nCr

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int N = 1e6 + 5, mod = 1e9 + 7;
5 vector<long long> fact(N + 5), inv(N + 5);
6
7 long long fp(long long b, long long p) {
8     long long res = 1;
9     while (p) {
10         if (p & 1) {
11             res = (1LL * res * b) % mod;
12         }
13         b = (1LL * b * b) % mod;
14         p >>= 1;
15     }
16     return res;
17 }
18
19 void preCalc() {
20     fact[0] = inv[0] = 1;
21     for (int i = 1; i < N; i++) {
22         fact[i] = (1LL * i * fact[i - 1]) % mod;
23         inv[i] = fp(fact[i], mod - 2);
24     }
25 }
26
27 long long nCr(long long n, long long r) {
28     if (r > n) return 0;
29     return (1LL * ((1LL * fact[n] * inv[r]) % mod) * inv[n - r]) % mod;
30 }
31
32 // n up to 1e10 and r up to 1e5
33 int nCr(int n, int r) {
34     long long ret = 1;
35     for (int i = 0; i < r; ++i) {
36         ret = ret * ((n - i) % mod) % mod;
37     }
38     ret *= inv[r];
39     ret %= mod;
40     return ret;
41 }
42
43 // without mod
44 long long nCr(long long n, long long r) {
45     long long ans = 1;
46     for (int i = 1; i <= r; ++i) {
47         ans = ans * (n - r + i) / i;
48     }
49     return ans;
50 }
51 }

```

2.2 stars_and_bars

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int N = 2e6 + 5, mod = 1e9 + 7;
5 vector<int> fact(N + 5), inv(N + 5);
6
7 int fp(int b, int p) {
8     int res = 1;
9     while (p) {

```

```

10     if (p & 1) {
11         res = (1LL * res * b) % mod;
12     }
13     b = (1LL * b * b) % mod;
14     p >>= 1;
15 }
16 return res;
17 }
18
19 void preCalc() {
20     fact[0] = inv[0] = 1;
21     for (int i = 1; i < N; i++) {
22         fact[i] = (1LL * i * fact[i - 1]) % mod;
23         inv[i] = fp(fact[i], mod - 2);
24     }
25 }
26
27 int nCr(int n, int r) {
28     if (r > n) return 0;
29     return (1LL * ((1LL * fact[n] * inv[r]) % mod) * inv[n - r]) % mod;
30 }
31
32 // n balls, m buckets
33 int starsAndBars(int n, int m) {
34     if (n < 0) return 0;
35     return nCr(n + m - 1, m - 1);
36 }

```

3 Data Structures

3.1 2D BIT

```

1 // https://codeforces.com/gym/100247/problem/K
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 struct BIT2D {
6     int n;
7     vector<vector<int>> > vals, bit;
8
9     BIT2D(int n, vector<array<int, 2>> &todo) : n(n), vals(n + 1), bit(n + 1) {
10         sort(todo.begin(), todo.end(), [](auto a, auto b) {
11             return a[1] < b[1];
12         });
13
14         // TODO 1-Based
15         for (int i = 1; i <= n; i++) {
16             vals[i].push_back(0);
17         }
18
19         for (auto [r, c]: todo) {
20             while (r <= n) {
21                 if (vals[r].back() != c) {
22                     vals[r].push_back(c);
23                 }
24                 r += r & -r;
25             }
26         }
27
28         for (int i = 1; i <= n; i++) {
29             bit[i].resize(vals[i].size());
30         }
31     }
32
33     int get(vector<int> &v, int c) {
34         return upper_bound(v.begin(), v.end(), c) - v.begin() - 1;
35     }

```

```

36
37 // add val to point(r, c)
38 void add(int r, int c, int val) {
39     while (r <= n) {
40         int idx = get(vals[r], c);
41         while (idx < bit[r].size()) {
42             bit[r][idx] += val;
43             idx += idx & -idx;
44         }
45         r += r & -r;
46     }
47 }
48
49 // sum of points with row in [1, r] and column in [1, c]
50 int query(int r, int c) {
51     int sum = 0;
52     while (r > 0) {
53         int idx = get(vals[r], c);
54         while (idx > 0) {
55             sum += bit[r][idx];
56             idx -= idx & -idx;
57         }
58         r -= r & -r;
59     }
60     return sum;
61 }
62
63 // sum of points with row in [r1, r2] and column in [c1, c2]
64 int query(int r1, int c1, int r2, int c2) {
65     return query(r2, c2) - query(r2, c1 - 1) - query(r1 - 1, c2) + query(r1 - 1, c1 - 1);
66 }
67 };
68
69 int main() {
70     ios::sync_with_stdio(false);
71     cin.tie(nullptr);
72
73     int n;
74     cin >> n;
75     vector<array<int, 3>> teams(n);
76     for (int i = 0; i < n; i++) {
77         cin >> teams[i][0] >> teams[i][1] >> teams[i][2];
78     }
79     sort(teams.begin(), teams.end());
80
81     vector<array<int, 2>> todo;
82     for (int i = 0; i < n; i++) {
83         todo.push_back({teams[i][1], teams[i][2]});
84     }
85
86     BIT2D bit(n, todo);
87
88     // bad is a1 < a2 and b1 < b2 and c1 < c2
89     long long tot = 1LL * n * (n - 1) / 2, bad = 0;
90     for (int i = 0; i < n; i++) {
91         bad += bit.query(teams[i][1] - 1, teams[i][2] - 1);
92         bit.add(teams[i][1], teams[i][2], 1);
93     }
94     cout << tot - bad;
95
96     return 0;
97 }

```

3.2 4D prefix sum

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3
4 int n, m, q;
5 long long pref[45][45][45][45];
6 int pref2D[45][45], zeros[45][45][45][45];
7
8 void build4D() {
9     for (int a = 1; a <= n; a++) {
10         for (int b = 1; b <= m; b++) {
11             for (int c = 1; c <= n; c++) {
12                 for (int d = 1; d <= m; d++) {
13                     long long ret = zeros[a][b][c][d];
14                     for (int mask = 1; mask < (1 << 4); mask++) {
15                         int na = (mask & (1 << 0)) ? a - 1 : a;
16                         int nb = (mask & (1 << 1)) ? b - 1 : b;
17                         int nc = (mask & (1 << 2)) ? c - 1 : c;
18                         int nd = (mask & (1 << 3)) ? d - 1 : d;
19                         if (na >= 1 && nb >= 1 && nc >= 1 && nd >= 1) {
20                             int sign = (__builtin_popcount(mask) & 1 ? 1 : -1);
21                             ret += sign * pref[na][nb][nc][nd];
22                         }
23                     }
24                     pref[a][b][c][d] = ret;
25                 }
26             }
27         }
28     }
29 }
30
31 // TODO 1-based
32 long long query(int a, int b, int c, int d) {
33     long long res = 0;
34     for (int mask = 0; mask < (1 << 4); mask++) {
35         int na = (mask & (1 << 0)) ? a - 1 : c;
36         int nb = (mask & (1 << 1)) ? b - 1 : d;
37         int nc = (mask & (1 << 2)) ? a - 1 : c;
38         int nd = (mask & (1 << 3)) ? b - 1 : d;
39         if (na >= 1 && nb >= 1 && nc >= 1 && nd >= 1) {
40             int sign = (__builtin_popcount(mask) & 1 ? -1 : 1);
41             res += sign * pref[na][nb][nc][nd];
42         }
43     }
44     return res;
45 }
46
47 int main() {
48     ios::sync_with_stdio(false);
49     cin.tie(nullptr);
50
51     cin >> n >> m >> q;
52     vector<vector<int>> > grid(n + 1, vector<int>(m + 1));
53     for (int i = 1; i <= n; i++) {
54         string s;
55         cin >> s;
56         for (int j = 1; j <= m; j++) {
57             grid[i][j] = s[j - 1] - '0';
58             pref2D[i][j] = grid[i][j] + pref2D[i - 1][j] + pref2D[i][j - 1] - pref2D[i - 1][j - 1];
59         }
60     }
61
62     for (int a = 1; a <= n; a++) {
63         for (int b = 1; b <= m; b++) {
64             for (int c = a; c <= n; c++) {
65                 for (int d = b; d <= m; d++) {
66                     zeros[a][b][c][d] = (pref2D[c][d] - pref2D[a - 1][d] - pref2D[c][b - 1] + pref2D[a - 1][b - 1] == 0);
67                 }
68             }
69         }
70     }

```

```

69     }
70 }
71
72 build4D();
73
74 while (q--) {
75     int a, b, c, d;
76     cin >> a >> b >> c >> d;
77     cout << query(a, b, c, d) << '\n';
78 }
79
80 return 0;
81 }

```

3.3 BIT

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template<typename T>
5 struct BIT {
6     int n;
7     vector<T> bit; // TODO 1-Based
8
9     BIT(int _n = 0) {
10         init(_n);
11     }
12
13     void init(int _n) {
14         n = _n;
15         bit.assign(n + 1, 0);
16     }
17
18     void build(const vector<T> &a) {
19         for (int i = 1; i <= n; ++i) {
20             bit[i] += a[i];
21         }
22
23         for (int i = 1; i <= n; ++i) {
24             int j = i + (i & -i);
25             if (j <= n) {
26                 bit[j] += bit[i];
27             }
28         }
29     }
30
31     void add(int idx, T delta) {
32         while (idx <= n) {
33             bit[idx] += delta;
34             idx += idx & -idx;
35         }
36     }
37
38     T sum(int idx) {
39         T ret = 0;
40         while (idx > 0) {
41             ret += bit[idx];
42             idx -= idx & -idx;
43         }
44         return ret;
45     }
46
47     T query(int l, int r) {
48         if (r < 1) return 0;
49         return sum(r) - sum(l - 1);
50     }
51 }

```

```

52 // returns smallest index such that sum(i) >= k.
53 int lower_bound(T k) {
54     if (k <= 0) return 1;
55
56     int pos = 0, bitMask = 1;
57     while ((bitMask << 1) <= n) {
58         bitMask <<= 1;
59     }
60
61     for (; bitMask; bitMask >>= 1) {
62         int nxt = pos + bitMask;
63         if (nxt <= n && bit[nxt] < k) {
64             k -= bit[nxt];
65             pos = nxt;
66         }
67     }
68     return pos + 1;
69 }
70 };

```

3.4 Coordinate Compression

```

1 void compression(vector<int> &a, int &n) {
2     vector<int> comp(a.begin() + 1, a.end()); // skip index 0 (1-based)
3
4     sort(comp.begin(), comp.end());
5     comp.erase(unique(comp.begin(), comp.end()), comp.end());
6
7     for (int i = 1; i <= n; i++) {
8         a[i] = lower_bound(comp.begin(), comp.end(), a[i]) - comp.begin() + 1;
9     }
10    n = comp.size();
11 }
12
13
14
15 map<long long, int> id;
16 void compression(vector<int> a, int &n) {
17     sort(a.begin(), a.end());
18     a.erase(unique(a.begin(), a.end()), a.end());
19
20     for (int i = 0; i < (int) a.size(); i++) {
21         id[a[i]] = i + 1;
22     }
23     n = a.size();
24 }

```

3.5 DSU with array sz

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct DSU {
5     vector<int> par, sz;
6
7     DSU(int n) {
8         par.resize(n + 1);
9         sz.resize(n + 1);
10        for (int u = 1; u <= n; u++) {
11            par[u] = u;
12            sz[u] = 1;
13        }
14    }
15
16    int parent(int u) {
17        if (par[u] == u) return u;

```

```

18        return par[u] = parent(par[u]);
19    }
20
21    void join(int u, int v) {
22        u = parent(u), v = parent(v);
23        if (u == v) return;
24        if (sz[u] < sz[v]) swap(u, v);
25        par[v] = u;
26        sz[u] += sz[v];
27    }
28
29    bool same(int u, int v) {
30        return parent(u) == parent(v);
31    }
32 };

```

3.6 DSU

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct DSU {
5     int comp;
6     vector<int> p;
7
8     DSU(int n) {
9         comp = n;
10        p.resize(n + 1, -1);
11    }
12
13    int parent(int u) {
14        if (p[u] < 0) return u;
15        return p[u] = parent(p[u]);
16    }
17
18    void join(int u, int v) {
19        int pu = parent(u), pv = parent(v);
20        if (pu == pv) return;
21        if (p[pu] > p[pv]) swap(pu, pv);
22        p[pu] += p[pv];
23        p[pv] = pu;
24        comp--;
25    }
26
27    int sz(int u) {
28        return -1 * p[parent(u)];
29    }
30
31    bool same(int u, int v) {
32        return parent(u) == parent(v);
33    }
34
35    int count() {
36        return comp;
37    }
38 };
39
40 int main() {
41     return 0;
42 }

```

3.7 Dynamic Segment Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

```

4 class SparseSegtree {
5 private:
6     struct Node {
7         long long freqmn = 0, freqmx = 0, lazy = 0;
8         Node *left = nullptr;
9         Node *right = nullptr;
10    };
11
12    Node *root = new Node;
13    const long long n;
14
15    long long combine_min(long long a, long long b) {
16        return min(a, b);
17    }
18
19    long long combine_max(long long a, long long b) {
20        return max(a, b);
21    }
22
23    void apply(Node *cur, long long val) {
24        (cur->lazy) += val;
25        (cur->freqmn) += val;
26        (cur->freqmx) += val;
27    }
28
29    void push_down(Node *cur, long long l, long long r) {
30        if ((cur->left) == nullptr) {
31            (cur->left) = new Node;
32        }
33
34        if ((cur->right) == nullptr) {
35            (cur->right) = new Node;
36        }
37
38        apply(cur->left, cur->lazy);
39        apply(cur->right, cur->lazy);
40        cur->lazy = 0;
41    }
42
43    void range_set(Node *cur, long long l, long long r, long long ql, long long qr, long long
    val) {
44        if (qr < l || ql > r) { return; }
45        if (ql <= l && r <= qr) {
46            apply(cur, val);
47        } else {
48            push_down(cur, l, r);
49            long long m = (l + r) / 2;
50            range_set(cur->left, l, m, ql, qr, val);
51            range_set(cur->right, m + 1, r, ql, qr, val);
52            (cur->freqmn) = combine_min((cur->left)->freqmn, (cur->right)->freqmn);
53            (cur->freqmx) = combine_max((cur->left)->freqmx, (cur->right)->freqmx);
54        }
55    }
56
57    pair<long long, long long> range_sum(Node *cur, long long l, long long r, long long ql, long
    long qr) {
58        if (qr < l || ql > r) {
59            return make_pair(2e18, -2e18);
60        }
61        if (ql <= l && r <= qr) {
62            return make_pair(cur->freqmn, cur->freqmx);
63        }
64        push_down(cur, l, r);
65        long long m = (l + r) / 2;
66        auto n1 = range_sum(cur->left, l, m, ql, qr);
67        auto n2 = range_sum(cur->right, m + 1, r, ql, qr);
68        return {combine_min(n1.first, n2.first), combine_max(n1.second, n2.second)};
69    }

```

```

70
71 public:
72     SparseSegtree(long long n) : n(n) {
73     }
74
75     void range_set(long long ql, long long qr, long long val) {
76         range_set(root, 0, n - 1, ql, qr, val);
77     }
78
79     pair<long long, long long> range_sum(long long ql, long long qr) {
80         return range_sum(root, 0, n - 1, ql, qr);
81     }
82 };
83
84 int main() {
85     ios::sync_with_stdio(false);
86     cin.tie(nullptr);
87
88     const long long RANGE_SIZE = 2e18;
89     SparseSegtree seg(RANGE_SIZE + 1);
90
91     int q;
92     cin >> q;
93     while (q--) {
94         char c;
95         long long l, p;
96         cin >> c >> l >> p;
97         if (c == '|') {
98             seg.range_set(p, p, l);
99             cout << 'S';
100        } else {
101            auto [mn, mx] = seg.range_sum(p, p + 1 - 1);
102            if (mn == mx) {
103                seg.range_set(p, p + 1 - 1, 1);
104                cout << 'S';
105            } else {
106                cout << 'U';
107            }
108        }
109    }
110
111    return 0;
112 }

```

3.8 Lazy Segment Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template<typename T>
5 struct segtree {
6     int sz = 1;
7
8     T neutral;
9
10    vector<T> t, lazy;
11    function<T(T, T)> combine;
12
13    segtree(int n, T init, function<T(T, T)> com) {
14        while (sz < n) sz += sz;
15        combine = com;
16        neutral = init;
17        t.resize(2 * sz);
18        lazy.resize(2 * sz);
19    }
20
21    void pull(int x) {

```

```

22     t[x] = combine(t[2 * x], t[2 * x + 1]);
23 }
24
25 void push(int x, int lx, int rx) {
26     if (!lazy[x]) return;
27     //TODO
28     t[x] += lazy[x];
29     if (lx != rx) {
30         lazy[2 * x] += lazy[x];
31         lazy[2 * x + 1] += lazy[x];
32     }
33     lazy[x] = 0;
34 }
35
36 void build(int x, int lx, int rx, vector<T> &v) {
37     if (lx == rx) {
38         //TODO 1-based
39         if (lx < (int) v.size()) {
40             t[x] = v[lx];
41         }
42         return;
43     }
44     int m = (lx + rx) >> 1;
45     build(2 * x, lx, m, v);
46     build(2 * x + 1, m + 1, rx, v);
47     pull(x);
48 }
49
50 void build(vector<T> &v) {
51     build(1, 1, sz, v);
52 }
53
54 void update(int x, int lx, int rx, int l, int r, T val) {
55     push(x, lx, rx);
56     if (lx > r || rx < l) return;
57     if (lx >= l && rx <= r) {
58         lazy[x] += val;
59         push(x, lx, rx);
60         return;
61     }
62     int m = (lx + rx) >> 1;
63     update(2 * x, lx, m, l, r, val);
64     update(2 * x + 1, m + 1, rx, l, r, val);
65     pull(x);
66 }
67
68 void update(int l, int r, T val) {
69     update(1, 1, sz, l, r, val);
70 }
71
72 T query(int x, int lx, int rx, int l, int r) {
73     push(x, lx, rx);
74     if (rx < l || lx > r) return neutral;
75     if (lx >= l && rx <= r) return t[x];
76     int m = (lx + rx) >> 1;
77     return combine(query(2 * x, lx, m, l, r), query(2 * x + 1, m + 1, rx, l, r));
78 }
79
80 T query(int l, int r) {
81     return query(1, 1, sz, l, r);
82 }
83 };
84
85 int main() {
86     ios::sync_with_stdio(false);
87     cin.tie(nullptr);
88 }

```

```

90     int n;
91     cin >> n;
92
93     vector<long long> arr(n + 1);
94     for (int i = 1; i <= n; i++) {
95         cin >> arr[i];
96     }
97
98     segtree<long long> mn(n + 1, LLONG_MAX, [](long long a, long long b) {
99         return min(a, b);
100     });
101     mn.build(arr);
102
103
104     return 0;
105 }

```

3.9 Max Sum in Subarray With update

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct node {
5      long long mx, sum, pre, mn_suf;
6
7      node() {
8          mx = sum = pre = mn_suf = 0;
9      }
10
11     node(long long v) {
12         sum = v, mx = pre = mn_suf = max(0LL, v);
13     }
14 };
15
16 node neutral() {
17     node ret;
18     ret.sum = 0;
19     ret.mx = ret.pre = ret.mn_suf = -1e18;
20     return ret;
21 }
22
23 struct segtree {
24     int sz;
25     vector<node> t;
26
27     segtree() {
28     }
29
30     segtree(int n) {
31         sz = 1;
32         while (sz < n) sz += sz;
33         t.resize(2 * sz);
34     }
35
36     node combine(node a, node b) {
37         node ret(0);
38         ret.sum = a.sum + b.sum;
39         ret.pre = max(a.pre, a.sum + b.pre);
40         ret.mn_suf = max(b.mn_suf, a.mn_suf + b.sum);
41         ret.mx = max({a.mx, b.mx, a.mn_suf + b.pre});
42         return ret;
43     }
44
45     void build(int lx, int rx, int x, vector<long long> &arr) {
46         if (lx == rx) {
47             if (lx < (int) arr.size()) {
48                 t[x] = node(arr[lx]);

```

```

49     }
50     return;
51 }
52 int md = (lx + rx) / 2;
53 build(lx, md, 2 * x, arr);
54 build(md + 1, rx, 2 * x + 1, arr);
55 t[x] = combine(t[2 * x], t[2 * x + 1]);
56 }
57
58 void build(vector<long long> &arr) {
59     build(1, sz, 1, arr);
60 }
61
62 void set(int lx, int rx, int x, int idx, long long val) {
63     if (lx == rx) {
64         t[x] = node(val);
65         return;
66     }
67
68     int md = (lx + rx) / 2;
69     if (idx <= md) set(lx, md, 2 * x, idx, val);
70     else set(md + 1, rx, 2 * x + 1, idx, val);
71
72     t[x] = combine(t[2 * x], t[2 * x + 1]);
73 }
74
75 void set(int idx, long long val) {
76     set(1, sz, 1, idx, val);
77 }
78
79 node query(int x, int lx, int rx, int l, int r) {
80     if (r < lx || l > rx) return neutral();
81     if (lx >= l && rx <= r) {
82         return t[x];
83     }
84     int m = (lx + rx) >> 1;
85     node a = query(2 * x, lx, m, l, r);
86     node b = query(2 * x + 1, m + 1, rx, l, r);
87     return combine(a, b);
88 }
89
90 node query(int l, int r) {
91     return query(1, 1, sz, l, r);
92 }
93 };
94
95 int main() {
96     ios::sync_with_stdio(false);
97     cin.tie(nullptr);
98
99     int n, q;
100     cin >> n >> q;
101     vector<long long> arr(n + 1);
102     for (int i = 1; i <= n; i++) {
103         cin >> arr[i];
104     }
105
106     segtree seg(n);
107
108     seg.build(arr);
109
110     while (q--) {
111         int t;
112         cin >> t;
113         if (t == 1) {
114             long long p, x;
115             cin >> p >> x;
116             seg.set(p, x);

```

```

117     } else {
118         int l, r;
119         cin >> l >> r;
120         cout << seg.query(l, r).mx << '\n';
121     }
122 }
123
124 return 0;
125 }

```

3.10 Merge Sort Tree With update

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template<typename T>
5
6 struct mstree {
7     int sz = 1;
8
9     vector<T> a;
10    vector<multiset<T> > t;
11
12    mstree(int n) {
13        while (sz < n) sz += sz;
14        t.resize(2 * sz);
15        a.assign(sz + 1, T());
16    }
17
18    void build(int x, int lx, int rx, vector<T> &v) {
19        if (lx == rx) {
20            if (lx < (int) v.size()) {
21                t[x].insert(v[lx]);
22                a[lx] = v[lx];
23            }
24            return;
25        }
26
27        int md = (lx + rx) >> 1;
28        build(2 * x, lx, md, v);
29        build(2 * x + 1, md + 1, rx, v);
30        t[x].insert(t[2 * x].begin(), t[2 * x].end());
31        t[x].insert(t[2 * x + 1].begin(), t[2 * x + 1].end());
32    }
33
34    void build(const vector<T> &v) {
35        build(1, 1, sz, v);
36    }
37
38    // count of elements >= val in [l, r]
39    int query(int x, int lx, int rx, int l, int r, T &val) {
40        if (rx < l || lx > r) return 0;
41
42        if (lx >= l && rx <= r) {
43            auto it = t[x].lower_bound(val);
44            return (int) distance(it, t[x].end()); // count how many elements are from it up to
45            the end of the multiset
46        }
47
48        int md = (lx + rx) >> 1;
49        return query(2 * x, lx, md, l, r, val) + query(2 * x + 1, md + 1, rx, l, r, val);
50    }
51
52    int query(int l, int r, T &val) {
53        return query(1, 1, sz, l, r, val);
54    }

```

```

55 void update(int x, int lx, int rx, int pos, T &oldv, T &newv) {
56     auto it = t[x].find(oldv);
57     if (it != t[x].end()) {
58         t[x].erase(it);
59     }
60
61     t[x].insert(newv);
62     if (lx == rx) {
63         a[pos] = newv;
64         return;
65     }
66
67     int md = (lx + rx) >> 1;
68     if (pos <= md) update(2 * x, lx, md, pos, oldv, newv);
69     else update(2 * x + 1, md + 1, rx, pos, oldv, newv);
70 }
71
72 void update(int pos, T &newVal) {
73     T oldVal = a[pos];
74     update(1, 1, sz, pos, oldVal, newVal);
75 }
76 };

```

3.11 Merge sort tree

```

1  template<typename T>
2  struct mstree {
3      int sz = 1;
4
5      //TODO
6      T neutral = 0;
7
8      vector<vector<T> > t;
9
10     mstree(int n) {
11         while (sz < n) sz += sz;
12         t.resize(2 * sz);
13     }
14
15     T combine(T a, T b) {
16         //TODO
17         return a + b;
18     }
19
20     void pull(int x) {
21         t[x] = combine(t[2 * x], t[2 * x + 1]);
22     }
23
24     void build(int x, int lx, int rx, vector<T> &v) {
25         if (lx == rx) {
26             //TODO 1-based
27             if (lx <= (int) (v.size())) {
28                 t[x].push_back(v[lx]);
29             }
30             return;
31         }
32         int m = (lx + rx) >> 1;
33         build(2 * x, lx, m, v);
34         build(2 * x + 1, m + 1, rx, v);
35         t[x].resize(rx - lx + 1);
36         merge(t[2 * x].begin(), t[2 * x].end(), t[2 * x + 1].begin(), t[2 * x + 1].end(), t[x].begin());
37     }
38
39     void build(vector<T> &v) {
40         // assert(v.size() == sz);
41         build(1, 1, sz, v);

```

```

42     }
43
44     // count of elements >= x and <= y
45     // return upper_bound(t[x].begin(), t[x].end(), y) - lower_bound(t[x].begin(), t[x].end(), qx);
46
47     // count of elements <= x in this node's range
48     // return upper_bound(tree[node].begin(), tree[node].end(), x) - tree[node].begin();
49
50     // count of elements >= x in this node's range
51     // return t[x].end() - lower_bound(t[x].begin(), t[x].end(), val);
52
53     // count of elements < x in this node's range
54     // lower_bound(tree[node].begin(), tree[node].end(), x) - tree[node].begin();
55
56     // count of elements > x in this node's range
57     // tree[node].end() - upper_bound(tree[node].begin(), tree[node].end(), x);
58
59     T query(int x, int lx, int rx, int l, int r, int val) {
60         if (rx < l || lx > r) return neutral;
61         if (lx >= l && rx <= r) {
62             //TODO
63             return t[x].end() - lower_bound(t[x].begin(), t[x].end(), val);
64         }
65         int m = (lx + rx) >> 1;
66         return combine(query(2 * x, lx, m, l, r, val), query(2 * x + 1, m + 1, rx, l, r, val));
67     }
68
69     T query(int l, int r, int x) {
70         // assert(l <= r && l >= 1 && r <= sz);
71         return query(1, 1, sz, l, r, x);
72     }
73 };

```

3.12 MO

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  //TODO
5  const int MAX = 1e6 + 5;
6
7  struct Query {
8      //TODO
9      int l, r, idx;
10 };
11
12 struct Mo {
13     vector<Query> queries;
14     vector<int> v, frq, ans;
15     int n, block_size, cur = 0, mo_left = 0, mo_right = -1; // 0-Based
16
17     Mo(int n_, vector<int> &v_) : n(n_), v(v_) {
18         block_size = ceil(sqrt(n));
19         frq.assign(MAX, 0);
20     }
21
22     void add_query(int l, int r, int idx) {
23         queries.push_back({l, r, idx});
24     }
25
26     void add(int idx) {
27         //TODO
28         frq[v[idx]]++;
29         if (frq[v[idx]] == 1) {
30             cur++;
31         }

```

```

32 }
33
34 void remove(int idx) {
35     //TODO
36     frq[v[idx]]--;
37     if (frq[v[idx]] == 0) {
38         cur--;
39     }
40 }
41
42 void process() {
43     sort(queries.begin(), queries.end(), [&](const Query &a, const Query &b) {
44         return make_pair(a.l / block_size, ((a.l / block_size) & 1) ? -a.r : +a.r) <
45         make_pair(b.l / block_size, ((b.l / block_size) & 1) ? -b.r : +b.r);
46     });
47     ans.resize(queries.size());
48
49     for (const auto &[l, r, idx]: queries) {
50         while (mo_left > l) add(--mo_left);
51         while (mo_right < r) add(++mo_right);
52         while (mo_left < l) remove(mo_left++);
53         while (mo_right > r) remove(mo_right--);
54         ans[idx] = cur;
55     }
56 }
57
58 void print_answers() {
59     for (auto &x: ans) {
60         cout << x << '\n';
61     }
62 }
63 };
64
65 int main() {
66     ios::sync_with_stdio(false);
67     cin.tie(nullptr);
68
69     int n;
70     cin >> n;
71     vector<int> v(n);
72     for (int i = 0; i < n; i++) {
73         cin >> v[i];
74     }
75
76     Mo mo(n, v);
77
78     int q;
79     cin >> q;
80     for (int i = 0; i < q; i++) {
81         int l, r;
82         cin >> l >> r;
83         mo.add_query(--l, --r, i);
84     }
85
86     mo.process();
87     mo.print_answers();
88
89     return 0;
90 }

```

3.13 Monotonic Queue

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Sliding Window Maximum / Minimum

```

```

5 // find the Maximum / Minimum element in a window of fixed size.
6
7 struct MonotonicQueue {
8     stack<int> st2;
9     stack<pair<int, int>> st1;
10
11     void push(int x) {
12         int mx = st1.empty() ? x : max(x, st1.top().second);
13         st1.push(make_pair(x, mx));
14     }
15
16     void pop() {
17         if (st2.empty()) {
18             while (!st1.empty()) {
19                 int nxt = st1.top().first;
20                 st2.push((st2.empty() ? nxt : max(st2.top(), nxt)));
21                 st1.pop();
22             }
23         }
24         st2.pop();
25     }
26
27     int get() {
28         if (st1.empty() || st2.empty()) {
29             return (st1.empty() ? st2.top() : st1.top().second);
30         }
31         return max(st1.top().second, st2.top());
32     }
33
34     bool empty() {
35         return (st1.empty() && st2.empty());
36     }
37
38     int size() {
39         return (st1.size() + st2.size());
40     }
41 };

```

3.14 Monotonic Stack

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> nextMax(vector<int> &a, int n) {
5     stack<int> st;
6     vector<int> ret(n, n);
7     for (int j = 0; j < n; j++) {
8         while (!st.empty() && a[st.top()] < a[j]) {
9             ret[st.top()] = j;
10            st.pop();
11        }
12        st.push(j);
13    }
14    return ret;
15 }
16
17 vector<int> prevMax(vector<int> &a, int n) {
18     stack<int> st;
19     vector<int> ret(n, -1);
20     for (int j = n - 1; j >= 0; j--) {
21         while (!st.empty() && a[st.top()] <= a[j]) {
22             ret[st.top()] = j;
23             st.pop();
24         }
25         st.push(j);
26     }
27     return ret;

```

```

28 }
29
30 vector<int> nextMin(vector<int> &a, int n) {
31     stack<int> st;
32     vector<int> ret(n, n);
33     for (int j = 0; j < n; j++) {
34         while (!st.empty() && a[st.top()] > a[j]) {
35             ret[st.top()] = j;
36             st.pop();
37         }
38         st.push(j);
39     }
40     return ret;
41 }
42
43 vector<int> prevMin(vector<int> &a, int n) {
44     stack<int> st;
45     vector<int> ret(n, -1);
46     for (int j = n - 1; j >= 0; j--) {
47         while (!st.empty() && a[st.top()] >= a[j]) {
48             ret[st.top()] = j;
49             st.pop();
50         }
51         st.push(j);
52     }
53     return ret;
54 }

```

3.15 Order Set

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 #define ordered_set tree<int, null_type, less<int>, rb_tree_tag,
   tree_order_statistics_node_update>
5 // find_by_order() returns an iterator to the k-th smallest element (counting from zero)
6 // order_of_key() returns the number of items in a set that are strictly smaller than our item
7 #define ordered_multiset tree<int, null_type, less_equal<int>, rb_tree_tag,
   tree_order_statistics_node_update>
8 #define ordered_multiset1 tree<int, null_type, greater_equal<int>, rb_tree_tag,
   tree_order_statistics_node_update>

```

3.16 Palindromic Tree Persistent

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 const int N = 1e6 + 9;
5
6 // cnt contains the number of palindromic suffixes of the node
7 // t[cur].smart_link[c] -> link to the maximum length of a palindromic suffix of t[cur]
8 // s.t. ch is the immediate previous char of that suffix
9 struct PalindromicTree {
10     static const int A = 2;
11     struct node {
12         int nxt[A], len, st, en, link, cnt, oc, smart_link[A];
13         node() {
14             memset(nxt, 0, sizeof nxt);
15             for (int i = 0; i < A; i++) {
16                 smart_link[i] = 1;
17             }
18         }
19     };
20     string s; vector<node> t;
21     vector<array<int, 4>> changes;
22     int sz, last;

```

```

23     vector<int> pref;
24     PalindromicTree() {
25         s = ""; t.resize(3);
26         sz = 2, last = 2;
27         t[1].len = -1, t[1].link = 1;
28         t[2].len = 0, t[2].link = 1;
29         changes.clear();
30     }
31     int extend(char c) {
32         int cur = last, curlen = 0, pos = s.size();
33         pref.push_back(0);
34         if (pos) {
35             pref[pos] = pref[pos - 1];
36         }
37         s += c; int ch = c - 'a';
38         if (pos - t[cur].len - 1 < 0 || s[pos - t[cur].len - 1] != c) {
39             cur = t[cur].smart_link[ch];
40         }
41         if (t[cur].nxt[ch]) {
42             changes.push_back({last, t[cur].nxt[ch], -1, -1});
43             last = t[cur].nxt[ch];
44             t[last].oc++;
45             return 0;
46         }
47         changes.push_back({last, -1, cur, ch});
48         sz++; last = sz;
49         t.emplace_back();
50         t[sz].oc = 1; t[sz].len = t[cur].len + 2;
51         pref[pos] = max(pref[pos], t[sz].len);
52         t[cur].nxt[ch] = sz;
53         t[sz].en = pos; t[sz].st = pos - t[sz].len + 1;
54         if (t[sz].len == 1) {
55             t[sz].link = 2; t[sz].cnt = 1;
56             t[sz].smart_link[ch] = 2;
57             return 1;
58         }
59         else {
60             t[sz].link = t[t[cur].smart_link[ch]].nxt[ch];
61             for (int i = 0; i < A; i++) {
62                 int x = t[sz].link;
63                 if (pos - t[x].len >= 0 && s[pos - t[x].len] - 'a' == i) {
64                     t[sz].smart_link[i] = x;
65                 }
66                 else {
67                     t[sz].smart_link[i] = t[x].smart_link[i];
68                 }
69             }
70         }
71         t[sz].cnt = 1 + t[t[sz].link].cnt;
72         return 1;
73     }
74     void rollback() {
75         if (s.size() == 0) return;
76         s.pop_back();
77         pref.pop_back();
78         auto x = changes.back();
79         int prvlst = x[0], oc = x[1], c = x[2], ch = x[3];
80         changes.pop_back();
81         last = prvlst;
82         if (oc == -1) {
83             t[c].nxt[ch] = 0;
84             t.pop_back();
85             sz--;
86         }
87         else {
88             t[oc].oc--;
89         }
90     }

```

```

91 }t;
92 int32_t main() {
93     ios_base::sync_with_stdio(0);
94     cin.tie(0);
95     int q; cin >> q;
96     while (q--) {
97         int ty; cin >> ty;
98         if (ty == 1) {
99             int k; cin >> k;
100             t.extend(char(k + 'a'));
101         }
102         else {
103             t.rollback();
104         }
105         cout << (t.s.empty() ? 0 : t.pref[t.s.size() - 1]) << '\n';
106     }
107     return 0;
108 }
109 // https://www.codechef.com/problems/BINPALIN

```

3.17 Persistent Trie

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  // find maximum value (x^a[j]) in the range (l,r) where l<=j<=r
6  const int N = 1e5 + 100;
7  const int K = 15;
8
9  struct node_t;
10 typedef node_t * pnode;
11
12 struct node_t {
13     int time;
14     pnode to[2];
15     node_t() : time(0) {
16         to[0] = to[1] = 0;
17     }
18     bool go(int l) const {
19         if (!this) return false;
20         return time >= l;
21     }
22     pnode clone() {
23         pnode cur = new node_t();
24         if (this) {
25             cur->time = time;
26             cur->to[0] = to[0];
27             cur->to[1] = to[1];
28         }
29         return cur;
30     }
31 };
32
33 pnode last;
34 pnode version[N];
35
36 void insert(int a, int time) {
37     pnode v = version[time] = last = last->clone();
38     for (int i = K - 1; i >= 0; --i) {
39         int bit = (a >> i) & 1;
40         pnode &child = v->to[bit];
41         child = child->clone();
42         v = child;
43         v->time = time;
44     }
45 }

```

```

46
47 int query(pnode v, int x, int l) {
48     int ans = 0;
49     for (int i = K - 1; i >= 0; --i) {
50         int bit = (x >> i) & 1;
51         if (v->to[bit]->go(l)) { // checking if this bit was inserted before the range
52             ans |= 1 << i;
53             v = v->to[bit];
54         } else {
55             v = v->to[bit ^ 1];
56         }
57     }
58     return ans;
59 }
60
61 void solve() {
62     int n, q;
63     scanf("%d %d", &n, &q);
64     last = 0;
65     for (int i = 0; i < n; ++i) {
66         int a;
67         scanf("%d", &a);
68         insert(a, i);
69     }
70     while (q--) {
71         int x, l, r;
72         scanf("%d %d %d", &x, &l, &r);
73         --l, --r;
74         printf("%d\n", query(version[r], ~x, l));
75         // Trie version[r] contains the trie for [0...r] elements
76     }
77 }
78 // credit: mochow13

```

3.18 Segment Tree Beats

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 2e5 + 9;
5
6  using ll = long long;
7
8  struct SGTBeats {
9      const ll inf = 1e18;
10     int n, n0;
11     ll max_v[4 * N], smax_v[4 * N], max_c[4 * N];
12     ll min_v[4 * N], smin_v[4 * N], min_c[4 * N];
13     ll sum[4 * N];
14     ll len[4 * N], ladd[4 * N], lval[4 * N];
15
16     void update_node_max(int k, ll x) {
17         sum[k] += (x - max_v[k]) * max_c[k];
18
19         if (max_v[k] == min_v[k]) {
20             max_v[k] = min_v[k] = x;
21         } else if (max_v[k] == smin_v[k]) {
22             max_v[k] = smin_v[k] = x;
23         } else {
24             max_v[k] = x;
25         }
26
27         if (lval[k] != inf && x < lval[k]) {
28             lval[k] = x;
29         }
30     }
31     void update_node_min(int k, ll x) {

```

```

32     sum[k] += (x - min_v[k]) * min_c[k];
33
34     if (max_v[k] == min_v[k]) {
35         max_v[k] = min_v[k] = x;
36     } else if (smax_v[k] == min_v[k]) {
37         min_v[k] = smax_v[k] = x;
38     } else {
39         min_v[k] = x;
40     }
41
42     if (lval[k] != inf && lval[k] < x) {
43         lval[k] = x;
44     }
45 }
46 void push(int k) {
47     if (n0 - 1 <= k) return;
48     if (lval[k] != inf) {
49         updateall(2 * k + 1, lval[k]);
50         updateall(2 * k + 2, lval[k]);
51         lval[k] = inf;
52         return;
53     }
54     if (ladd[k] != 0) {
55         addall(2 * k + 1, ladd[k]);
56         addall(2 * k + 2, ladd[k]);
57         ladd[k] = 0;
58     }
59     if (max_v[k] < max_v[2 * k + 1]) {
60         update_node_max(2 * k + 1, max_v[k]);
61     }
62     if (min_v[2 * k + 1] < min_v[k]) {
63         update_node_min(2 * k + 1, min_v[k]);
64     }
65
66     if (max_v[k] < max_v[2 * k + 2]) {
67         update_node_max(2 * k + 2, max_v[k]);
68     }
69     if (min_v[2 * k + 2] < min_v[k]) {
70         update_node_min(2 * k + 2, min_v[k]);
71     }
72 }
73 void update(int k) {
74     sum[k] = sum[2 * k + 1] + sum[2 * k + 2];
75
76     if (max_v[2 * k + 1] < max_v[2 * k + 2]) {
77         max_v[k] = max_v[2 * k + 2];
78         max_c[k] = max_c[2 * k + 2];
79         smax_v[k] = max(max_v[2 * k + 1], smax_v[2 * k + 2]);
80     } else if (max_v[2 * k + 1] > max_v[2 * k + 2]) {
81         max_v[k] = max_v[2 * k + 1];
82         max_c[k] = max_c[2 * k + 1];
83         smax_v[k] = max(smax_v[2 * k + 1], max_v[2 * k + 2]);
84     } else {
85         max_v[k] = max_v[2 * k + 1];
86         max_c[k] = max_c[2 * k + 1] + max_c[2 * k + 2];
87         smax_v[k] = max(smax_v[2 * k + 1], smax_v[2 * k + 2]);
88     }
89
90     if (min_v[2 * k + 1] < min_v[2 * k + 2]) {
91         min_v[k] = min_v[2 * k + 1];
92         min_c[k] = min_c[2 * k + 1];
93         smin_v[k] = min(smin_v[2 * k + 1], min_v[2 * k + 2]);
94     } else if (min_v[2 * k + 1] > min_v[2 * k + 2]) {
95         min_v[k] = min_v[2 * k + 2];
96         min_c[k] = min_c[2 * k + 2];
97         smin_v[k] = min(min_v[2 * k + 1], smin_v[2 * k + 2]);
98     } else {
99         min_v[k] = min_v[2 * k + 1];

```

```

100     min_c[k] = min_c[2 * k + 1] + min_c[2 * k + 2];
101     smin_v[k] = min(smin_v[2 * k + 1], smin_v[2 * k + 2]);
102 }
103 }
104 void _update_min(ll x, int a, int b, int k, int l, int r) {
105     if (b <= 1 || r <= a || max_v[k] <= x) {
106         return;
107     }
108     if (a <= 1 && r <= b && smax_v[k] < x) {
109         update_node_max(k, x);
110         return;
111     }
112     push(k);
113     _update_min(x, a, b, 2 * k + 1, l, (l + r) / 2);
114     _update_min(x, a, b, 2 * k + 2, (l + r) / 2, r);
115     update(k);
116 }
117 void _update_max(ll x, int a, int b, int k, int l, int r) {
118     if (b <= 1 || r <= a || x <= min_v[k]) {
119         return;
120     }
121     if (a <= 1 && r <= b && x < smin_v[k]) {
122         update_node_min(k, x);
123         return;
124     }
125     push(k);
126     _update_max(x, a, b, 2 * k + 1, l, (l + r) / 2);
127     _update_max(x, a, b, 2 * k + 2, (l + r) / 2, r);
128     update(k);
129 }
130 void addall(int k, ll x) {
131     max_v[k] += x;
132     if (smax_v[k] != -inf) smax_v[k] += x;
133     min_v[k] += x;
134     if (smin_v[k] != inf) smin_v[k] += x;
135
136     sum[k] += len[k] * x;
137     if (lval[k] != inf) {
138         lval[k] += x;
139     } else {
140         ladd[k] += x;
141     }
142 }
143 void updateall(int k, ll x) {
144     max_v[k] = x; smax_v[k] = -inf;
145     min_v[k] = x; smin_v[k] = inf;
146     max_c[k] = min_c[k] = len[k];
147
148     sum[k] = x * len[k];
149     lval[k] = x; ladd[k] = 0;
150 }
151 void _add_val(ll x, int a, int b, int k, int l, int r) {
152     if (b <= 1 || r <= a) {
153         return;
154     }
155     if (a <= 1 && r <= b) {
156         addall(k, x);
157         return;
158     }
159     push(k);
160     _add_val(x, a, b, 2 * k + 1, l, (l + r) / 2);
161     _add_val(x, a, b, 2 * k + 2, (l + r) / 2, r);
162     update(k);
163 }
164 void _update_val(ll x, int a, int b, int k, int l, int r) {
165     if (b <= 1 || r <= a) {
166         return;
167     }

```

```

168     if (a <= 1 && r <= b) {
169         updateall(k, x);
170         return;
171     }
172     push(k);
173     _update_val(x, a, b, 2 * k + 1, 1, (1 + r) / 2);
174     _update_val(x, a, b, 2 * k + 2, (1 + r) / 2, r);
175     update(k);
176 }
177
178 ll _query_max(int a, int b, int k, int l, int r) {
179     if (b <= 1 || r <= a) {
180         return -inf;
181     }
182     if (a <= 1 && r <= b) {
183         return max_v[k];
184     }
185     push(k);
186     ll lv = _query_max(a, b, 2 * k + 1, 1, (1 + r) / 2);
187     ll rv = _query_max(a, b, 2 * k + 2, (1 + r) / 2, r);
188     return max(lv, rv);
189 }
190
191 ll _query_min(int a, int b, int k, int l, int r) {
192     if (b <= 1 || r <= a) {
193         return inf;
194     }
195     if (a <= 1 && r <= b) {
196         return min_v[k];
197     }
198     push(k);
199     ll lv = _query_min(a, b, 2 * k + 1, 1, (1 + r) / 2);
200     ll rv = _query_min(a, b, 2 * k + 2, (1 + r) / 2, r);
201     return min(lv, rv);
202 }
203
204 ll _query_sum(int a, int b, int k, int l, int r) {
205     if (b <= 1 || r <= a) {
206         return 0;
207     }
208     if (a <= 1 && r <= b) {
209         return sum[k];
210     }
211     push(k);
212     ll lv = _query_sum(a, b, 2 * k + 1, 1, (1 + r) / 2);
213     ll rv = _query_sum(a, b, 2 * k + 2, (1 + r) / 2, r);
214     return lv + rv;
215 }
216
217 SGTBeats(int n) {
218     SGTBeats(n, nullptr);
219 }
220
221 SGTBeats(int n, ll *a) : n(n) {
222     n0 = 1;
223     while (n0 < n) n0 <<= 1;
224     for (int i = 0; i < 2 * n0; ++i) ladd[i] = 0, lval[i] = inf;
225     len[0] = n0;
226     for (int i = 0; i < n0 - 1; ++i) len[2 * i + 1] = len[2 * i + 2] = (len[i] >> 1);
227
228     for (int i = 0; i < n; ++i) {
229         max_v[n0 - 1 + i] = min_v[n0 - 1 + i] = sum[n0 - 1 + i] = (a != nullptr ? a[i] : 0);
230         smax_v[n0 - 1 + i] = -inf;
231         smin_v[n0 - 1 + i] = inf;
232         max_c[n0 - 1 + i] = min_c[n0 - 1 + i] = 1;
233     }
234     for (int i = n; i < n0; ++i) {
235         max_v[n0 - 1 + i] = smax_v[n0 - 1 + i] = -inf;

```

```

236     min_v[n0 - 1 + i] = smin_v[n0 - 1 + i] = inf;
237     max_c[n0 - 1 + i] = min_c[n0 - 1 + i] = 0;
238 }
239 for (int i = n0 - 2; i >= 0; i--) {
240     update(i);
241 }
242 }
243
244 // all queries are performed on [l, r) segment
245
246 // range minimize query
247 void update_min(int a, int b, ll x) {
248     _update_min(x, a, b, 0, 0, n0);
249 }
250 // range maximize query
251 void update_max(int a, int b, ll x) {
252     _update_max(x, a, b, 0, 0, n0);
253 }
254 // range add query
255 void add_val(int a, int b, ll x) {
256     _add_val(x, a, b, 0, 0, n0);
257 }
258 // range update query
259 void update_val(int a, int b, ll x) {
260     _update_val(x, a, b, 0, 0, n0);
261 }
262 // range minimum query
263 ll query_max(int a, int b) {
264     return _query_max(a, b, 0, 0, n0);
265 }
266 // range maximum query
267 ll query_min(int a, int b) {
268     return _query_min(a, b, 0, 0, n0);
269 }
270 // range sum query
271 ll query_sum(int a, int b) {
272     return _query_sum(a, b, 0, 0, n0);
273 }
274 };
275
276 ll a[N];
277 int32_t main() {
278     ios_base::sync_with_stdio(0);
279     cin.tie(0);
280     int n, q; cin >> n >> q;
281     for (int i = 0; i < n; i++) {
282         cin >> a[i];
283     }
284     SGTBeats t(n, a);
285     while (q--) {
286         int ty, l, r; cin >> ty >> l >> r;
287         ll x; if (ty < 3) cin >> x;
288         if (ty == 0) {
289             t.update_min(l, r, x);
290         }
291         else if (ty == 1) {
292             t.update_max(l, r, x);
293         }
294         else if (ty == 2) {
295             t.add_val(l, r, x);
296         }
297         else {
298             cout << t.query_sum(l, r) << '\n';
299         }
300     }
301     return 0;
302 }
303 // https://judge.yosupo.jp/problem/range_chmin_chmax_add_range_sum

```

3.19 Segment Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template<typename T>
5 struct segtree {
6     int sz = 1;
7
8     T neutral;
9
10    vector<T> t;
11    function<T(T, T)> combine;
12
13    segtree(int n, T init, function<T(T, T)> com) {
14        while (sz < n) sz += sz;
15        combine = com;
16        neutral = init;
17        t.resize(2 * sz);
18    }
19
20    void pull(int x) {
21        t[x] = combine(t[2 * x], t[2 * x + 1]);
22    }
23
24    void build(int x, int lx, int rx, vector<T> &v) {
25        if (lx == rx) {
26            //TODO 1-based
27            if (lx < (int) v.size()) {
28                t[x] = v[lx];
29            }
30            return;
31        }
32        int m = (lx + rx) >> 1;
33        build(2 * x, lx, m, v);
34        build(2 * x + 1, m + 1, rx, v);
35        pull(x);
36    }
37
38    void build(vector<T> &v) {
39        build(1, 1, sz, v);
40    }
41
42    void set(int lx, int rx, int x, int idx, T val) {
43        if (lx == rx) {
44            // TODO
45            t[x] = val;
46            return;
47        }
48        int m = (lx + rx) >> 1;
49        if (idx <= m) {
50            set(lx, m, 2 * x, idx, val);
51        } else {
52            set(m + 1, rx, 2 * x + 1, idx, val);
53        }
54        t[x] = combine(t[2 * x], t[2 * x + 1]);
55    }
56
57    void set(int idx, T val) {
58        set(1, sz, 1, idx, val);
59    }
60
61    T query(int x, int lx, int rx, int l, int r) {
62        if (rx < l || lx > r) return neutral;
63        if (lx >= l && rx <= r) return t[x];
64        int m = (lx + rx) >> 1;
65        return combine(query(2 * x, lx, m, l, r), query(2 * x + 1, m + 1, rx, l, r));
66    }

```

```

67
68    T query(int l, int r) {
69        return query(1, 1, sz, l, r);
70    }
71 };
72
73 int main() {
74     segtree<long long> seg(n + 1, 0, [](long long a, long long b) {
75         return a + b;
76     });
77
78     return 0;
79 }

```

3.20 Segtree Tree Iterative

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template<typename T>
5
6 struct segtree {
7     int sz = 1;
8
9     // TODO
10    T neutral = 0;
11
12    segtree() {
13    }
14
15    vector<T> t;
16
17    segtree(int n, T init) {
18        sz = n;
19        t.resize(2 * sz);
20        neutral = init;
21    }
22
23    T combine(T a, T b) {
24        // TODO
25        return a + b;
26    }
27
28    void build(vector<T> &v) {
29        assert(sz == (int)v.size());
30        for (int i = 0; i < sz; i++) t[sz + i] = v[i];
31        for (int i = sz - 1; i >= 0; i--) {
32            t[i] = combine(t[2 * i], t[2 * i + 1]);
33        }
34    }
35
36    void update(int idx, T val) {
37        assert(idx >= 1 && idx <= sz);
38        t[idx + sz - 1] += val;
39        for (idx += sz - 1; idx > 1; idx >>= 1) {
40            t[idx >> 1] = combine(t[idx], t[idx ^ 1]);
41        }
42    }
43
44    T query(int l, int r) {
45        T ret = neutral;
46        for (l += sz - 1, r += sz - 1; l <= r; l >>= 1, r >>= 1) {
47            if (l & 1) ret = combine(ret, t[l++]);
48            if (~r & 1) ret = combine(ret, t[r--]);
49        }
50        return ret;
51    }

```

52 };

3.21 Sparse Segment Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class SparseSegtree {
5 private:
6     struct Node {
7         long long freqmn = 0, freqmx = 0, lazy = 0;
8         Node *left = nullptr;
9         Node *right = nullptr;
10    };
11
12    Node *root = new Node;
13    const long long n;
14
15    long long combine_min(long long a, long long b) {
16        return min(a, b);
17    }
18
19    long long combine_max(long long a, long long b) {
20        return max(a, b);
21    }
22
23    void apply(Node *cur, long long val) {
24        (cur->lazy) += val;
25        (cur->freqmn) += val;
26        (cur->freqmx) += val;
27    }
28
29    void push_down(Node *cur, long long l, long long r) {
30        if ((cur->left) == nullptr) {
31            (cur->left) = new Node;
32        }
33
34        if ((cur->right) == nullptr) {
35            (cur->right) = new Node;
36        }
37
38        long long m = (l + r) / 2;
39        apply(cur->left, cur->lazy);
40        apply(cur->right, cur->lazy);
41        cur->lazy = 0;
42    }
43
44    void range_set(Node *cur, long long l, long long r, long long ql, long long qr, long long
    val) {
45        if (qr < l || ql > r) { return; }
46        if (ql <= l && r <= qr) {
47            apply(cur, val);
48        } else {
49            push_down(cur, l, r);
50            long long m = (l + r) / 2;
51            range_set(cur->left, l, m, ql, qr, val);
52            range_set(cur->right, m + 1, r, ql, qr, val);
53            (cur->freqmn) = combine_min((cur->left)->freqmn, (cur->right)->freqmn);
54            (cur->freqmx) = combine_max((cur->left)->freqmx, (cur->right)->freqmx);
55        }
56    }
57
58    pair<long long, long long> range_sum(Node *cur, long long l, long long r, long long ql, long
    long qr) {
59        if (qr < l || ql > r) {
60            return make_pair(2e18, -2e18);
61        }

```

```

62        if (ql <= l && r <= qr) {
63            return make_pair(cur->freqmn, cur->freqmx);
64        }
65        push_down(cur, l, r);
66        long long m = (l + r) / 2;
67        auto n1 = range_sum(cur->left, l, m, ql, qr);
68        auto n2 = range_sum(cur->right, m + 1, r, ql, qr);
69        return {combine_min(n1.first, n2.first), combine_max(n1.second, n2.second)};
70    }
71
72 public:
73     SparseSegtree(long long n) : n(n) {
74     }
75
76     void range_set(long long ql, long long qr, long long val) {
77         range_set(root, 0, n - 1, ql, qr, val);
78     }
79
80     pair<long long, long long> range_sum(long long ql, long long qr) {
81         return range_sum(root, 0, n - 1, ql, qr);
82     }
83 };
84
85
86 int main() {
87     ios::sync_with_stdio(false);
88     cin.tie(nullptr);
89
90     const long long RANGE_SIZE = 2e18;
91     SparseSegtree seg(RANGE_SIZE + 1);
92 }

```

3.22 Sparse_Table

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template<typename T>
5
6 struct sparseTable {
7     int sz, lg;
8     vector<vector<T>> > t;
9     function<T(T, T)> combine;
10
11     sparseTable(int n, function<T(T, T)> F) {
12         sz = n;
13         lg = __lg(n);
14         combine = F;
15         t.resize(lg + 1, vector<T>(n + 1));
16     }
17
18     void build(vector<T> &v) {
19         for (int i = 0; i < sz; i++) {
20             // TODO 0-based
21             t[0][i] = v[i];
22         }
23         for (int p = 1; p <= lg; p++) {
24             for (int i = 0; i + (1 << p) <= sz; i++) {
25                 t[p][i] = combine(t[p - 1][i], t[p - 1][i + (1 << (p - 1))]);
26             }
27         }
28     }
29
30     // TODO 0-based
31     T query(int l, int r) {
32         assert(l <= r);
33         int k = 31 - __builtin_clz(r - l + 1);

```

```

34     return combine(t[k][l], t[k][r - (1 << k) + 1]);
35 }
36
37 T queryLog(int l, int r, T neutral) {
38     assert(l <= r);
39     T ret = neutral;
40     while (l <= r) {
41         int k = 31 - __builtin_clz(r - l + 1);
42         ret = combine(ret, t[k][l]);
43         l += (1 << k);
44     }
45     return ret;
46 }
47 };
48
49 int main() {
50
51     sparseTable<long long> sp(n, [](long long a, long long b) {
52         // TODO
53         return min(a, b);
54     });
55
56     vector<sparseTable<int>> spMn;
57     for (int i = 0; i <= 26; i++) {
58         spMn.emplace_back(sparseTable<int>(n + 1, [](int a, int b) {
59             return min(a, b);
60         }));
61         spMn.back().build(len[i]);
62     }
63 }
64

```

3.23 SQRT

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Sqrt {
5     int block_size;
6     vector<int> nums;
7     vector<long long> blocks;
8
9     Sqrt(int sqtrn, vector<int> &arr) : block_size(sqtrn), blocks(sqtrn, 0) {
10         nums = arr;
11         //TODO 0-Based
12         for (int i = 0; i < nums.size(); i++) {
13             //TODO
14             blocks[i / block_size] += nums[i];
15         }
16     }
17
18     // O(1) update to set nums[i] to v
19     void update(int i, int v) {
20         //TODO
21         blocks[i / block_size] -= nums[i];
22         nums[i] = v;
23         blocks[i / block_size] += nums[i];
24     }
25
26     // O(sqrt(n)) query for sum of [0, r)
27     long long query(int r) {
28         //TODO
29         long long ret = 0;
30         for (int i = 0; i < r / block_size; i++) {
31             ret += blocks[i];
32         }
33         for (int i = (r / block_size) * block_size; i < r; i++) {

```

```

34         ret += nums[i];
35     }
36     return ret;
37 }
38
39 // O(sqrt(n)) query for sum of [l, r)
40 long long query(int l, int r) {
41     return query(r) - query(l - 1);
42 }
43 };
44
45 int main() {
46     Sqrt sq((int) ceil(sqrt(n)), arr);
47
48     // l, r 1-Based
49     sq.update(l - 1, r);
50     cout << sq.query(l, r);
51 }

```

3.24 Treap persistent

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 const int N = 5e4 + 9;
5
6 //Implicit Persistent Treap
7 //0-indexed
8 mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
9 struct node {
10     node *l, *r;
11     int val, prior, sz;
12     node() {}
13     node(int id) {
14         l = r = nullptr;
15         val = id;
16         prior = rnd();
17         sz = 1;
18     }
19 };
20 namespace PersistentTreap {
21 inline void copy(node* &a, node* b) {
22     if (!b) a = nullptr;
23     else {
24         a = new node();
25         *a = *b;
26     }
27 }
28 int size(node* t) {
29     return t ? t->sz : 0;
30 }
31 void update_size(node* &t) {
32     if (t) t->sz = size(t->l) + size(t->r) + 1;
33 }
34 void split(node *cur, int pos, node *l, node *r, int add = 0) {
35     node* t;
36     copy(t, cur);
37     if (t == nullptr) {
38         l = r = nullptr;
39         return;
40     }
41     int id = add + size(t -> l);
42     if (id <= pos) {
43         split(t->r, pos, l, r, id + 1);
44         t->r = l;
45         l = t;
46     } else {

```

```

47     split(t->l, pos, l, r, add);
48     t->l = r;
49     r = t;
50 }
51 update_size(t);
52 }
53 node* merge(node *l, node *r) {
54     node* t;
55     if (!l || !r) copy(t, l ? l : r);
56     else if (l->prior < r->prior) {
57         copy(t, l);
58         t->r = merge(t->r, r);
59     } else {
60         copy(t, r);
61         t->l = merge(l, t->l);
62     }
63     update_size(t);
64     return t;
65 }
66 node* insert(node* t, int pos, int val) {
67     node *l, *r;
68     split(t, pos - 1, l, r);
69     return merge(merge(l, new node(val)), r);
70 }
71 node* insert(node* t, int pos, node* x) {
72     node *l, *r;
73     split(t, pos - 1, l, r);
74     return merge(merge(l, x), r);
75 }
76 node* erase(node* t, int L, int R) {
77     node *l, *r, *mid, *mr;
78     split(t, L - 1, l, r);
79     split(r, R - L, mid, mr);
80     return merge(l, mr);
81 }
82 //creating a treap for string s in O(n)
83 node* build(int l, int r, string &s) {
84     int mid = l + r >> 1;
85     node* cur = new node(s[mid]);
86     if (l < mid) cur->l = build(l, mid - 1, s);
87     if (mid < r) cur->r = build(mid + 1, r, s);
88     update_size(cur);
89     return cur;
90 }
91 int c = 0;
92 void print_nodes(node* t) {
93     if (!t) return;
94     print_nodes(t->l);
95     char ch = t->val;
96     c += ch == 'c';
97     cout << ch;
98     print_nodes(t->r);
99 }
100 void print(node* t, int L, int R) {
101     node *l, *r, *mid, *mr;
102     split(t, L - 1, l, r);
103     split(r, R - L, mid, mr);
104     print_nodes(mid);
105 }
106 }
107 using namespace PersistentTreap;
108 node* root[N];
109 int32_t main() {
110     ios_base::sync_with_stdio(0);
111     cin.tie(0);
112     int version = 0;
113     int q;
114     cin >> q;

```

```

115 while (q --) {
116     int ty;
117     cin >> ty;
118     if (ty == 1) {
119         int k;
120         string s;
121         cin >> k >> s;
122         k -= c;
123         node* t = build(0, (int)s.size() - 1, s);
124         root[version + 1] = insert(root[version], k, t);
125         version++;
126     } else if (ty == 2) {
127         int l, len;
128         cin >> l >> len;
129         l -= c, len -= c;
130         --l;
131         root[version + 1] = erase(root[version], l, l + len - 1);
132         version++;
133     } else {
134         int v, l, len;
135         cin >> v >> l >> len;
136         v -= c, l -= c, len -= c;
137         --l;
138         print(root[v], l, l + len - 1);
139         cout << '\n';
140     }
141 }
142 return 0;
143 }
144 //https://vjudge.net/problem/UVA-12538

```

4 DP Optimizations

4.1 Convex Hull Trick

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 #define ll long long
5 #define eb emplace_back
6 #define nl '\n'
7 #define deb(x) cerr << #x" = " << x << nl
8 #define in() ( { int a ; scanf("%d", &a); a; } )
9
10 const int N = 3e5 + 9;
11 const int mod = 1e9 + 7;
12
13 struct CHT {
14     vector<ll> m, b;
15     int ptr = 0;
16
17     bool bad(int l1, int l2, int l3) {
18         return 1.0 * (b[l3] - b[l1]) * (m[l1] - m[l2]) <= 1.0 * (b[l2] - b[l1]) * (m[l1] - m[l3]
19         ); //(slope dec+query min), (slope inc+query max)
20         return 1.0 * (b[l3] - b[l1]) * (m[l1] - m[l2]) > 1.0 * (b[l2] - b[l1]) * (m[l1] - m[l3]
21         ); //(slope dec+query max), (slope inc+query min)
22     }
23
24     void add(ll _m, ll _b) {
25         m.push_back(_m);
26         b.push_back(_b);
27         int s = m.size();
28         while(s >= 3 && bad(s - 3, s - 2, s - 1)) {
29             s--;
30             m.erase(m.end() - 2);
31             b.erase(b.end() - 2);
32         }
33     }
34 }

```

```

31     }
32
33     ll f(int i, ll x) {
34         return m[i] * x + b[i];
35     }
36
37      //(slope dec+query min), (slope inc+query max) -> x increasing
38      //(slope dec+query max), (slope inc+query min) -> x decreasing
39     ll query(ll x) {
40         if(ptr >= m.size()) ptr = m.size() - 1;
41         while(ptr < m.size() - 1 && f(ptr + 1, x) < f(ptr, x)) ptr++;
42         return f(ptr, x);
43     }
44
45     ll bs(int l, int r, ll x) {
46         int mid = (l + r) / 2;
47         if(mid + 1 < m.size() && f(mid + 1, x) < f(mid, x)) return bs(mid + 1, r, x); // > for
48         max
49         if(mid - 1 >= 0 && f(mid - 1, x) < f(mid, x)) return bs(l, mid - 1, x); // > for max
50         return f(mid, x);
51     }
52 };
53
54 ll a[N], b[N];
55 CHT cht;
56 int32_t main() {
57     ios_base::sync_with_stdio(0);
58     cin.tie(0);
59
60     int n;
61     cin >> n;
62     for(int i = 0; i < n; i++) cin >> a[i];
63     for(int i = 0; i < n; i++) cin >> b[i];
64     cht.add(b[0], 0);
65     ll ans = 0;
66     for(int i = 1; i < n; i++) {
67         ans = cht.query(a[i]);
68         cht.add(b[i], ans);
69     }
70     cout << ans << nl;
71     return 0;
72 }

```

```

21
22 #define __builtin_popcountl __builtin_popcountll
23 #define __builtin_ctzl __builtin_ctzll
24
25 #include <bitset>
26
27 #pragma pop_macro("__cplusplus")
28 #pragma pop_macro("__SIZEOF_LONG__")
29 #undef unsigned
30 #undef __builtin_popcountl
31 #undef __builtin_ctzl
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     int n;
38     cin >> n;
39     vector<int> v(n);
40     for (int i = 0; i < n; i++) {
41         cin >> v[i];
42     }
43
44     bitset<100005> dp;
45     dp[0] = 1;
46     for (auto &it: v) {
47         dp |= (dp << it);
48     }
49
50     vector<int> ans;
51     for (int i = 1; i <= 100005; i++) {
52         if (dp[i]) {
53             ans.push_back(i);
54         }
55     }
56
57     cout << ans.size() << '\n';
58     for (auto &it: ans) {
59         cout << it << ' ';
60     }
61
62     return 0;
63 }

```

5 Fast

5.1 dp bitset

```

1 // https://cses.fi/problemset/task/1745
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 #pragma GCC optimize("O3")
7 #pragma GCC optimize("unroll-loops")
8 #pragma GCC optimize("Ofast")
9
10 #include <string>
11 #include <bits/functional_hash.h>
12 #include <iosfwd>
13 #include <bits/cxxabi_forced.h>
14 #include <bits/functional_hash.h>
15
16 #pragma push_macro("__SIZEOF_LONG__")
17 #pragma push_macro("__cplusplus")
18 #define __SIZEOF_LONG__ __SIZEOF_LONG_LONG__
19 #define unsigned unsigned long
20 #define __cplusplus 201102L

```

6 Geometry

6.1 Angle Between Vectors

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T dot(pt v, pt w) {
10     return v.x * w.x + v.y * w.y;
11 }
12
13 double angle(pt v, pt w) {
14     double cosTheta = dot(v, w) / abs(v) / abs(w);
15     return acos(max(-1.0, min(1.0, cosTheta)));
16 }
17
18 int main() {
19     ios::sync_with_stdio(false);

```

```

20     cin.tie(nullptr), cout.tie(nullptr);
21
22     double a, b, c, d;
23     cin >> a >> b >> c >> d;
24
25     cout << fixed << setprecision(9) << angle({a, b}, {c, d}) << '\n';
26
27     return 0;
28 }

```

6.2 Area of Polygon

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef long double T;
5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  const T PI = acos(-1.0);
10
11 T cross(pt v, pt w) {
12     return v.x * w.y - v.y * w.x;
13 }
14
15 T areaPolygon(vector<pt> p) {
16     T area = 0.0;
17     for (int i = 0, n = p.size(); i < n; i++) {
18         area += cross(p[i], p[(i + 1) % n]); // wrap back to 0 if i == n-1
19     }
20     // if area >= 0, the points are ordered counter-clockwise else ordered clockwise
21     return abs(area) / 2.0;
22 }
23
24 // calculates the area of a regular polygon when n is number of sides of the polygon and r is
25 // radius of the circumscribed circle (distance from center to a vertex)
26 T area_polygon(int n, T r) {
27     return 0.5L * n * r * r * sin(2.0L * PI / n);
28 }
29
30 int main() {
31     ios::sync_with_stdio(false);
32     cin.tie(nullptr), cout.tie(nullptr);
33
34     int n;
35     cin >> n;
36
37     vector<pt> p(n);
38     for (int i = 0; i < n; i++) {
39         int a, b;
40         cin >> a >> b;
41         p[i] = pt(a, b);
42     }
43
44     cout << fixed << setprecision(9) << areaPolygon(p) << '\n';
45
46     return 0;
47 }

```

6.3 Area of Triangle

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef long double T;

```

```

5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  T cross(pt v, pt w) {
10     return v.x * w.y - v.y * w.x;
11 }
12
13 // knowing base and perpendicular height
14 // area = 0.5 * base * height
15 T areaTriangle(int b, int h) {
16     return 0.5L * b * h;
17 }
18
19 // Heron's Formula
20 // area = sqrt(s * (s - a) * (s - b) * (s - c))
21 // where s is the semi-perimeter: (a + b + c) / 2
22 T areaTriangle(T a, T b, T c) {
23     T s = (a + b + c) / 2.0;
24     return sqrtl(s * (s - a) * (s - b) * (s - c));
25 }
26
27 // knowing two sides and the included angle
28 // area = 0.5 * a * b * sin(c)
29 T areaTriangle(T a, T b, T c) {
30     return 0.5L * a * b * sin(c);
31 }
32
33 // knowing the 2D coordinates of all three vertices
34 T areaTriangle(pt a, pt b, pt c) {
35     return abs(cross(b - a, c - a)) / 2.0;
36 }
37
38 // given one side (len) and two adjacent angles (alpha, beta)
39 // area = c^2 / (2 * (cot(alpha) + cot(beta)))
40 // cot(x) = 1 / tan(x)
41 // area = (c * c) / (2.0 * (1.0 / tan(alpha) + 1.0 / tan(beta)))
42 T areaTriangle(int c, T alpha, T beta) {
43     return (1.0L * c * c) / (2.0 * (1.0 / tan(alpha) + 1.0 / tan(beta)));
44 }

```

6.4 Bisector

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef double T;
5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 pt perp(pt p) {
14     return {-p.y, p.x};
15 }
16
17 T dot(pt v, pt w) {
18     return v.x * w.x + v.y * w.y;
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24

```

```

25 struct line {
26     pt v;
27     T c;
28
29     // From direction vector v and offset c
30     line(pt v, T c) : v(v), c(c) {
31     }
32
33     // From equation ax + by = c
34     line(T a, T b, T c) : v({b, -a}), c(c) {
35     }
36
37     // From points P and Q
38     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
39     }
40
41     T side(pt p) {
42         return cross(v, p) - c;
43     }
44
45     T dist(pt p) {
46         return abs(side(p)) / abs(v);
47     }
48
49     line perpThrough(pt p) {
50         return {p, p + perp(v)};
51     }
52
53     // Sorting along a line
54     bool cmpProj(pt p, pt q) {
55         return dot(v, p) < dot(v, q);
56     }
57
58     // translate a line l by vector t
59     line translate(pt t) {
60         return {v, c + cross(v, t)};
61     }
62
63     // s shifting line l to the left by a certain distance
64     line shiftLeft(T dist) {
65         return {v, c + dist * abs(v)};
66     }
67
68     pt proj(pt p) {
69         return p - perp(v) * side(p) / sq(v);
70     }
71
72     pt refl(pt p) {
73         return p - perp(v) * (T) 2.0 * side(p) / sq(v);
74     }
75 };
76
77 // An angle bisector of two (non-parallel) lines l1 and l2 is a line that forms equal angles
78 // with l1 and l2.
79 line bisector(line l1, line l2, bool interior) {
80     assert(cross(l1.v, l2.v) != 0);
81     T sign = interior ? 1 : -1; // Internal or Exterior angle bisector
82     return {
83         l2.v / abs(l2.v) + l1.v / abs(l1.v) * sign,
84         l2.c / abs(l2.v) + l1.c / abs(l1.v) * sign
85     };
86
87 int main() {
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr), cout.tie(nullptr);
90
91     double a1, b1, c1, d, e, f;

```

```

92     cin >> a1 >> b1 >> c1 >> d >> e >> f;
93
94     line l1({a1, b1}, {c1, d}), l2({a1, b1}, {e, f});
95     line ret = bisector(l1, l2, true);
96
97     // a * x + b * y - c = 0
98     T a = ret.v.imag(), b = -ret.v.real(), c = ret.c;
99
100     cout << fixed << setprecision(9) << a << ' ' << b << ' ' << c << '\n';
101
102     return 0;
103 }

```

6.5 Center Circle Pass 3 points

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef long double T;
5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  const long double EPS = 1e-9;
10
11 T sq(pt p) {
12     return p.x * p.x + p.y * p.y;
13 }
14
15 T dist(pt a, pt b) {
16     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
17 }
18
19 pt perp(pt p) {
20     return {-p.y, p.x};
21 }
22
23 T cross(pt v, pt w) {
24     return v.x * w.y - v.y * w.x;
25 }
26
27 bool isColinear(pt a, pt b, pt c) {
28     return fabs((b.x - a.x) * (c.y - a.y) - (c.x - a.x) * (b.y - a.y)) < EPS;
29 }
30
31 // you can determine circle with 3 points
32 // return center and radius
33 pair<pt, T> circumCenter(pt a, pt b, pt c) {
34     b = b - a, c = c - a; // consider coordinates relative to A
35     assert(cross(b, c) != 0); // no circumcircle if A,B,C aligned
36     return make_pair(a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / 2.0L,
37         abs(perp(b * sq(c) - c * sq(b)) / cross(b, c) / 2.0L));
38 }

```

6.6 Check if polygon is rectangle and Square

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef int T;
5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  vector<pt> points(9);
10

```

```

11 T dot(pt v, pt w) {
12     return v.x * w.x + v.y * w.y;
13 }
14
15 T sq(pt p) {
16     return p.x * p.x + p.y * p.y;
17 }
18
19 T dist(pt v, pt w) {
20     return (w.x - v.x) * (w.x - v.x) + (w.y - v.y) * (w.y - v.y);
21 }
22
23 bool check_rectangle(pt a, pt b, pt c, pt d) {
24     pt ab = pt(a.x - b.x, a.y - b.y);
25     pt bc = pt(b.x - c.x, b.y - c.y);
26     pt cd = pt(c.x - d.x, c.y - d.y);
27     pt da = pt(d.x - a.x, d.y - a.y);
28
29     if (!sq(ab) || !sq(bc) || !sq(cd) || !sq(da)) {
30         return false;
31     }
32
33     if (!(!dot(ab, bc) && !dot(bc, cd) && !dot(cd, da) && !dot(da, ab))) {
34         return false;
35     }
36
37     return ((sq(ab) == sq(cd)) && (sq(da) == sq(bc)));
38 }
39
40 bool check_square(pt p1, pt p2, pt p3, pt p4) {
41     set<int> st;
42     st.insert(dist(p1, p2));
43     st.insert(dist(p2, p3));
44     st.insert(dist(p3, p4));
45     st.insert(dist(p4, p1));
46
47     return (st.size() == 1 && check_rectangle(p1, p2, p3, p4));
48 }

```

6.7 Dist Between 2 Segment

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 pt perp(pt p) {
14     return {-p.y, p.x};
15 }
16
17 T dot(pt v, pt w) {
18     return v.x * w.x + v.y * w.y;
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25 struct line {
26     pt v;

```

```

27     T c;
28
29     // From direction vector v and offset c
30     line(pt v, T c) : v(v), c(c) {
31     }
32
33     // TODO
34     // From equation ax + by - c = 0
35     line(T a, T b, T c) : v(b, -a), c(-c) {
36     }
37
38     // From points P and Q
39     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
40     }
41
42     T side(pt p) {
43         return cross(v, p) - c;
44     }
45
46     T dist(pt p) {
47         return abs(side(p)) / abs(v);
48     }
49
50     line perpThrough(pt p) {
51         return {p, p + perp(v)};
52     }
53
54     // Sorting along a line
55     bool cmpProj(pt p, pt q) {
56         return dot(v, p) < dot(v, q);
57     }
58
59     // translate a line l by vector t
60     line translate(pt t) {
61         return {v, c + cross(v, t)};
62     }
63
64     // s shifting line l to the left by a certain distance
65     line shiftLeft(T dist) {
66         return {v, c + dist * abs(v)};
67     }
68
69     pt proj(pt p) {
70         return p - perp(v) * side(p) / sq(v);
71     }
72
73     pt refl(pt p) {
74         return p - perp(v) * (T) 2.0 * side(p) / sq(v);
75     }
76 };
77
78 // distance between segment [AB] and point P
79 T segPoint(pt a, pt b, pt p) {
80     if (a != b) {
81         line l(a, b);
82         if (l.cmpProj(a, p) && l.cmpProj(p, b)) {
83             // if closest to projection
84             return l.dist(p); // output distance to line
85         }
86     }
87     return min(abs(p - a), abs(p - b)); // otherwise distance to A or B
88 }
89
90 // determining the direction of the points (right, left, or on a line).
91 T orient(pt a, pt b, pt c) {
92     return cross(b - a, c - a);
93 }
94

```

```

95 // intersection between two segments
96 bool properInter(pt a, pt b, pt c, pt d, pt &out) {
97     T oa = orient(c, d, a), ob = orient(c, d, b), oc = orient(a, b, c), od = orient(a, b, d);
98     // Proper intersection exists if opposite signs
99     if (oa * ob < 0 && oc * od < 0) {
100         out = (a * ob - b * oa) / (ob - oa);
101         return true;
102     }
103     return false;
104 }
105
106 // distance between two segments
107 T segSeg(pt a, pt b, pt c, pt d) {
108     pt dummy;
109     if (properInter(a, b, c, d, dummy)) return 0;
110     return min({
111         segPoint(a, b, c), segPoint(a, b, d),
112         segPoint(c, d, a), segPoint(c, d, b)
113     });
114 }
115
116 int main() {
117     ios::sync_with_stdio(false);
118     cin.tie(nullptr), cout.tie(nullptr);
119
120     long double ax, ay, bx, by, cx, cy, dx, dy;
121     cin >> ax >> ay >> bx >> by >> cx >> cy >> dx >> dy;
122
123     cout << fixed << setprecision(15) << segSeg({ax, ay}, {bx, by}, {cx, cy}, {dx, dy}) << '\n';
124
125     return 0;
126 }

```

6.8 Dist from point to line

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 pt perp(pt p) {
14     return {-p.y, p.x};
15 }
16
17 T dot(pt v, pt w) {
18     return v.x * w.x + v.y * w.y;
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25 struct line {
26     pt v;
27     T c;
28
29     // From direction vector v and offset c
30     line(pt v, T c) : v(v), c(c) {
31     }
32 }

```

```

33 // TODO
34 // From equation ax + by - c = 0
35 line(T a, T b, T c) : v({b, -a}), c(-c) {
36 }
37
38 // From points P and Q
39 line(pt p, pt q) : v(q - p), c(cross(v, p)) {
40 }
41
42 T side(pt p) {
43     return cross(v, p) - c;
44 }
45
46 T dist(pt p) {
47     return abs(side(p)) / abs(v);
48 }
49
50 line perpThrough(pt p) {
51     return {p, p + perp(v)};
52 }
53
54 // Sorting along a line
55 bool cmpProj(pt p, pt q) {
56     return dot(v, p) < dot(v, q);
57 }
58
59 // translate a line l by vector t
60 line translate(pt t) {
61     return {v, c + cross(v, t)};
62 }
63
64 // s shifting line l to the left by a certain distance
65 line shiftLeft(T dist) {
66     return {v, c + dist * abs(v)};
67 }
68
69 pt proj(pt p) {
70     return p - perp(v) * side(p) / sq(v);
71 }
72
73 pt refl(pt p) {
74     return p - perp(v) * (T) 2.0 * side(p) / sq(v);
75 }
76 };
77
78 int main() {
79     ios::sync_with_stdio(false);
80     cin.tie(nullptr), cout.tie(nullptr);
81
82     double x1, y1, a, b, c;
83     cin >> x1 >> y1 >> a >> b >> c;
84
85     line l(a, b, c);
86     cout << fixed << setprecision(9) << l.dist({x1, y1}) << '\n';
87
88     return 0;
89 }

```

6.9 Dist from point to seg

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()

```

```

8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 pt perp(pt p) {
14     return {-p.y, p.x};
15 }
16
17 T dot(pt v, pt w) {
18     return v.x * w.x + v.y * w.y;
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25 struct line {
26     pt v;
27     T c;
28
29     // From direction vector v and offset c
30     line(pt v, T c) : v(v), c(c) {
31     }
32
33     // TODO
34     // From equation ax + by - c = 0
35     line(T a, T b, T c) : v(b, -a), c(-c) {
36     }
37
38     // From points P and Q
39     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
40     }
41
42     T side(pt p) {
43         return cross(v, p) - c;
44     }
45
46     T dist(pt p) {
47         return abs(side(p)) / abs(v);
48     }
49
50     line perpThrough(pt p) {
51         return {p, p + perp(v)};
52     }
53
54     // Sorting along a line
55     bool cmpProj(pt p, pt q) {
56         return dot(v, p) < dot(v, q);
57     }
58
59     // translate a line l by vector t
60     line translate(pt t) {
61         return {v, c + cross(v, t)};
62     }
63
64     // s shifting line l to the left by a certain distance
65     line shiftLeft(T dist) {
66         return {v, c + dist * abs(v)};
67     }
68
69     pt proj(pt p) {
70         return p - perp(v) * side(p) / sq(v);
71     }
72
73     pt refl(pt p) {
74         return p - perp(v) * (T) 2.0 * side(p) / sq(v);
75     }

```

```

76 };
77
78 // distance between segment [AB] and point P
79 T segPoint(pt a, pt b, pt p) {
80     if (a != b) {
81         line l(a, b);
82         if (l.cmpProj(a, p) && l.cmpProj(p, b)) {
83             // if closest to projection
84             return l.dist(p); // output distance to line
85         }
86     }
87     return min(abs(p - a), abs(p - b)); // otherwise distance to A or B
88 }
89
90 // find minimum distance between two moving points
91 // first point moves from a -> b
92 // second point moves from c -> d
93 // and both move at the same time and same speed
94 T segWalkDist(pt a, pt b, pt c, pt d) {
95     pt nb = b - (d - c);
96     return segPoint(a, nb, c);
97 }
98
99 int main() {
100     ios::sync_with_stdio(false);
101     cin.tie(nullptr), cout.tie(nullptr);
102
103     double x1, y1, x2, y2, x3, y3;
104     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;
105
106     cout << fixed << setprecision(9) << segPoint({x2, y2}, {x3, y3}, {x1, y1}) << '\n';
107
108     return 0;
109 }

```

6.10 Distance between 2 polygons

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 T dot(pt v, pt w) {
14     return v.x * w.x + v.y * w.y;
15 }
16
17 pt perp(pt p) {
18     return {-p.y, p.x};
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25 struct line {
26     pt v;
27     T c;
28
29     // From points P and Q
30     line(pt p, pt q) : v(q - p), c(cross(v, p)) {

```

```

31 }
32
33 T side(pt p) {
34     return cross(v, p) - c;
35 }
36
37 T dist(pt p) {
38     return abs(side(p)) / abs(v);
39 }
40
41 // Sorting along a line
42 bool cmpProj(pt p, pt q) {
43     return dot(v, p) < dot(v, q);
44 }
45 };
46
47 void reorder_polygon(vector<pt> &p) {
48     int pos = 0;
49     for (int i = 1; i < p.size(); i++) {
50         if (p[i].y < p[pos].y || (p[i].y == p[pos].y && p[i].x < p[pos].x)) {
51             pos = i;
52         }
53     }
54     rotate(p.begin(), p.begin() + pos, p.end());
55 }
56
57 // computes the Minkowski sum of two convex polygons p and q
58 // p must be counter clockwise
59 vector<pt> minkowski(vector<pt> p, vector<pt> q) {
60     // the first vertex must be the lowest
61     reorder_polygon(p);
62     reorder_polygon(q);
63
64     // we must ensure cyclic indexing
65     p.push_back(p[0]);
66     p.push_back(p[1]);
67     q.push_back(q[0]);
68     q.push_back(q[1]);
69
70     vector<pt> ret;
71     int i = 0, j = 0;
72     while (i < p.size() - 2 || j < q.size() - 2) {
73         ret.push_back(p[i] + q[j]);
74         auto cros = cross(p[i + 1] - p[i], q[j + 1] - q[j]);
75         i += (cros >= 0 && i < p.size() - 2);
76         j += (cros <= 0 && j < q.size() - 2);
77     }
78     return ret;
79 }
80
81 // true if P at least as high as A (blue part)
82 bool above(pt a, pt p) {
83     return p.y >= a.y;
84 }
85
86 // determining the direction of the points (right -1, left 1, or on a line 0).
87 T orient(pt a, pt b, pt c) {
88     return cross(b - a, c - a);
89 }
90
91 // check if [PQ] crosses ray from A
92 bool crossesRay(pt a, pt p, pt q) {
93     return (above(a, q) - above(a, p)) * orient(a, p, q) > 0;
94 }
95
96 // checks if a point P lies on the disk of diameter [AB].
97 bool inDisk(pt a, pt b, pt p) {
98     return dot(a - p, b - p) <= 0;

```

```

99 }
100
101 // check if a point P lies on segment [AB].
102 bool onSegment(pt a, pt b, pt p) {
103     return orient(a, b, p) == 0 && inDisk(a, b, p);
104 }
105
106 // if strict, returns false when A is on the boundary
107 bool inPolygon(vector<pt> p, pt a, bool strict) {
108     if (p.size() < 3) return false;
109     int numCrossings = 0;
110     for (int i = 0, n = p.size(); i < n; i++) {
111         if (onSegment(p[i], p[(i + 1) % n], a))
112             return !strict;
113         numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
114     }
115     return numCrossings & 1;
116 }
117
118 // check if point is inside a convex polygon (O(log n))
119 bool point_in_convex_polygon(pt p, const vector<pt> &poly) {
120     pt origin = poly[0];
121
122     int l = 1, r = (int) poly.size() - 2, ret = 1;
123     while (l <= r) {
124         int md = (l + r) / 2;
125         if (orient(origin, poly[md], p) >= 1) {
126             ret = md;
127             l = md + 1;
128         } else {
129             r = md - 1;
130         }
131     }
132     return inPolygon({origin, poly[ret], poly[ret + 1]}, p, false);
133 }
134
135 // distance between segment [AB] and point P
136 T segPoint(pt p, pt a, pt b) {
137     if (a != b) {
138         line l(a, b);
139         if (l.cmpProj(a, p) && l.cmpProj(p, b)) {
140             // if closest to projection
141             return l.dist(p); // output distance to line
142         }
143     }
144     return min(abs(p - a), abs(p - b)); // otherwise distance to A or B
145 }
146
147 // distance between two polygons (or simply checking whether they intersect)
148 pair<bool, T> polygons_intersection_distance(vector<pt> p, vector<pt> q) {
149     // reflect q for Minkowski method
150     // any point in Minkowski become ((x - xq), (y - yq))
151     // dist is sqrt((x - xq)^2 + (y - yq)^2)
152     // min dist is min(sqrt(x^2, y^2)) from Minkowski Sum
153     vector<pt> negq = q;
154     for (auto &v: negq) {
155         v = {-v.x, -v.y};
156     }
157
158     // compute Minkowski Sum
159     vector<pt> sum_poly = minkowski(p, negq);
160
161     // check if origin (0,0) is inside sumPoly
162     bool intersect = point_in_convex_polygon({0, 0}, sum_poly);
163     if (intersect) {
164         return make_pair(true, 0.0);
165     }
166 }

```

```

167 // if not intersecting, compute minimum distance to origin
168 T mn_dist = 1e18;
169 for (int i = 0; i < sum_poly.size(); i++) {
170     pt a = sum_poly[i];
171     pt b = sum_poly[(i + 1) % sum_poly.size()];
172     mn_dist = min(mn_dist, segPoint({0, 0}, a, b));
173 }
174 return make_pair(false, mn_dist);
175 }
176
177 int main() {
178     vector<pt> p = {{0, 0}, {2, 0}, {2, 2}, {0, 2}}; // square
179     vector<pt> q = {{3, 1}, {4, 0}, {4, 2}}; // triangle
180
181     auto [intersect, dist] = polygons_intersection_distance(p, q);
182
183     if (intersect) {
184         cout << "Polygons intersect\n";
185     } else {
186         cout << "Polygons do NOT intersect, distance " << dist << "\n";
187     }
188 }

```

6.11 Distance between rays

```

1 //
2 // Created by SHAHD on 5/16/2025.
3 //

```

6.12 Equation Line by point and vector

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr), cout.tie(nullptr);
12
13     long double xp, yp, vx, vy;
14     cin >> xp >> yp >> vx >> vy;
15
16     cout << fixed << setprecision(15) << vx << ' ' << vy << ' ' << -(vx * xp + vy * yp) << '\n';
17
18     return 0;
19 }

```

6.13 Equation Line passing through 2 points

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12

```

```

13 pt perp(pt p) {
14     return {-p.y, p.x};
15 }
16
17 T dot(pt v, pt w) {
18     return v.x * w.x + v.y * w.y;
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25 struct line {
26     pt v;
27     T c;
28
29     // From direction vector v and offset c
30     line(pt v, T c) : v(v), c(c) {}
31
32     // TODO
33     // From equation ax + by - c = 0
34     line(T a, T b, T c) : v({b, -a}), c(-c) {}
35
36     // From points P and Q
37     line(pt p, pt q) : v(q - p), c(cross(v, p)) {}
38
39     T side(pt p) {
40         return cross(v, p) - c;
41     }
42
43     T dist(pt p) {
44         return abs(side(p)) / abs(v);
45     }
46
47     line perpThrough(pt p) {
48         return {p, p + perp(v)};
49     }
50
51     // Sorting along a line
52     bool cmpProj(pt p, pt q) {
53         return dot(v, p) < dot(v, q);
54     }
55
56     // translate a line l by vector t
57     line translate(pt t) {
58         return {v, c + cross(v, t)};
59     }
60
61     // s shifting line l to the left by a certain distance
62     line shiftLeft(T dist) {
63         return {v, c + dist * abs(v)};
64     }
65
66     pt proj(pt p) {
67         return p - perp(v) * side(p) / sq(v);
68     }
69
70     pt refl(pt p) {
71         return p - perp(v) * (T) 2.0 * side(p) / sq(v);
72     }
73 };
74
75 int main() {
76     ios::sync_with_stdio(false);
77     cin.tie(nullptr), cout.tie(nullptr);
78
79 }
80

```

```

81     long double x1, y1, x2, y2;
82     cin >> x1 >> y1 >> x2 >> y2;
83
84     line l({x1, y1}, {x2, y2});
85
86
87     cout << fixed << setprecision(15) << l.v.imag() << ' ' << -l.v.real() << ' ' << l.c << '\n';
88
89     return 0;
90 }

```

6.14 Finding common tangents to two circles

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef double T;
5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16
17 T sq(pt p) {
18     return p.x * p.x + p.y * p.y;
19 }
20
21 T dot(pt v, pt w) {
22     return v.x * w.x + v.y * w.y;
23 }
24
25 pt perp(pt p) {
26     return {-p.y, p.x};
27 }
28
29 T cross(pt v, pt w) {
30     return v.x * w.y - v.y * w.x;
31 }
32
33 struct line {
34     pt v;
35     T c;
36
37     // From direction vector v and offset c
38     line(pt v, T c) : v(v), c(c) {}
39 }
40
41 // From equation ax + by = c
42 line(T a, T b, T c) : v(b, -a), c(c) {}
43 }
44
45 // From points P and Q
46 line(pt p, pt q) : v(q - p), c(cross(v, p)) {}
47 }
48
49 T side(pt p) {
50     return cross(v, p) - c;
51 }
52
53 T dist(pt p) {
54     return abs(side(p)) / abs(v);

```

```

55 }
56
57 T sqDist(pt p) {
58     return side(p) * side(p) / (double) sq(v);
59 }
60
61 line perpThrough(pt p) {
62     return {p, p + perp(v)};
63 }
64
65 // Sorting along a line
66 bool cmpProj(pt p, pt q) {
67     return dot(v, p) < dot(v, q);
68 }
69
70 // translate a line l by vector t
71 line translate(pt t) {
72     return {v, c + cross(v, t)};
73 }
74
75 // s shifting line l to the left by a certain distance
76 line shiftLeft(T dist) {
77     return {v, c + dist * abs(v)};
78 }
79
80 pt proj(pt p) {
81     return p - perp(v) * side(p) / sq(v);
82 }
83
84 pt refl(pt p) {
85     return p - perp(v) * (T) 2.0 * side(p) / sq(v);
86 }
87 };
88
89 // finds 1 tangent between two circles assuming:
90 // first circle is centered at (0, 0) with radius r1
91 // second circle is centered at c with radius r2
92 void tangents(pt c, T r1, T r2, vector<line> &ans) {
93     T r = r2 - r1, z = sq(c), d = z - r * r;
94     if (d < -EPS || z < EPS) return;
95     d = sqrt(max((T) 0.0, d));
96
97     // compute coefficients (a, b, c) of line: ax + by = c
98     T a = (c.x * r + c.y * d) / z;
99     T b = (c.y * r - c.x * d) / z;
100     T cc = r1;
101
102     // convert (a, b, c) to line structure
103     ans.emplace_back(a, b, cc);
104 }
105
106 // finds all common tangents between two circles
107 vector<line> tangents(pt ca, T ra, pt cb, T rb) {
108     vector<line> ans;
109
110     pt c = cb - ca;
111     for (int i = -1; i <= 1; i += 2) {
112         for (int j = -1; j <= 1; j += 2) {
113             tangents(c, ra * i, rb * j, ans);
114         }
115     }
116
117     for (line &l: ans) {
118         l.c -= cross(l.v, ca);
119     }
120     return ans;
121 }
122 }

```

6.15 Intersec Between 2 line

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 pt perp(pt p) {
14     return {-p.y, p.x};
15 }
16
17 T dot(pt v, pt w) {
18     return v.x * w.x + v.y * w.y;
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25 struct line {
26     pt v;
27     T c;
28
29     // From direction vector v and offset c
30     line(pt v, T c) : v(v), c(c) {
31     }
32
33     // TODO
34     // From equation ax + by - c = 0
35     line(T a, T b, T c) : v(b, -a), c(-c) {
36     }
37
38     // From points P and Q
39     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
40     }
41
42     T side(pt p) {
43         return cross(v, p) - c;
44     }
45
46     T dist(pt p) {
47         return abs(side(p)) / abs(v);
48     }
49
50     line perpThrough(pt p) {
51         return {p, p + perp(v)};
52     }
53
54     // Sorting along a line
55     bool cmpProj(pt p, pt q) {
56         return dot(v, p) < dot(v, q);
57     }
58
59     // translate a line l by vector t
60     line translate(pt t) {
61         return {v, c + cross(v, t)};
62     }
63
64     // s shifting line l to the left by a certain distance
65     line shiftLeft(T dist) {
66         return {v, c + dist * abs(v)};

```

```

67     }
68
69     pt proj(pt p) {
70         return p - perp(v) * side(p) / sq(v);
71     }
72
73     pt refl(pt p) {
74         return p - perp(v) * (T) 2.0 * side(p) / sq(v);
75     }
76 };
77
78 // intersection point between two lines l1 and l2
79 bool inter(line l1, line l2, pt &out) {
80     T d = cross(l1.v, l2.v);
81     if (d == 0) return false;
82     out = (l2.v * l1.c - l1.v * l2.c) / d;
83     return true;
84 }
85
86 int main() {
87     ios::sync_with_stdio(false);
88     cin.tie(nullptr), cout.tie(nullptr);
89
90     long double a1, b1, c1, a2, b2, c2;
91     cin >> a1 >> b1 >> c1 >> a2 >> b2 >> c2;
92
93     pt out;
94     bool ok = inter(line(a1, b1, c1), line(a2, b2, c2), out);
95
96     cout << fixed << setprecision(15) << out.real() << ' ' << out.imag() << '\n';
97
98     return 0;
99 }

```

6.16 Intersection between 2 Circle

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16
17 T sq(pt p) {
18     return p.x * p.x + p.y * p.y;
19 }
20
21 pt perp(pt p) {
22     return {-p.y, p.x};
23 }
24
25 // find intersection points of two circles
26 int circleCircle(pt o1, T r1, pt o2, T r2, pair<pt, pt> &out) {
27     pt d = o2 - o1;
28     T d2 = sq(d);
29     if (!sgn(d2)) {
30         // concentric circles
31         assert(r1 != r2);

```

```

32     return 0;
33 }
34
35 T pd = (d2 + r1 * r1 - r2 * r2) / 2; // = |O_1P| * d
36 T h2 = r1 * r1 - pd * pd / d2; // = h^2
37 if (sgn(h2) != -1) {
38     pt p = o1 + d * pd / d2, h = perp(d) * sqrt(h2 / d2);
39     // if 1 intersection h = 0
40     out = {p - h, p + h};
41 }
42 return 1 + sgn(h2); // number of intersection points
43 }

```

6.17 Intersection Between Circle and Line

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16
17 T sq(pt p) {
18     return p.x * p.x + p.y * p.y;
19 }
20
21 pt perp(pt p) {
22     return {-p.y, p.x};
23 }
24
25 T dot(pt v, pt w) {
26     return v.x * w.x + v.y * w.y;
27 }
28
29 T cross(pt v, pt w) {
30     return v.x * w.y - v.y * w.x;
31 }
32
33 struct line {
34     pt v;
35     T c;
36
37     // From direction vector v and offset c
38     line(pt v, T c) : v(v), c(c) {}
39 }
40
41 // From equation ax + by = c
42 line(T a, T b, T c) : v(b, -a), c(c) {}
43 }
44
45 // From points P and Q
46 line(pt p, pt q) : v(q - p), c(cross(v, p)) {}
47 }
48
49 // Determine the direction of a point relative to a straight line
50 T side(pt p) {
51     return cross(v, p) - c;
52 }

```

```

53 T dist(pt p) {
54     return abs(side(p)) / abs(v);
55 }
56
57 T sqDist(pt p) {
58     return side(p) * side(p) / (double) sq(v);
59 }
60
61 line perpThrough(pt p) {
62     return {p, p + perp(v)};
63 }
64
65 // Sorting along a line
66 bool cmpProj(pt p, pt q) {
67     return dot(v, p) < dot(v, q);
68 }
69
70 // translate a line l by vector t
71 line translate(pt t) {
72     return {v, c + cross(v, t)};
73 }
74
75 // s shifting line l to the left by a certain distance
76 line shiftLeft(T dist) {
77     return {v, c + dist * abs(v)};
78 }
79
80 pt proj(pt p) {
81     return p - perp(v) * side(p) / sq(v);
82 }
83
84 pt refl(pt p) {
85     return p - perp(v) * (T) 2.0 * side(p) / sq(v);
86 }
87
88 };
89
90 // o : center of circle, r : radius, l : line, out : intersection point
91 int circleLine(pt o, double r, line l, pair<pt, pt> &out) {
92     double h2 = r * r - l.sqDist(o);
93     if (h2 >= 0) {
94         // the line touches the circle
95         pt p = l.proj(o); // point P
96         pt h = l.v * sqrt(h2) / abs(l.v); // vector parallel to l, of length h
97         out = {p - h, p + h};
98     }
99     return 1 + sgn(h2); // number of intersection points
100 }

```

6.18 Intersection between line and segment

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16

```

```

17 T dist(pt a, pt b) {
18     return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
19 }
20
21 T sq(pt p) {
22     return p.x * p.x + p.y * p.y;
23 }
24
25 pt perp(pt p) {
26     return {-p.y, p.x};
27 }
28
29 T dot(pt v, pt w) {
30     return v.x * w.x + v.y * w.y;
31 }
32
33 T cross(pt v, pt w) {
34     return v.x * w.y - v.y * w.x;
35 }
36
37 struct line {
38     pt v;
39     T c;
40
41     // From points P and Q
42     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
43     }
44
45     T side(pt p) {
46         return cross(v, p) - c;
47     }
48
49     pt proj(pt p) {
50         return p - perp(v) * side(p) / sq(v);
51     }
52
53     // Sorting along a line
54     bool cmpProj(pt p, pt q) {
55         return dot(v, p) < dot(v, q);
56     }
57 };
58
59 // determining the direction of the points (right -1, left 1, or on a line 0).
60 T orient(pt a, pt b, pt c) {
61     return cross(b - a, c - a);
62 }
63
64 // checks if a point P lies on the disk of diameter [AB].
65 bool inDisk(pt a, pt b, pt p) {
66     return sgn(dot(a - p, b - p)) <= 0;
67 }
68
69 // check if a point P lies on segment [AB].
70 bool onSegment(pt a, pt b, pt p) {
71     return sgn(orient(a, b, p)) == 0 && inDisk(a, b, p);
72 }
73
74 // intersection strictly inside segments (not endpoints)
75 // if the intersection occurs at any of the endpoints a, b, c, d, it is not considered proper,
76 // but an endpoint intersection.
77 bool properInter(pt a, pt b, pt c, pt d, pt &inter) {
78     T oa = orient(c, d, a), ob = orient(c, d, b), oc = orient(a, b, c), od = orient(a, b, d);
79     if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0) {
80         T s = oa / (oa - ob);
81         inter = a + s * (b - a);
82         return true;
83     }
84     return false;

```

```

84 }
85
86 // find all intersection points between 2 segments
87 vector<pt> segmentsIntersectPoint(pt a, pt b, pt c, pt d) {
88     // check proper intersection (inside segments)
89     pt p;
90     if (properInter(a, b, c, d, p)) {
91         return {p};
92     }
93
94     // check endpoints on the other segment
95     vector<pt> pts;
96     if (onSegment(c, d, a)) pts.push_back(a);
97     if (onSegment(c, d, b)) pts.push_back(b);
98     if (onSegment(a, b, c)) pts.push_back(c);
99     if (onSegment(a, b, d)) pts.push_back(d);
100
101     // sort lexicographically (x first, then y)
102     sort(pts.begin(), pts.end(), [](pt a, pt b) {
103         if (sgn(abs(a.x - b.x))) return a.x < b.x;
104         return a.y < b.y;
105     });
106
107     // remove duplicates
108     pts.resize(unique(pts.begin(), pts.end()) - pts.begin());
109
110     return pts;
111 }
112
113 void interSegLine(line l, pt a, pt b, vector<pt> &out) {
114     out = segmentsIntersectPoint(a, b, l.proj(a), l.proj(b));
115 }

```

6.19 Intersection between two segments

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16
17 T dist(pt a, pt b) {
18     return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
19 }
20
21 T sq(pt p) {
22     return p.x * p.x + p.y * p.y;
23 }
24
25 pt perp(pt p) {
26     return {-p.y, p.x};
27 }
28
29 T dot(pt v, pt w) {
30     return v.x * w.x + v.y * w.y;
31 }
32

```

```

33 T cross(pt v, pt w) {
34     return v.x * w.y - v.y * w.x;
35 }
36
37 struct line {
38     pt v;
39     T c;
40
41     // From points P and Q
42     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
43     }
44
45     T side(pt p) {
46         return cross(v, p) - c;
47     }
48
49     pt proj(pt p) {
50         return p - perp(v) * side(p) / sq(v);
51     }
52
53     // Sorting along a line
54     bool cmpProj(pt p, pt q) {
55         return dot(v, p) < dot(v, q);
56     }
57 };
58
59 // determining the direction of the points (right -1, left 1, or on a line 0).
60 T orient(pt a, pt b, pt c) {
61     return cross(b - a, c - a);
62 }
63
64 // checks if a point P lies on the disk of diameter [AB].
65 bool inDisk(pt a, pt b, pt p) {
66     return sgn(dot(a - p, b - p)) <= 0;
67 }
68
69 // check if a point P lies on segment [AB].
70 bool onSegment(pt a, pt b, pt p) {
71     return sgn(orient(a, b, p)) == 0 && inDisk(a, b, p);
72 }
73
74 // intersection strictly inside segments (not endpoints)
75 // if the intersection occurs at any of the endpoints a, b, c, d, it is not considered proper,
76 // but an endpoint intersection.
77 bool properInter(pt a, pt b, pt c, pt d, pt &inter) {
78     T oa = orient(c, d, a), ob = orient(c, d, b), oc = orient(a, b, c), od = orient(a, b, d);
79     if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0) {
80         T s = oa / (oa - ob);
81         inter = a + s * (b - a);
82         return true;
83     }
84     return false;
85 }
86
87 // find all intersection points between 2 segments
88 vector<pt> segmentsIntersectPoint(pt a, pt b, pt c, pt d) {
89     // check proper intersection (inside segments)
90     pt p;
91     if (properInter(a, b, c, d, p)) {
92         return {p};
93     }
94
95     // check endpoints on the other segment
96     vector<pt> pts;
97     if (onSegment(c, d, a)) pts.push_back(a);
98     if (onSegment(c, d, b)) pts.push_back(b);
99     if (onSegment(a, b, c)) pts.push_back(c);
100    if (onSegment(a, b, d)) pts.push_back(d);

```

```

101    // sort lexicographically (x first, then y)
102    sort(pts.begin(), pts.end(), [](pt a, pt b) {
103        if (sgn(abs(a.x - b.x))) return a.x < b.x;
104        return a.y < b.y;
105    });
106
107    // remove duplicates
108    pts.resize(unique(pts.begin(), pts.end()) - pts.begin());
109
110    return pts;
111 }

```

6.20 Is 2 points on same side of line

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const long double EPS = 1e-9;
5
6  typedef long double T;
7  typedef complex<T> pt;
8  #define x real()
9  #define y imag()
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16
17 bool sameSide(pt p1, pt p2, T a, T b, T c) {
18     T eq1 = a * p1.x + b * p1.y + c, eq2 = a * p2.x + b * p2.y + c;
19     return (sgn(eq1 * eq2) >= 0);
20 }
21
22 int main() {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr), cout.tie(nullptr);
25
26     T x1, y1, x2, y2, a, b, c;
27     cin >> x1 >> y1 >> x2 >> y2 >> a >> b >> c;
28
29     cout << (sameSide({x1, y1}, {x2, y2}, a, b, c) ? "YES" : "NO") << '\n';
30
31     return 0;
32 }

```

6.21 Is 2 Segment Intersect

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef long double T;
5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  T cross(pt v, pt w) {
10     return v.x * w.y - v.y * w.x;
11 }
12
13 T dot(pt v, pt w) {
14     return v.x * w.x + v.y * w.y;
15 }
16

```

```

17 // determining the direction of the points (right, left, or on a line).
18 T orient(pt a, pt b, pt c) {
19     return cross(b - a, c - a);
20 }
21
22 // checks if a point P lies on the disk of diameter [AB].
23 bool inDisk(pt a, pt b, pt p) {
24     return dot(a - p, b - p) <= 0;
25 }
26
27 // check if a point P lies on segment [AB].
28 bool onSegment(pt a, pt b, pt p) {
29     return orient(a, b, p) == 0 && inDisk(a, b, p);
30 }
31
32 // Returns true if segments two intersect
33 bool segmentsIntersect(pt p1, pt q1, pt p2, pt q2) {
34     T o1 = orient(p1, q1, p2);
35     T o2 = orient(p1, q1, q2);
36     T o3 = orient(p2, q2, p1);
37     T o4 = orient(p2, q2, q1);
38
39     // General Case: They intersect if the ends of one are on opposite sides of the other.
40     if (o1 * o2 < 0 && o3 * o4 < 0) {
41         return true;
42     }
43
44     if (o1 == 0 && onSegment(p1, q1, p2)) return true;
45     if (o2 == 0 && onSegment(p1, q1, q2)) return true;
46     if (o3 == 0 && onSegment(p2, q2, p1)) return true;
47     if (o4 == 0 && onSegment(p2, q2, q1)) return true;
48
49     return false;
50 }
51
52 int main() {
53     ios::sync_with_stdio(false);
54     cin.tie(nullptr), cout.tie(nullptr);
55
56     T x1, y1, x2, y2, x3, y3, x4, y4;
57     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3 >> x4 >> y4;
58
59     cout << (segmentsIntersect({x1, y1}, {x2, y2}, {x3, y3}, {x4, y4}) ? "YES" : "NO") << '\n';
60
61     return 0;
62 }
63

```

6.22 Is point on segment

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T dot(pt v, pt w) {
10     return v.x * w.x + v.y * w.y;
11 }
12
13 T cross(pt v, pt w) {
14     return v.x * w.y - v.y * w.x;
15 }
16
17 // determining the direction of the points (right, left, or on a line).

```

```

18 T orient(pt a, pt b, pt c) {
19     return cross(b - a, c - a);
20 }
21
22 // checks if a point P lies on the disk of diameter [AB].
23 bool inDisk(pt a, pt b, pt p) {
24     return dot(a - p, b - p) <= 0;
25 }
26
27 // check if a point P lies on segment [AB].
28 bool onSegment(pt a, pt b, pt p) {
29     return orient(a, b, p) == 0 && inDisk(a, b, p);
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr), cout.tie(nullptr);
35
36     long double px, py, sx, sy, ex, ey;
37     cin >> px >> py >> sx >> sy >> ex >> ey;
38
39     cout << (onSegment({sx, sy}, {ex, ey}, {px, py}) ? "YES\n" : "NO\n");
40
41     return 0;
42 }

```

6.23 Is the point on line

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long double EPS = 1e-9;
5
6 typedef long double T;
7 typedef complex<T> pt;
8 #define x real()
9 #define y imag()
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr), cout.tie(nullptr);
14
15     long double px, py, a, b, c;
16     cin >> px >> py >> a >> b >> c;
17
18     long double eq = px * a + py * b + c;
19
20     cout << (abs(eq) < EPS ? "YES" : "NO") << '\n';
21
22     return 0;
23 }

```

6.24 Is the point on ray

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long double EPS = 1e-9;
5
6 typedef long double T;
7 typedef complex<T> pt;
8 #define x real()
9 #define y imag()
10
11 T dot(pt v, pt w) {
12     return v.x * w.x + v.y * w.y;

```

```

13 }
14
15 T cross(pt v, pt w) {
16     return v.x * w.y - v.y * w.x;
17 }
18
19 bool isPointOnRay(pt p, pt s, pt e) {
20     pt sp = p - s;
21     pt se = e - s;
22
23     bool same_dir_vector = abs(cross(se, sp)) < EPS; // vector se must be in the same direction
24     // as vector sp.
25     bool same_dir_ray = dot(se, sp) >= -EPS; // The point must be in the same direction as the
26     // ray.
27
28     return (same_dir_vector && same_dir_ray);
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr), cout.tie(nullptr);
34
35     long double px, py, sx, sy, ex, ey;
36     cin >> px >> py >> sx >> sy >> ex >> ey;
37
38     pt p = {px, py}, s = {sx, sy}, e = {ex, ey};
39
40     cout << (isPointOnRay(p, s, e) ? "YES" : "NO") << '\n';
41
42     return 0;
43 }

```

6.25 Lattice Points

```

1 // Lattice points are points on a grid whose coordinates are integers, like: (0,0), (2,3), (-1,5)
2 // They are usually divided into two types:
3 // 1) Boundary Lattice Points (B)
4 // 2) Points with integer coordinates that lie on the edges of the polygon.
5 // 3) Interior Lattice Points (I)
6 // Points with integer coordinates that lie strictly inside the polygon (not on edges).
7
8 #include <bits/stdc++.h>
9 using namespace std;
10
11 typedef long long T;
12 typedef complex<T> pt;
13 #define x real()
14 #define y imag()
15
16 T cross(pt v, pt w) {
17     return v.x * w.y - v.y * w.x;
18 }
19
20 T areaPolygon(vector<pt> p) {
21     T area = 0;
22     for (int i = 0, n = p.size(); i < n; i++) {
23         area += cross(p[i], p[(i + 1) % n]); // wrap back to 0 if i == n-1
24     }
25     return abs(area);
26 }
27
28 int main() {
29     ios::sync_with_stdio(false);
30     cin.tie(nullptr), cout.tie(nullptr);
31
32     int n;
33     cin >> n;

```

```

34     vector<pt> p(n);
35     for (int i = 0; i < n; i++) {
36         int a, b;
37         cin >> a >> b;
38         p[i] = pt(a, b);
39     }
40
41     T area = areaPolygon(p), B = 0;
42
43     // Pick Theorem -> area = I + (B / 2) - 1
44     // Can you use Pick's theorem? If a polygon has all its vertices at integer coordinates (like
45     // (0,0), (3,2), (-1,4), etc.), then it is called a lattice polygon.
46     for (int i = 0; i < n; i++) {
47         pt p1 = p[i], p2 = p[(i + 1) % n];
48         // the number of lattice points on a segment is gcd(|dx|, |dy|) + 1 including 2 points of
49         // segment
50         B += gcd(abs(p1.x - p2.x), abs(p1.y - p2.y));
51     }
52
53     T two_I = area - B + 2, I = two_I / 2;
54     cout << I << ' ' << B << '\n';
55
56     return 0;
57 }

```

6.26 Length of the union of segments

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // the length of the union of intervals (how much they cover in total, without double-counting
5 // overlaps).
6 // [1, 5], [3, 7], [10, 12]
7 // [1 ----- 5]
8 // [3 ----- 7] [10 -- 12]
9 // [1 ----- 7] [10 -- 12]
10 // so the total covered length is (7 - 1) + (12 - 10) = 8
11
12 int length_union(vector<pair<int, int> > &a) {
13     int n = (int) a.size();
14     vector<pair<int, bool> > v(2 * n);
15     for (int i = 0; i < n; i++) {
16         v[2 * i] = make_pair(a[i].first, false); // start
17         v[2 * i + 1] = make_pair(a[i].second, true); // end
18     }
19     sort(v.begin(), v.end());
20
21     int len = 0, cnt_open = 0;
22     for (int i = 0; i < n * 2; i++) {
23         if (i > 0 && v[i].first > v[i - 1].first && cnt_open > 0) {
24             len += v[i].first - v[i - 1].first;
25         }
26         cnt_open += (v[i].second ? -1 : 1);
27     }
28     return len;
29 }

```

6.27 Location of Point in line

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()

```

```

7 #define y imag()
8
9 const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16
17 T cross(pt v, pt w) {
18     return v.x * w.y - v.y * w.x;
19 }
20
21 // determine whether c is located on the left or right side of the line or if it touches the line
22 // when we are looking from a to b.
23 // determining the direction of the points (right -1, left 1, or on a line 0).
24 T orient(pt a, pt b, pt c) {
25     return cross(b - a, c - a);
26 }
27
28 int main() {
29     ios::sync_with_stdio(false);
30     cin.tie(nullptr), cout.tie(nullptr);
31
32     T x1, y1, x2, y2, x3, y3;
33     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;
34
35     T side = orient({x1, y1}, {x2, y2}, {x3, y3});
36     if (sgn(side) == -1) {
37         cout << "RIGHT\n";
38     } else if (sgn(side) == 1) {
39         cout << "LEFT\n";
40     } else {
41         cout << "TOUCH\n";
42     }
43
44     return 0;
45 }

```

6.28 MEC

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 T dist(pt a, pt b) {
14     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
15 }
16
17 T dist2(pt a, pt b) {
18     return sqrt(dist(a, b));
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25 pt perp(pt p) {

```

```

26     return {-p.y, p.x};
27 }
28
29 // you can determine circle with 3 points
30 pt circumCenter(pt a, pt b, pt c) {
31     b = b - a, c = c - a; // consider coordinates relative to A
32     // assert(cross(b, c) != 0); // no circumcircle if A,B,C aligned
33     return a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / 2.0L;
34 }
35
36 // a circle with minimum possible radius that contains all the n points, inside the circle or on
37 // its boundary
38 pair<pt, T> mec(vector<pt> &p) {
39     if (p.empty()) {
40         return make_pair(pt(0, 0), 0.0L);
41     }
42
43     shuffle(p.begin(), p.end(), mt19937(time(0)));
44
45     pt center = p[0];
46     T r = 0, EPS = 1e-8 + 1;
47     for (int i = 0; i < p.size(); i++) {
48         if (dist2(center, p[i]) > r * EPS) {
49             center = p[i], r = 0;
50             for (int j = 0; j < i; j++) {
51                 if (dist2(center, p[j]) > r * EPS) {
52                     center = (p[i] + p[j]) / 2.0L;
53                     r = dist2(center, p[i]);
54                     for (int k = 0; k < j; k++) {
55                         if (dist2(center, p[k]) > r * EPS) {
56                             center = circumCenter(p[i], p[j], p[k]);
57                             r = dist2(center, p[i]);
58                         }
59                     }
60                 }
61             }
62         }
63     }
64     return make_pair(center, r);
65 }

```

6.29 Minkowski sum of convex polygons

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 T dot(pt v, pt w) {
14     return v.x * w.x + v.y * w.y;
15 }
16
17 pt perp(pt p) {
18     return {-p.y, p.x};
19 }
20
21 T cross(pt v, pt w) {
22     return v.x * w.y - v.y * w.x;
23 }
24
25

```

```

25 struct line {
26     pt v;
27     T c;
28
29     // From points P and Q
30     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
31     }
32
33     T side(pt p) {
34         return cross(v, p) - c;
35     }
36
37     T dist(pt p) {
38         return abs(side(p)) / abs(v);
39     }
40
41     // Sorting along a line
42     bool cmpProj(pt p, pt q) {
43         return dot(v, p) < dot(v, q);
44     }
45 };
46
47 void reorder_polygon(vector<pt> &p) {
48     int pos = 0;
49     for (int i = 1; i < p.size(); i++) {
50         if (p[i].y < p[pos].y || (p[i].y == p[pos].y && p[i].x < p[pos].x)) {
51             pos = i;
52         }
53     }
54     rotate(p.begin(), p.begin() + pos, p.end());
55 }
56
57 // computes the Minkowski sum of two convex polygons p and q
58 vector<pt> minkowski(vector<pt> p, vector<pt> q) {
59     // the first vertex must be the lowest
60     reorder_polygon(p);
61     reorder_polygon(q);
62
63     // we must ensure cyclic indexing
64     p.push_back(p[0]);
65     p.push_back(p[1]);
66     q.push_back(q[0]);
67     q.push_back(q[1]);
68
69     vector<pt> ret;
70     int i = 0, j = 0;
71     while (i < p.size() - 2 || j < q.size() - 2) {
72         ret.push_back(p[i] + q[j]);
73         auto cros = cross(p[i + 1] - p[i], q[j + 1] - q[j]);
74         i += (cros >= 0 && i < p.size() - 2);
75         j += (cros <= 0 && j < q.size() - 2);
76     }
77     return ret;
78 }

```

6.30 Point Location Relative to Polygon

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T dot(pt v, pt w) {
10     return v.x * w.x + v.y * w.y;

```

```

11 }
12
13 T cross(pt v, pt w) {
14     return v.x * w.y - v.y * w.x;
15 }
16
17 // determining the direction of the points (right -1, left 1, or on a line 0).
18 T orient(pt a, pt b, pt c) {
19     return cross(b - a, c - a);
20 }
21
22 // true if P at least as high as A (blue part)
23 bool above(pt a, pt p) {
24     return p.y >= a.y;
25 }
26
27 // check if [PQ] crosses ray from A
28 bool crossesRay(pt a, pt p, pt q) {
29     return (above(a, q) - above(a, p)) * orient(a, p, q) > 0;
30 }
31
32 // checks if a point P lies on the disk of diameter [AB].
33 bool inDisk(pt a, pt b, pt p) {
34     return dot(a - p, b - p) <= 0;
35 }
36
37 // check if a point P lies on segment [AB].
38 bool onSegment(pt a, pt b, pt p) {
39     return orient(a, b, p) == 0 && inDisk(a, b, p);
40 }
41
42 // if strict, returns false when A is on the boundary
43 bool inPolygon(vector<pt> p, pt a, bool strict) {
44     int numCrossings = 0;
45     for (int i = 0, n = p.size(); i < n; i++) {
46         if (onSegment(p[i], p[(i + 1) % n], a))
47             return !strict;
48         numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
49     }
50     return numCrossings & 1;
51 }
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr), cout.tie(nullptr);
56
57     int n;
58     cin >> n;
59
60     vector<pt> p(n);
61     for (int i = 0; i < n; i++) {
62         T a, b;
63         cin >> a >> b;
64         p[i] = pt(a, b);
65     }
66
67     T px, py;
68     cin >> px >> py;
69
70     bool ok = inPolygon(p, {px, py}, true), ok2 = inPolygon(p, {px, py}, false);
71     if (ok && ok2) {
72         cout << "INSIDE\n";
73     } else if (!ok && !ok2) {
74         cout << "OUTSIDE\n";
75     } else {
76         cout << "BOUNDARY\n";
77     }
78 }

```

```

79     return 0;
80 }

```

6.31 Polar Angle

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr), cout.tie(nullptr);
7
8      double a, b;
9      cin >> a >> b;
10
11     // atan2(y, x)
12     // from origin
13     // if not origin atan(dy, dx)
14     double angle = atan2(b, a);
15     if (angle < 0) angle += 2 * acos(-1.0);
16     cout << fixed << setprecision(10) << angle;
17
18     return 0;
19 }

```

6.32 Polar Sort

```

1  // cross product will be positive if the rotation from v to w is performed counter-clockwise and
   // negative otherwise
2  // so for two vectors that are "close" in angle (within 180 deg) the sign of the cross product
   // correctly tells which has the larger angle.
3  // but if two vectors are more than 180 deg apart, the cross is no longer sufficient
4  // to avoid this ambiguity we partition vectors into two groups (halves) by angle, typically:
5  // "Blue half" = angles in (0,pi] (upper half-plane, plus the negative x-axis),
6  // and "Other half" = angles in (-pi,0] (lower half-plane, plus positive x-axis).
7  // if two vectors point exactly in the same direction, cross(v, w) == 0
8  // sort will leave them in arbitrary relative order (sq)
9
10 #include <bits/stdc++.h>
11 using namespace std;
12
13 typedef long long T;
14 typedef complex<T> pt;
15 #define x real()
16 #define y imag()
17
18 T sq(pt p) {
19     return p.x * p.x + p.y * p.y;
20 }
21
22 T cross(pt v, pt w, pt origin = {0, 0}) {
23     return (v.x - origin.x) * (w.y - origin.y) - (v.y - origin.y) * (w.x - origin.x);
24 }
25
26 // true if in blue half
27 bool half(pt p, pt origin = {0, 0}) {
28     T dx = p.x - origin.x, dy = p.y - origin.y;
29     assert(dx != 0 || dy != 0); // (0, 0) has no angle
30     // TODO
31     return dy > 0 || (dy == 0 && dx < 0); // or dx > 0
32 }
33
34 // around origin
35 void polarSort(vector<pt> &v) {
36     sort(v.begin(), v.end(), [](const pt &v, const pt &w) {
37         // TODO

```

```

38     if (half(v) != half(w)) {
39         // or half(v) > half(w) depending on which half first
40         return half(v) < half(w);
41     }
42     if (cross(v, w) != 0) {
43         return cross(v, w) > 0;
44     }
45     return sq(v) < sq(w);
46 });
47 }
48
49 void polarSortAround(pt o, vector<pt> &v) {
50     sort(v.begin(), v.end(), [o](const pt &v, const pt &w) {
51         // TODO
52         if (half(v, o) != half(w, o)) {
53             // or half(v) > half(w) depending on which half first
54             return half(v, o) < half(w, o);
55         }
56         if (cross(v, w, o) != 0) {
57             return cross(v, w, o) > 0;
58         }
59         return sq(v - o) < sq(w - o);
60     });
61 }
62
63 // sort by atan2(x, y)
64 // sort the points in counterclockwise order that starts from the half line x <= 0, y = 0
65 // start from the 3rd quadrant and end at the 2nd quadrant
66 void sort_argument(vector<pt> &v) {
67     // atan2(y, x)
68     sort(v.begin(), v.end(), [&](pt a, pt b) {
69         return atan2l(a.y, a.x) < atan2l(b.y, b.x);
70     });
71 }
72
73 // sort points counter-clockwise around center
74 void sort_ccw(vector<pt> &v) {
75     pt center(0, 0);
76     for (auto p: v) {
77         center += p;
78     }
79     center /= (int) v.size();
80
81     sort(v.begin(), v.end(), [&](pt a, pt b) {
82         T ang1 = atan2l(a.y - center.y, a.x - center.x);
83         T ang2 = atan2l(b.y - center.y, b.x - center.x);
84         return ang1 < ang2;
85     });
86 }

```

6.33 struct line

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef double T;
5  typedef complex<T> pt;
6  #define x real()
7  #define y imag()
8
9  T sq(pt p) {
10     return p.x * p.x + p.y * p.y;
11 }
12
13 T dot(pt v, pt w) {
14     return v.x * w.x + v.y * w.y;
15 }

```

```

16 pt perp(pt p) {
17     return {-p.y, p.x};
18 }
19
20 T cross(pt v, pt w) {
21     return v.x * w.y - v.y * w.x;
22 }
23
24 struct line {
25     pt v;
26     T c;
27
28     // From direction vector v and offset c
29     line(pt v, T c) : v(v), c(c) {
30     }
31
32     // From equation ax + by = c
33     line(T a, T b, T c) : v(b, -a), c(c) {
34     }
35
36     // From points P and Q
37     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
38     }
39
40     T side(pt p) {
41         return cross(v, p) - c;
42     }
43
44     T dist(pt p) {
45         return abs(side(p)) / abs(v);
46     }
47
48     T sqDist(pt p) {
49         return side(p) * side(p) / (double) sq(v);
50     }
51
52     line perpThrough(pt p) {
53         return {p, p + perp(v)};
54     }
55
56     // Sorting along a line
57     bool cmpProj(pt p, pt q) {
58         return dot(v, p) < dot(v, q);
59     }
60
61     // translate a line l by vector t
62     line translate(pt t) {
63         return {v, c + cross(v, t)};
64     }
65
66     // s shifting line l to the left by a certain distance
67     line shiftLeft(T dist) {
68         return {v, c + dist * abs(v)};
69     }
70
71     pt proj(pt p) {
72         return p - perp(v) * side(p) / sq(v);
73     }
74
75     pt refl(pt p) {
76         return p - perp(v) * (T) 2.0 * side(p) / sq(v);
77     }
78 };
79
80 int main() {
81     ios::sync_with_stdio(false);
82     cin.tie(nullptr), cout.tie(nullptr);
83

```

```

84
85     return 0;
86 }

```

6.34 type of intersection between Circle and Line

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 const long double EPS = 1e-9;
10
11 int sgn(T val) {
12     if (val > EPS) return 1;
13     if (val < -EPS) return -1;
14     return 0;
15 }
16
17 T sq(pt p) {
18     return p.x * p.x + p.y * p.y;
19 }
20
21 T dot(pt v, pt w) {
22     return v.x * w.x + v.y * w.y;
23 }
24
25 pt perp(pt p) {
26     return {-p.y, p.x};
27 }
28
29 T cross(pt v, pt w) {
30     return v.x * w.y - v.y * w.x;
31 }
32
33 struct line {
34     pt v;
35     T c;
36
37     // From direction vector v and offset c
38     line(pt v, T c) : v(v), c(c) {
39     }
40
41     // From equation ax + by = c
42     line(T a, T b, T c) : v(b, -a), c(c) {
43     }
44
45     // From points P and Q
46     line(pt p, pt q) : v(q - p), c(cross(v, p)) {
47     }
48
49     T side(pt p) {
50         return cross(v, p) - c;
51     }
52
53     T dist(pt p) {
54         return abs(side(p)) / abs(v);
55     }
56
57     T sqDist(pt p) {
58         return side(p) * side(p) / (double) sq(v);
59     }
60
61     line perpThrough(pt p) {

```

```

62     return {p, p + perp(v)};
63 }
64
65 // Sorting along a line
66 bool cmpProj(pt p, pt q) {
67     return dot(v, p) < dot(v, q);
68 }
69
70 // translate a line l by vector t
71 line translate(pt t) {
72     return {v, c + cross(v, t)};
73 }
74
75 // s shifting line l to the left by a certain distance
76 line shiftLeft(T dist) {
77     return {v, c + dist * abs(v)};
78 }
79
80 pt proj(pt p) {
81     return p - perp(v) * side(p) / sq(v);
82 }
83
84 pt refl(pt p) {
85     return p - perp(v) * (T) 2.0 * side(p) / sq(v);
86 }
87 };
88
89 // determining the direction of the points (right -1, left 1, or on a line 0).
90 T orient(pt a, pt b, pt c) {
91     return cross(b - a, c - a);
92 }
93
94 // The center of the circle passing through three points
95 pt circumCenter(pt a, pt b, pt c) {
96     b = b - a, c = c - a; // consider coordinates relative to A
97     assert(cross(b,c) != 0); // no circumcircle if A,B,C aligned
98     return a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / 2.0;
99 }
100
101 // Points of intersection of a line with a circle in the plane and return type of intersection
102 int circleLine(pt o, double r, line l, pair<pt, pt> &out) {
103     T h2 = r * r - l.sqDist(o);
104     if (h2 >= 0) {
105         // the line touches the circle
106         pt p = l.proj(o); // point P
107         pt h = l.v * sqrt(h2) / abs(l.v); // vector parallel to l, of length h
108         out = {p - h, p + h};
109     }
110     return 1 + sgn(h2);
111 }
112
113 int main() {
114     ios::sync_with_stdio(false);
115     cin.tie(nullptr), cout.tie(nullptr);
116
117     return 0;
118 }

```

6.35 vector length

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long double T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()

```

```

8
9 T dist(pt p, pt p2) {
10     T dx = abs(p.x - p2.x), dy = abs(p.y - p2.y);
11     return sqrt(dx * dx + dy * dy);
12 }
13
14 int main() {
15     ios::sync_with_stdio(false);
16     cin.tie(nullptr), cout.tie(nullptr);
17
18     long double x1, y1, x2, y2;
19     cin >> x1 >> y1 >> x2 >> y2;
20
21     cout << fixed << setprecision(15) << dist({x1, y1}, {x2, y2}) << '\n';
22
23     return 0;
24 }

```

6.36 winding Number

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T dot(pt v, pt w) {
10     return v.x * w.x + v.y * w.y;
11 }
12
13 double angle(pt v, pt w) {
14     double cosTheta = dot(v, w) / abs(v) / abs(w);
15     return acos(max(-1.0, min(1.0, cosTheta)));
16 }
17
18 T cross(pt v, pt w) {
19     return v.x * w.y - v.y * w.x;
20 }
21
22 // determining the direction of the points (right -1, left 1, or on a line 0).
23 T orient(pt a, pt b, pt c) {
24     return cross(b - a, c - a);
25 }
26
27 // amplitude travelled around point A, from P to Q
28 double angleTravelled(pt a, pt p, pt q) {
29     T ampli = angle(p - a, q - a);
30     if (orient(a, p, q) > 0) return ampli;
31     else return -ampli;
32 }
33
34 // The number of times the polygon wraps around this point
35 int windingNumber(vector<pt> p, pt a) {
36     T ampli = 0;
37     for (int i = 0, n = p.size(); i < n; i++) {
38         ampli += angleTravelled(a, p[i], p[(i + 1) % n]);
39     }
40     return round(ampli / (2 * M_PI));
41 }
42
43 int main() {
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr), cout.tie(nullptr);
46
47     return 0;

```

48

}

6.37 WindingNumber for Integers

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long T;
5 typedef complex<T> pt;
6 #define x real()
7 #define y imag()
8
9 T dot(pt v, pt w) {
10     return v.x * w.x + v.y * w.y;
11 }
12
13 double angle(pt v, pt w) {
14     double cosTheta = dot(v, w) / abs(v) / abs(w);
15     return acos(max(-1.0, min(1.0, cosTheta)));
16 }
17
18 T cross(pt v, pt w) {
19     return v.x * w.y - v.y * w.x;
20 }
21
22 // determining the direction of the points (right -1, left 1, or on a line 0).
23 T orient(pt a, pt b, pt c) {
24     return cross(b - a, c - a);
25 }
26
27 // amplitude travelled around point A, from P to Q
28 double angleTravelled(pt a, pt p, pt q) {
29     T ampli = angle(p - a, q - a);
30     if (orient(a, p, q) > 0) return ampli;
31     else return -ampli;
32 }
33
34 bool half(pt p) {
35     // true if in blue half
36     assert(p.x != 0 || p.y != 0);
37     return p.y > 0 || (p.y == 0 && p.x < 0);
38 }
39
40 // checks if a point P lies on the disk of diameter [AB].
41 bool inDisk(pt a, pt b, pt p) {
42     return dot(a - p, b - p) <= 0;
43 }
44
45 // check if a point P lies on segment [AB].
46 bool onSegment(pt a, pt b, pt p) {
47     return orient(a, b, p) == 0 && inDisk(a, b, p);
48 }
49
50 struct anglle {
51     pt d;
52     int t = 0; // direction and number of full turns
53
54     // Turn 180 degrees counter-clockwise
55     anglle t180() {
56         return {d * (-1LL), t + half(d)};
57     }
58
59     // Turn 360 degrees counter-clockwise (full turn)
60     anglle t360() {
61         return {d, t + 1};
62     }
63 };

```

64

}

```

65 bool operator<(anglle a, anglle b) {
66     return make_tuple(a.t, half(a.d), 0) < make_tuple(b.t, half(b.d), cross(a.d, b.d));
67 }
68
69 anglle moveTo(anglle a, pt newD) {
70     // check that segment [DD'] doesn't go through the origin
71     assert(!onSegment(a.d, newD, {0,0}));
72     anglle b{newD, a.t};
73     if (a.t180() < b) // if b more than half a turn bigger
74         b.t--; //decrease b by a full turn
75     if (b.t180() < a) // if b more than half a turn smaller
76         b.t++; // increase b by a full turn
77     return b;
78 }
79
80 int windingNumber2(vector<pt> p, pt a) {
81     anglle a{p.back()}; // start at last vertex
82     for (pt d: p) {
83         a = moveTo(a, d); // move to first vertex, second, etc.
84     }
85     return a.t;
86 }
87
88 int main() {
89     ios::sync_with_stdio(false);
90     cin.tie(nullptr), cout.tie(nullptr);
91
92     return 0;
93 }

```

7 Graph Algorithms

7.1 articulation_points

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // O(N + M)
5
6 int timer, n, m;
7 vector<int> tin, low;
8 vector<vector<int>> > al;
9 vector<bool> vis, is_cutpoint;
10
11 void dfs(int u, int p) {
12     vis[u] = true;
13     int children = 0;
14     tin[u] = low[u] = timer++;
15     for (auto &v: al[u]) {
16         if (v == p) continue;
17         if (vis[v]) {
18             // back-edge
19             low[u] = min(low[u], tin[v]);
20         } else {
21             dfs(v, u);
22             low[u] = min(low[u], low[v]);
23             // tree-edge
24             if (low[v] >= tin[u] && p != -1) {
25                 is_cutpoint[u] = true;
26             }
27             ++children;
28         }
29     }
30     if (p == -1 && children > 1) {
31         is_cutpoint[u] = true; // Root with multiple children
32     }
33 }

```

```

33 }
34
35 void find_cutpoint() {
36     timer = 1;
37     vis.assign(n + 1, false);
38     tin.assign(n + 1, -1);
39     low.assign(n + 1, -1);
40     is_cutpoint.assign(n + 1, false);
41     for (int u = 1; u <= n; ++u) {
42         if (!vis[u]) {
43             dfs(u, -1);
44         }
45     }
46 }
47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr), cout.tie(nullptr);
51
52     return 0;
53 }

```

7.2 BCC

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  stack<int> order;
5  int timer, n, m, cur_id;
6  vector<int> tin, low, comp_id, d;
7  vector<bool> vis, is_cutpoint;
8  vector<vector<int>> al, comps, tree, up;
9
10 void dfs(int u, int p) {
11     vis[u] = true;
12     tin[u] = low[u] = timer++;
13     order.push(u);
14     int children = 0;
15     for (int v: al[u]) {
16         if (v == p) continue;
17         if (vis[v]) {
18             low[u] = min(low[u], tin[v]);
19         } else {
20             dfs(v, u);
21             low[u] = min(low[u], low[v]);
22             if (low[v] >= tin[u]) {
23                 is_cutpoint[u] = (p != -1 || children > 1);
24
25                 int w;
26                 vector<int> comp;
27                 do {
28                     w = order.top();
29                     order.pop();
30                     comp.push_back(w);
31                 } while (!order.empty() && w != v);
32                 comp.push_back(u);
33                 comps.push_back(comp);
34             }
35             children++;
36         }
37     }
38     if (p == -1 && children > 1) {
39         is_cutpoint[u] = true;
40     }
41 }
42
43 void build_tree() {

```

```

44     for (int u = 1; u <= n; u++) {
45         if (is_cutpoint[u]) {
46             comp_id[u] = cur_id++;
47             tree.emplace_back();
48         }
49     }
50
51     for (auto &comp: comps) {
52         int u = cur_id++;
53         tree.emplace_back();
54         for (int v: comp) {
55             if (is_cutpoint[v]) {
56                 tree[u].push_back(comp_id[v]);
57                 tree[comp_id[v]].push_back(u);
58             } else {
59                 comp_id[v] = u;
60             }
61         }
62     }
63 }
64
65 void BCC() {
66     timer = 1;
67     low.assign(n + 1, -1);
68     tin.assign(n + 1, -1);
69     vis.assign(n + 1, false);
70     is_cutpoint.assign(n + 1, false);
71
72     for (int u = 1; u <= n; u++) {
73         if (!vis[u]) {
74             dfs(u, -1);
75         }
76     }
77
78     cur_id = 1;
79     comp_id = vector<int>(n + 1, -1);
80     tree = vector<vector<int>>(1);
81     build_tree();
82 }

```

7.3 Bellman-Ford

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int INF = 1e9;
5
6  int n;
7  vector<array<int, 3>> edges;
8  vector<vector<pair<int, int>>> al;
9
10 bool spfa(int src, vector<int> &d) {
11     d.assign(n, INF);
12     d[src] = 0;
13
14     vector<int> cnt(n + 1, 0);
15
16     vector<bool> inqueue(n + 1);
17     inqueue[src] = true;
18
19     queue<int> q;
20     q.push(src);
21     while (!q.empty()) {
22         int u = q.front();
23         q.pop();
24         inqueue[u] = false;
25         for (auto &[v, w]: al[u]) {

```

```

26         if (d[u] + w < d[v]) {
27             d[v] = d[u] + w;
28             if (!inqueue[v]) {
29                 q.push(v);
30                 inqueue[v] = true;
31                 cnt[v]++;
32                 if (cnt[v] > n) {
33                     return false; // negative cycle
34                 }
35             }
36         }
37     }
38 }
39 return true;
40 }
41
42 vector<int> detect_any_cycle() {
43     vector<int> cycle, par(n + 1);
44     vector<long long> dist(n + 1);
45
46     int lst = -1;
47     for (int i = 1; i <= n; i++) {
48         lst = -1; // DON'T MISS IT
49         for (auto &[u, v, w]: edges) {
50             if (dist[v] > dist[u] + w) {
51                 dist[v] = dist[u] + w;
52                 par[v] = u;
53                 lst = v;
54             }
55         }
56     }
57
58     if (~lst) {
59         for (int i = 1; i <= n; i++) {
60             lst = par[lst];
61         }
62
63         int u = lst;
64         do {
65             cycle.push_back(lst);
66             lst = par[lst];
67         } while (lst != u);
68         cycle.push_back(u);
69
70         reverse(cycle.begin(), cycle.end());
71     }
72     return cycle;
73 }

```

7.4 Bridges

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // O(N + M)
5
6 int timer, n, m, cur_id;
7 vector<int> tin, low, comp_id;
8 vector<vector<int>> > tree;
9 vector<bool> vis, is_bridge;
10 vector<vector<pair<int, int>> > > al;
11
12 void dfs(int u, int p) {
13     vis[u] = true;
14     low[u] = tin[u] = timer++;
15     for (auto &[v, edge]: al[u]) {
16         if (v == p) continue;

```

```

17         if (vis[v]) {
18             // back-edge
19             low[u] = min(low[u], tin[v]);
20         } else {
21             dfs(v, u);
22             // tree-edge
23             low[u] = min(low[u], low[v]);
24             if (low[v] > tin[u]) {
25                 is_bridge[edge] = true;
26             }
27         }
28     }
29 }
30
31 void find_bridges() {
32     timer = 1;
33     vis.assign(n + 1, false);
34     tin.assign(n + 1, -1);
35     low.assign(n + 1, -1);
36     is_bridge.assign(m + 1, false);
37     for (int u = 1; u <= n; ++u) {
38         if (!vis[u]) {
39             dfs(u, -1);
40         }
41     }
42 }
43
44 void dfs_tree(int u) {
45     comp_id[u] = cur_id;
46     for (auto &[v, edge]: al[u]) {
47         if (is_bridge[edge] || ~comp_id[v]) continue;
48         dfs_tree(v);
49     }
50 }
51
52 void build_tree() {
53     cur_id = 0;
54     comp_id.assign(n + 1, -1);
55     for (int u = 1; u <= n; ++u) {
56         if (~comp_id[u]) continue;
57         cur_id++;
58         dfs_tree(u);
59     }
60
61     tree.assign(cur_id + 1, vector<int>());
62
63     vector<bool> added(m + 1, false);
64     for (int u = 1; u <= n; ++u) {
65         for (auto &[v, edge]: al[u]) {
66             if (is_bridge[edge] && !added[edge]) {
67                 added[edge] = true;
68                 tree[comp_id[u]].push_back(comp_id[v]);
69                 tree[comp_id[v]].push_back(comp_id[u]);
70             }
71         }
72     }
73 }
74
75 int main() {
76     ios::sync_with_stdio(false);
77     cin.tie(nullptr), cout.tie(nullptr);
78
79     return 0;
80 }

```

7.5 Bron-Kerbosch

```

1 // https://judge.yosupo.jp/problem/maximum_independent_set
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // Bron-Kerbosch
7 // Used to find the Maximum Independent Set (The maximum clique in complement graph)
8
9 int n;
10 vector<int> mx, cur;
11 vector<bitset<40>> > al;
12
13 void bronk(bitset<40> cur_clique, bitset<40> can_add, bitset<40> used) {
14     if (can_add.none() && used.none()) {
15         if (cur.size() > mx.size()) {
16             mx = cur;
17         }
18         return;
19     }
20
21     int u = -1; // choose a pivot u
22     for (int node = 0; node < n; node++) {
23         if (can_add[node] || used[node]) {
24             u = node;
25             break;
26         }
27     }
28
29     bitset<40> to_iterate = can_add;
30     if (u != -1) { // iterate only over vertices in can_add that are not neighbors of the pivot
31         to_iterate &= ~al[u];
32     }
33
34     for (int v = 0; v < n; v++) {
35         if (!to_iterate[v]) continue;
36
37         cur.push_back(v);
38
39         bronk(cur_clique | bitset<40>().set(v), can_add & al[v], used & al[v]);
40
41         cur.pop_back();
42
43         // move v from can_add to used
44         can_add.reset(v);
45         used.set(v);
46     }
47 }
48
49 int main() {
50     ios::sync_with_stdio(false);
51     cin.tie(nullptr);
52
53     int m;
54     cin >> n >> m;
55     al.assign(n, bitset<40>());
56     vector<vector<bool>> > g(n, vector<bool>(n));
57     for (int i = 1; i <= m; i++) {
58         int u, v;
59         cin >> u >> v;
60         g[u][v] = true;
61         g[v][u] = true;
62     }
63
64     // build the complement graph if two vertices are not connected in the original graph,
65     // connect them here
66     for (int u = 0; u < n; u++) {
67         for (int v = 0; v < n; v++) {

```

```

67         if (u != v && !g[u][v]) {
68             al[u].set(v);
69         }
70     }
71 }
72
73 bitset<40> cur_clique, can_add, used;
74 for (int u = 0; u < n; u++) {
75     can_add.set(u);
76 }
77
78 bronk(cur_clique, can_add, used);
79
80 cout << mx.size() << '\n';
81 for (auto &it: mx) {
82     cout << it << ' ';
83 }
84
85 return 0;
86 }

```

7.6 Dinics Algorithm

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 const int N = 5010;
5
6 const long long inf = 1LL << 61;
7 struct Dinic {
8     struct edge {
9         int to, rev;
10        long long flow, w;
11        int id;
12    };
13    int n, s, t, mxid;
14    vector<int> d, flow_through;
15    vector<int> done;
16    vector<vector<edge>> > g;
17    Dinic() {}
18    Dinic(int _n) {
19        n = _n + 10;
20        mxid = 0;
21        g.resize(n);
22    }
23    void add_edge(int u, int v, long long w, int id = -1) {
24        edge a = {v, (int)g[v].size(), 0, w, id};
25        edge b = {u, (int)g[u].size(), 0, 0, -2}; //for bidirectional edges cap(b) = w
26        g[u].emplace_back(a);
27        g[v].emplace_back(b);
28        mxid = max(mxid, id);
29    }
30    bool bfs() {
31        d.assign(n, -1);
32        d[s] = 0;
33        queue<int> q;
34        q.push(s);
35        while (!q.empty()) {
36            int u = q.front();
37            q.pop();
38            for (auto &e : g[u]) {
39                int v = e.to;
40                if (d[v] == -1 && e.flow < e.w) d[v] = d[u] + 1, q.push(v);
41            }
42        }
43        return d[t] != -1;
44    }

```

```

45 long long dfs(int u, long long flow) {
46     if (u == t) return flow;
47     for (int &i = done[u]; i < (int)g[u].size(); i++) {
48         edge &e = g[u][i];
49         if (e.w <= e.flow) continue;
50         int v = e.to;
51         if (d[v] == d[u] + 1) {
52             long long nw = dfs(v, min(flow, e.w - e.flow));
53             if (nw > 0) {
54                 e.flow += nw;
55                 g[v][e.rev].flow -= nw;
56                 return nw;
57             }
58         }
59     }
60     return 0;
61 }
62 long long max_flow(int _s, int _t) {
63     s = _s;
64     t = _t;
65     long long flow = 0;
66     while (bfs()) {
67         done.assign(n, 0);
68         while (long long nw = dfs(s, inf)) flow += nw;
69     }
70     flow_through.assign(mxid + 10, 0);
71     for(int i = 0; i < n; i++) for(auto e : g[i]) if(e.id >= 0) flow_through[e.id] = e.flow;
72     return flow;
73 }
74 };
75 int main() {
76     int n, m;
77     cin >> n >> m;
78     Dinic F(n + 1);
79     for (int i = 1; i <= m; i++) {
80         int u, v, w;
81         cin >> u >> v >> w;
82         F.add_edge(u, v, w);
83     }
84     cout << F.max_flow(1, n) << '\n';
85     return 0;
86 }

```

7.7 Floyd

```

1 for (int k = 0; k < n; ++k) {
2     for (int i = 0; i < n; ++i) {
3         for (int j = 0; j < n; ++j) {
4             if (d[i][k] < INF && d[k][j] < INF) {
5                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
6             }
7         }
8     }
9 }

```

7.8 Hungarian Algorithm

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 const int N = 509;
5
6 /* Complexity: O(n^3) but optimized
7 It finds minimum cost maximum matching.
8 For finding maximum cost maximum matching
9 add -cost and return -matching()

```

```

10 1-indexed */
11 struct Hungarian {
12     long long c[N][N], fx[N], fy[N], d[N];
13     int l[N], r[N], arg[N], trace[N];
14     queue<int> q;
15     int start, finish, n;
16     const long long inf = 1e18;
17     Hungarian() {}
18     Hungarian(int n1, int n2): n(max(n1, n2)) {
19         for (int i = 1; i <= n; ++i) {
20             fy[i] = l[i] = r[i] = 0;
21             for (int j = 1; j <= n; ++j) c[i][j] = inf;
22         }
23     }
24     void add_edge(int u, int v, long long cost) {
25         c[u][v] = min(c[u][v], cost);
26     }
27     inline long long getC(int u, int v) {
28         return c[u][v] - fx[u] - fy[v];
29     }
30     void initBFS() {
31         while (!q.empty()) q.pop();
32         q.push(start);
33         for (int i = 0; i <= n; ++i) trace[i] = 0;
34         for (int v = 1; v <= n; ++v) {
35             d[v] = getC(start, v);
36             arg[v] = start;
37         }
38         finish = 0;
39     }
40     void findAugPath() {
41         while (!q.empty()) {
42             int u = q.front();
43             q.pop();
44             for (int v = 1; v <= n; ++v) if (!trace[v]) {
45                 long long w = getC(u, v);
46                 if (!w) {
47                     trace[v] = u;
48                     if (!r[v]) {
49                         finish = v;
50                         return;
51                     }
52                     q.push(r[v]);
53                 }
54                 if (d[v] > w) {
55                     d[v] = w;
56                     arg[v] = u;
57                 }
58             }
59         }
60     }
61     void subX_addY() {
62         long long delta = inf;
63         for (int v = 1; v <= n; ++v) if (trace[v] == 0 && d[v] < delta) {
64             delta = d[v];
65         }
66         // Rotate
67         fx[start] += delta;
68         for (int v = 1; v <= n; ++v) if (trace[v]) {
69             int u = r[v];
70             fy[v] -= delta;
71             fx[u] += delta;
72         } else d[v] -= delta;
73         for (int v = 1; v <= n; ++v) if (!trace[v] && !d[v]) {
74             trace[v] = arg[v];
75             if (!r[v]) {
76                 finish = v;
77                 return;

```

```

78     }
79     q.push(r[v]);
80 }
81 }
82 void Enlarge() {
83     do {
84         int u = trace[finish];
85         int nxt = l[u];
86         l[u] = finish;
87         r[finish] = u;
88         finish = nxt;
89     } while (finish);
90 }
91 long long maximum_matching() {
92     for (int u = 1; u <= n; ++u) {
93         fx[u] = c[u][1];
94         for (int v = 1; v <= n; ++v) {
95             fx[u] = min(fx[u], c[u][v]);
96         }
97     }
98     for (int v = 1; v <= n; ++v) {
99         fy[v] = c[1][v] - fx[1];
100        for (int u = 1; u <= n; ++u) {
101            fy[v] = min(fy[v], c[u][v] - fx[u]);
102        }
103    }
104    for (int u = 1; u <= n; ++u) {
105        start = u;
106        initBFS();
107        while (!finish) {
108            findAugPath();
109            if (!finish) subX_addY();
110        }
111        Enlarge();
112    }
113    long long ans = 0;
114    for (int i = 1; i <= n; ++i) {
115        if (c[i][l[i]] != inf) ans += c[i][l[i]];
116        else l[i] = 0;
117    }
118    return ans;
119 }
120 };
121 int32_t main() {
122     ios_base::sync_with_stdio(0);
123     cin.tie(0);
124     int n1, n2, m;
125     cin >> n1 >> n2 >> m;
126     Hungarian M(n1, n2);
127     for (int i = 1; i <= m; i++) {
128         int u, v, w;
129         cin >> u >> v >> w;
130         M.add_edge(u, v, -w);
131     }
132     cout << -M.maximum_matching() << '\n';
133     for (int i = 1; i <= n1; i++) cout << M.l[i] << ' ';
134     return 0;
135 }

```

7.9 SCC_Kosaraju

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m;
5 vector<bool> vis;
6 stack<int> order;

```

```

7 vector<vector<int> > al, rev_al, components;
8
9 void dfs1(int u) {
10     vis[u] = true;
11     for (int v: al[u]) {
12         if (!vis[v]) {
13             dfs1(v);
14         }
15     }
16     order.push(u);
17 }
18
19 void kosaraju(int u, vector<int> &comp) {
20     vis[u] = true;
21     comp.push_back(u);
22     for (int v: rev_al[u]) {
23         if (!vis[v]) {
24             kosaraju(v, comp);
25         }
26     }
27 }
28
29 void scc() {
30     components.clear();
31     vis.assign(n + 1, false);
32
33     for (int u = 1; u <= n; u++) {
34         if (!vis[u]) {
35             dfs1(u);
36         }
37     }
38
39     vis.assign(n + 1, false);
40
41     while (!order.empty()) {
42         int u = order.top();
43         order.pop();
44         if (!vis[u]) {
45             components.emplace_back();
46             kosaraju(u, components.back());
47         }
48     }
49 }
50
51 int main() {
52     ios::sync_with_stdio(false);
53     cin.tie(nullptr), cout.tie(nullptr);
54
55
56     return 0;
57 }

```

7.10 SCC_Tarjan_s

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<bool> vis;
5 stack<int> order;
6 int timer, n, m, id;
7 vector<vector<int> > tree;
8 vector<int> tin, low, comp_id;
9 vector<vector<pair<int, int> > > al;
10
11 void tarjan(int u, int p) {
12     vis[u] = true;
13     order.push(u);

```

```

14     tin[u] = low[u] = timer++;
15     for (auto &[v, _]: al[u]) {
16         if (v == p) continue;
17         if (!vis[v]) {
18             // white-not-visited
19             tarjan(v, u);
20             low[u] = min(low[u], low[v]);
21         } else if (comp_id[v] == -1) {
22             // gray-in-stack
23             low[u] = min(low[u], low[v]);
24         }
25     }
26
27     if (low[u] == tin[u]) {
28         int v;
29         do {
30             v = order.top();
31             order.pop();
32             comp_id[v] = id;
33         } while (u != v);
34         id++;
35     }
36 }
37
38 void scc() {
39     timer = 1, id = 1;
40     vis.assign(n + 1, false);
41     tin.assign(n + 1, -1);
42     low.assign(n + 1, -1);
43     comp_id.assign(n + 1, -1);
44     for (int u = 1; u <= n; ++u) {
45         if (!vis[u]) {
46             tarjan(u, -1);
47         }
48     }
49 }
50
51 void build_tree() {
52     vector<bool> added(m + 1, false);
53     tree = vector<vector<int>> >(id + 1);
54     for (int u = 1; u <= n; ++u) {
55         for (auto &[v, edge]: al[u]) {
56             if (comp_id[v] != comp_id[u] && !added[edge]) {
57                 added[edge] = true;
58                 // TODO
59                 tree[comp_id[u]].push_back(comp_id[v]);
60             }
61         }
62     }
63 }
64
65 int main() {
66     ios::sync_with_stdio(false);
67     cin.tie(nullptr), cout.tie(nullptr);
68
69     return 0;
70 }

```

7.11 small to large

```

1 // https://cses.fi/problemset/result/13802419/
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int timer, cnt;

```

```

7 vector<vector<int>> > al;
8 vector<int> flatten, in, sz, frq, col, ans;
9
10 void add(int u) {
11     //TODO
12     cnt += (!frq[col[u]]);
13     frq[col[u]]++;
14 }
15
16 void remove(int u) {
17     //TODO
18     cnt -= (frq[col[u]] == 1);
19     frq[col[u]]--;
20 }
21
22 int get_answer() {
23     //TODO
24     return cnt;
25 }
26
27 void pre_dfs(int u, int p) {
28     sz[u] = 1;
29     in[u] = timer;
30     flatten[timer++] = u;
31     for (auto &v: al[u]) {
32         if (v == p) continue;
33         pre_dfs(v, u);
34         sz[u] += sz[v];
35     }
36 }
37
38 void dfs(int u, int p, bool keep) {
39     int mx = -1, big = -1;
40     for (auto &v: al[u]) { // get big child
41         if (v != p && sz[v] > mx) {
42             big = v;
43             mx = sz[v];
44         }
45     }
46
47     for (auto &v: al[u]) { // process smaller child first
48         if (v == p || v == big) continue;
49         dfs(v, u, false); // process smaller subtree first
50     }
51
52     if (!big) { // solve big child
53         dfs(big, u, true);
54     }
55
56     // add small children
57     add(u);
58     for (auto &v: al[u]) {
59         if (v == p || v == big) continue;
60         int l = in[v], r = in[v] + sz[v];
61         for (int t = l; t < r; t++) {
62             add(flatten[t]);
63         }
64     }
65
66     ans[u] = get_answer(); // get answer
67
68     if (!keep) { // remove subtree u
69         int l = in[u], r = in[u] + sz[u];
70         for (int t = l; t < r; t++) {
71             remove(flatten[t]);
72         }
73     }
74 }

```

```

75
76 void compression(vector<int> &a, int &n) {
77     vector<int> comp(a.begin() + 1, a.end()); // skip index 0 (1-based)
78
79     sort(comp.begin(), comp.end());
80     comp.erase(unique(comp.begin(), comp.end()), comp.end());
81
82     for (int i = 1; i <= n; i++) {
83         a[i] = lower_bound(comp.begin(), comp.end(), a[i]) - comp.begin() + 1;
84     }
85     n = comp.size();
86 }
87
88 int main() {
89     ios::sync_with_stdio(false);
90     cin.tie(nullptr);
91
92     int n;
93     cin >> n;
94
95     in.assign(n + 1, 0);
96     sz.assign(n + 1, 0);
97     frq.assign(n + 1, 0);
98     col.assign(n + 1, 0);
99     ans.assign(n + 1, 0);
100    flatten.assign(n + 5, 0);
101    al.assign(n + 1, vector<int>());
102
103    for (int u = 1; u <= n; u++) {
104        cin >> col[u];
105    }
106
107    int sz = n;
108    compression(col, sz);
109
110    for (int i = 1; i < n; i++) {
111        int u, v;
112        cin >> u >> v;
113        al[u].push_back(v);
114        al[v].push_back(u);
115    }
116
117    pre_dfs(1, 0);
118
119    dfs(1, 0, true);
120
121    for (int u = 1; u <= n; u++) {
122        cout << ans[u] << ' ';
123    }
124
125    return 0;
126 }

```

8 Math

8.1 bezout_sIdentity

```

1 // Extended Euclidean Algorithm
2 // Returns gcd(a, b), and finds x and y such that ax + by = gcd(a, b)
3 int extended_gcd(int a, int b, int &x, int &y) {
4     if (b == 0) {
5         x = 1, y = 0;
6         return a;
7     }
8     int x1, y1;
9     int g = extended_gcd(b, a % b, x1, y1);
10    x = y1, y = x1 - (a / b) * y1;

```

```

11     return g;
12 }
13
14 pair<int, int> clc(int a, int b) {
15     int x, y, g;
16     g = extended_gcd(a, b, x, y);
17     return make_pair(x, y);
18 }

```

8.2 Fast_Power

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long mod = 1e9 + 7;
5
6 long long fp(long long b, long long p) {
7     long long res = 1;
8     while (p) {
9         if (p & 1) {
10             res = (res * b) % mod;
11         }
12         b = (b * b) % mod;
13         p >>= 1;
14     }
15     return res;
16 }
17
18 int main() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22
23     return 0;
24 }

```

8.3 MatrixExpo

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int mod = 1e9 + 7, matrix_size = 2;
5
6 struct matrix {
7     vector<vector<long long>> mat;
8
9     matrix() {
10         mat.assign(matrix_size, vector<long long>(matrix_size));
11     }
12
13     matrix friend operator *(const matrix &a, const matrix &b) {
14         matrix c;
15         for (int i = 0; i < matrix_size; i++) {
16             for (int j = 0; j < matrix_size; j++) {
17                 c.mat[i][j] = 0;
18                 for (int u = 0; u < matrix_size; u++) {
19                     c.mat[i][j] += (a.mat[i][u] * b.mat[u][j]) % mod;
20                     c.mat[i][j] %= mod;
21                 }
22             }
23         }
24         return c;
25     }
26
27     matrix friend operator +(const matrix &a, const matrix &b) {
28         matrix c;

```

```

29     for (int i = 0; i < matrix_size; i++) {
30         for (int j = 0; j < matrix_size; j++) {
31             c.mat[i][j] = (a.mat[i][j] + b.mat[i][j]) % mod;
32             c.mat[i][j] %= mod;
33         }
34     }
35     return c;
36 }
37 };
38
39 matrix fp(matrix b, long long p) {
40     matrix r;
41     for (int i = 0; i < matrix_size; i++) {
42         r.mat[i][i] = 1;
43     }
44     while (p) {
45         if (p & 1) r = r * b;
46         b = b * b;
47         p >>= 1;
48     }
49     return r;
50 }
51
52 int main() {
53     ios::sync_with_stdio(false);
54     cin.tie(nullptr);
55
56     auto mul = [&](vector<int> &dp, matrix &mat) {
57         vector<int> new_dp(matrix_size);
58         for (int r = 0; r < matrix_size; r++) {
59             const auto &row = mat.mat[r];
60             for (int c = 0; c < matrix_size; c++) {
61                 new_dp[c] = (new_dp[c] + 1LL * dp[r] * row[c]) % mod;
62             }
63         }
64         return new_dp;
65     };
66
67     return 0;
68 }

```

8.4 Mod INV by phi

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 1e5 + 5;
5
6  int pw[N], ipw[N], mod_inv, mod;
7
8  int phi(int n) {
9      int ret = n;
10     for (int i = 2; i * i <= n; ++i) {
11         if (n % i == 0) {
12             while (n % i == 0) {
13                 n /= i;
14             }
15             ret -= ret / i;
16         }
17     }
18     if (n > 1) ret -= ret / n;
19     return ret;
20 }
21
22 long long fp(long long b, long long p) {
23     long long res = 1;
24     while (p) {

```

```

25         if (p & 1) {
26             res = (res * b) % mod;
27         }
28         b = (b * b) % mod;
29         p >>= 1;
30     }
31     return res;
32 }
33
34 void preClc() {
35     mod_inv = phi(mod) - 1;
36     pw[0] = ipw[0] = 1;
37     for (int i = 1; i < N; i++) {
38         pw[i] = (10LL * pw[i - 1]) % mod;
39         ipw[i] = fp(pw[i], mod_inv);
40     }
41 }

```

8.5 mod inverse for not prime

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // mod is NOT prime
5
6  long long egcd(long long a, long long b, long long &x, long long &y) {
7      if (b == 0) {
8          x = 1;
9          y = 0;
10         return a;
11     }
12     long long x1, y1;
13     long long g = egcd(b, a % b, x1, y1);
14     x = y1;
15     y = x1 - y1 * (a / b);
16     return g;
17 }
18
19 long long mod_inv(long long a, long long mod) {
20     long long x, y;
21     long long g = egcd(a, mod, x, y);
22     if (g != 1) return -1;
23     x = (x % mod + mod) % mod;
24     return x;
25 }

```

8.6 NTT

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 1 << 18;
5  const int mod = 998244353;
6  const int root = 3;
7  int lim, rev[N], w[N], wn[N], inv_lim;
8  void reduce(int &x) { x = (x + mod) % mod; }
9  int POW(int x, int y, int ans = 1) {
10     for (; y; y >>= 1, x = (long long) x * x % mod) if (y & 1) ans = (long long) ans * x % mod;
11     return ans;
12 }
13 void precompute(int len) {
14     lim = wn[0] = 1; int s = -1;
15     while (lim < len) lim <= 1, ++s;
16     for (int i = 0; i < lim; ++i) rev[i] = rev[i >> 1] >> 1 | (i & 1) << s;
17     const int g = POW(root, (mod - 1) / lim);
18     inv_lim = POW(lim, mod - 2);

```

```

19 for (int i = 1; i < lim; ++i) wn[i] = (long long) wn[i - 1] * g % mod;
20 }
21 void ntt(vector<int> &a, int typ) {
22     for (int i = 0; i < lim; ++i) if (i < rev[i]) swap(a[i], a[rev[i]]);
23     for (int i = 1; i < lim; i <= 1) {
24         for (int j = 0, t = lim / i / 2; j < i; ++j) w[j] = wn[j * t];
25         for (int j = 0; j < lim; j += i < 1) {
26             for (int k = 0; k < i; ++k) {
27                 const int x = a[k + j], y = (long long) a[k + j + i] * w[k] % mod;
28                 reduce(a[k + j] += y - mod), reduce(a[k + j + i] = x - y);
29             }
30         }
31     }
32     if (!typ) {
33         reverse(a.begin() + 1, a.begin() + lim);
34         for (int i = 0; i < lim; ++i) a[i] = (long long) a[i] * inv_lim % mod;
35     }
36 }
37 vector<int> multiply(vector<int> &f, vector<int> &g) {
38     int n=(int)f.size() + (int)g.size() - 1;
39     precompute(n);
40     vector<int> a = f, b = g;
41     a.resize(lim); b.resize(lim);
42     ntt(a, 1), ntt(b, 1);
43     for (int i = 0; i < lim; ++i) a[i] = (long long) a[i] * b[i] % mod;
44     ntt(a, 0);
45     //while((int)a.size() && a.back() == 0) a.pop_back();
46     return a;
47 }
48 int fact[N], ifact[N];
49 vector<int> shift(vector<int> &f, int c) { //f(x + c)
50     int n=(int)f.size();
51     precompute(n + n - 1);
52     vector<int> a = f; a.resize(lim);
53     for (int i = 0; i < n; ++i) a[i] = (long long) a[i] * fact[i] % mod;
54     reverse(a.begin(), a.begin() + n);
55     vector<int> b; b.resize(lim); b[0] = 1;
56     for (int i = 1; i < n; ++i) b[i] = (long long) b[i - 1] * c % mod;
57     for (int i = 0; i < n; ++i) b[i] = (long long) b[i] * ifact[i] % mod;
58     ntt(a, 1), ntt(b, 1);
59     for (int i = 0; i < lim; ++i) a[i] = (long long) a[i] * b[i] % mod;
60     ntt(a, 0), reverse(a.begin(), a.begin() + n);
61     vector<int> g; g.resize(n);
62     for (int i = 0; i < n; ++i) g[i] = (long long) a[i] * ifact[i] % mod;
63     return g;
64 }
65 vector<int> range_mul(int n) { //(x+1)*(x+2)*(x+3)...(x+n)
66     if (n == 0) return vector<int>({1});
67     if (n & 1) {
68         vector<int> f = range_mul(n - 1);
69         f.push_back(0);
70         for (int i = (int)f.size()-1; i; --i) f[i] = (f[i - 1] + (long long) n * f[i]) % mod;
71         f[0] = (long long) f[0] * n % mod;
72         return f;
73     }
74     else {
75         int n_ = n >> 1;
76         vector<int> f = range_mul(n_);
77         vector<int> tmp = shift(f, n_);
78         f.resize(n_ + 1);
79         tmp.resize(n_ + 1);
80         return multiply(f, tmp);
81     }
82 }
83 int f(int n, int k) {
84     if (n == 0 && k == 0) return 1;
85     if (n <= 0 || k <= 0) return 0;
86     vector<int> x = vector<int>({0, 1});

```

```

87 vector<int> y = range_mul(n - 1);
88 vector<int> ans = multiply(x, y);
89 if (k >= (int)ans.size()) return 0;
90 return ans[k];
91 }
92 int ncr(int n, int r) {
93     if (r < 0 || n < r) return 0;
94     return 1LL * fact[n] * ifact[r] % mod * ifact[n - r] % mod;
95 }
96 int main() {
97     fact[0] = 1;
98     for (int i = 1; i < N; ++i) fact[i] = (long long) fact[i - 1] * i % mod;
99     ifact[N - 1] = POW(fact[N - 1], mod - 2);
100     for (int i = N - 1; i; --i) ifact[i - 1] = (long long) ifact[i] * i % mod;
101     int n, a, b; cin >> n >> a >> b;
102     cout << 1LL * f(n - 1, a + b - 2) * ncr(a + b - 2, a - 1) % mod << '\n';
103     return 0;
104 }
105 //https://codeforces.com/problemset/problem/960/G

```

8.7 XOR Basis

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int bits = 60;
5
6 template<typename T>
7 struct basis {
8     T b[bits]{};
9     int sz = 0;
10
11     void insert(T x) {
12         for (int bit = bits - 1; bit >= 0; bit--) {
13             if (~x & (1LL << bit)) continue;
14             if (!b[bit]) {
15                 sz++;
16                 b[bit] = x;
17                 break;
18             }
19             x ^= b[bit];
20         }
21     }
22
23     bool exist(T x) {
24         for (int bit = bits - 1; bit >= 0; bit--) {
25             if (~x & (1LL << bit)) continue;
26             if (!b[bit]) {
27                 return false;
28             }
29             x ^= b[bit];
30         }
31         return true;
32     }
33
34     T get_max(T x) {
35         for (int bit = bits - 1; bit >= 0; bit--) {
36             x = max(x, x ^ b[bit]);
37         }
38         return x;
39     }
40
41     T get_min(T x) {
42         for (int bit = bits - 1; bit >= 0; bit--) {
43             x = min(x, x ^ b[bit]);
44         }
45         return x;

```

```

46     }
47
48     // it ensures that for every basis, its highest set bit is unique, and no other basis has a
49     // '1' in this bit position.
50     void reduce() {
51         for (int bit = 0; bit < bits; bit++) {
52             if (!b[bit]) continue;
53             for (int j = bit + 1; j < bits; j++) {
54                 if (b[j] & (1LL << bit)) {
55                     b[j] ^= b[bit];
56                 }
57             }
58         }
59     }
60     void merge(basis<int> &other) {
61         for (int bit = bits - 1; bit >= 0; bit--) {
62             insert(other.b[bit]);
63         }
64     }
65
66     // TODO mod
67     // sum of all distinct XOR values
68     T sum() {
69         T orz = 0;
70         for (int bit = bits - 1; bit >= 0; bit--) {
71             orz |= b[bit];
72             orz %= mod;
73         }
74         return (orz * fp(2, sz - 1)) % mod;
75     }
76 };

```

9 Number Theory

9.1 CRT

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4
5  typedef long long ll;
6
7  array<ll, 3> extend_euclid(ll a, ll b) {
8      array<ll, 3> x = {1, 0, a};
9      array<ll, 3> y = {0, 1, b};
10     while (y[2] > 0) {
11         ll k = x[2] / y[2];
12         for (int i = 0; i < 3; i++) { x[i] -= k * y[i]; }
13         swap(x, y);
14     }
15     return x;
16 }
17
18 ll inv(ll a, ll m) {
19     auto res = extend_euclid(a, m);
20     if (res[2] != 1) return -1; // Inverse doesn't exist
21     return (res[0] % m + m) % m;
22 }
23
24 // Merges two congruences: x = a1 (mod m1) and x = a2 (mod m2)
25 pair<ll, ll> merge(ll a1, ll m1, ll a2, ll m2) {
26     auto res = extend_euclid(m1, m2);
27     ll g = res[2];
28
29     // Consistency check: (a2 - a1) must be divisible by gcd(m1, m2)
30     if ((a2 - a1) % g != 0) return {-1, -1};

```

```

31
32     ll mod = (m1 / g) * m2; // This is LCM(m1, m2)
33
34     // We solve m1*k = (a2 - a1) (mod m2)
35     // k = (a2 - a1)/g * inv(m1/g, m2/g) (mod m2/g)
36     ll factor = (a2 - a1) / g;
37     ll m2_reduced = m2 / g;
38     ll k = (factor % m2_reduced * inv(m1 / g, m2_reduced)) % m2_reduced;
39     if (k < 0) k += m2_reduced;
40
41     ll x = (a1 + m1 * k) % mod;
42     return {x, mod};
43 }
44
45 pair<ll, ll> general_crt(const vector<ll> &remainders, const vector<ll> &moduli) {
46     ll cur_a = remainders[0];
47     ll cur_m = moduli[0];
48
49     for (int i = 1; i < (int)moduli.size(); i++) {
50         auto next = merge(cur_a, cur_m, remainders[i], moduli[i]);
51         if (next.second == -1) return {-1, -1}; // Inconsistent system
52         cur_a = next.first;
53         cur_m = next.second;
54     }
55     return {cur_a, cur_m};
56 }

```

9.2 divisors

```

1  const int N = 2e5;
2  vector<vector<int>> divisor(N + 1);
3
4  void divisors() {
5      for (int i = 2; i <= N; ++i) {
6          for (int j = i; j <= N; j += i) {
7              divisor[j].push_back(i);
8          }
9      }
10 }

```

9.3 Euler_s_totient

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  //TODO
5  const int N = 1e5;
6
7  vector<int> phi(N);
8
9  void phi_1_to_n() {
10     phi[0] = 0, phi[1] = 1;
11     for (int i = 2; i < N; i++) {
12         phi[i] = i - 1;
13     }
14
15     for (int i = 2; i < N; i++) {
16         for (int j = 2 * i; j < N; j += i) {
17             phi[j] -= phi[i];
18         }
19     }
20 }

```

9.4 is_prime

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Sieve of Eratosthenes O(n log long n)
5
6 vector<bool> isPrime(N + 1, true);
7
8 void sieve() {
9     isPrime[0] = isPrime[1] = false;
10    for (int i = 2; i <= N; i++) {
11        if (isPrime[i]) {
12            primes.push_back(i);
13            if (1LL * i * i <= N) {
14                for (int j = i * i; j <= N; j += i) {
15                    isPrime[j] = false;
16                }
17            }
18        }
19    }
20 }

```

9.5 linearSieve

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int N = 10000000;
5 vector<int> lp(N + 1);
6 vector<int> pr;
7
8 void linearSieve() {
9     for (int i = 2; i <= N; ++i) {
10        if (lp[i] == 0) {
11            lp[i] = i;
12            pr.push_back(i);
13        }
14        for (int j = 0; i * pr[j] <= N; ++j) {
15            lp[i * pr[j]] = pr[j];
16            if (pr[j] == lp[i]) {
17                break;
18            }
19        }
20    }
21 }
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27
28     return 0;
29 }

```

9.6 Miller-Rabin

```

1 using u64 = uint64_t;
2 using u128 = __uint128_t;
3
4 u64 binpower(u64 base, u64 e, u64 mod) {
5     u64 result = 1;
6     base %= mod;
7     while (e) {
8         if (e & 1)
9             result = (u128)result * base % mod;
10        base = (u128)base * base % mod;

```

```

11        e >>= 1;
12    }
13    return result;
14 }
15
16 bool check_composite(u64 n, u64 a, u64 d, int s) {
17     u64 x = binpower(a, d, n);
18     if (x == 1 || x == n - 1)
19         return false;
20     for (int r = 1; r < s; r++) {
21         x = (u128)x * x % n;
22         if (x == n - 1)
23             return false;
24     }
25     return true;
26 };
27
28 bool MillerRabin(u64 n, int iter=5) { // returns true if n is probably prime, else returns false
29
30     if (n < 4)
31         return n == 2 || n == 3;
32
33     int s = 0;
34     u64 d = n - 1;
35     while ((d & 1) == 0) {
36         d >>= 1;
37         s++;
38     }
39     for (int i = 0; i < iter; i++) {
40         int a = 2 + rand() % (n - 3);
41         if (check_composite(n, a, d, s))
42             return false;
43     }
44     return true;
45 }

```

9.7 Pollard Rho

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5 namespace PollardRho {
6     mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
7     const int P = 1e6 + 9;
8     ll seq[P];
9     int primes[P], spf[P];
10    inline ll add_mod(ll x, ll y, ll m) {
11        return (x + y) < m ? x + y : x - m;
12    }
13    inline ll mul_mod(ll x, ll y, ll m) {
14        ll res = __int128(x) * y % m;
15        return res;
16        // ll res = x * y - (ll)((long double)x * y / m + 0.5) * m;
17        // return res < 0 ? res + m : res;
18    }
19    inline ll pow_mod(ll x, ll n, ll m) {
20        ll res = 1 % m;
21        for (; n >>= 1) {
22            if (n & 1) res = mul_mod(res, x, m);
23            x = mul_mod(x, x, m);
24        }
25        return res;
26    }
27    // O(it * (logn)^3), it = number of rounds performed
28    inline bool miller_rabin(ll n) {

```

```

29     if (n <= 2 || (n & 1 ^ 1)) return (n == 2);
30     if (n < P) return spf[n] == n;
31     ll c, d, s = 0, r = n - 1;
32     for (; !(r & 1); r >>= 1, s++) {}
33     // each iteration is a round
34     for (int i = 0; primes[i] < n && primes[i] < 32; i++) {
35         c = pow_mod(primes[i], r, n);
36         for (int j = 0; j < s; j++) {
37             d = mul_mod(c, c, n);
38             if (d == 1 && c != 1 && c != (n - 1)) return false;
39             c = d;
40         }
41         if (c != 1) return false;
42     }
43     return true;
44 }
45 void init() {
46     int cnt = 0;
47     for (int i = 2; i < P; i++) {
48         if (!spf[i]) primes[cnt++] = spf[i] = i;
49         for (int j = 0, k; (k = i * primes[j]) < P; j++) {
50             spf[k] = primes[j];
51             if (spf[i] == spf[k]) break;
52         }
53     }
54 }
55 // returns O(n^(1/4))
56 ll pollard_rho(ll n) {
57     while (1) {
58         ll x = rnd() % n, y = x, c = rnd() % n, u = 1, v, t = 0;
59         ll *px = seq, *py = seq;
60         while (1) {
61             *py++ = y = add_mod(mul_mod(y, y, n), c, n);
62             *py++ = y = add_mod(mul_mod(y, y, n), c, n);
63             if ((x = *px++) == y) break;
64             v = u;
65             u = mul_mod(u, abs(y - x), n);
66             if (!u) return __gcd(v, n);
67             if (++t == 32) {
68                 t = 0;
69                 if ((u = __gcd(u, n)) > 1 && u < n) return u;
70             }
71         }
72         if (t && (u = __gcd(u, n)) > 1 && u < n) return u;
73     }
74 }
75 vector<ll> factorize(ll n) {
76     if (n == 1) return vector<ll>();
77     if (miller_rabin(n)) return vector<ll> {n};
78     vector<ll> v, w;
79     while (n > 1 && n < P) {
80         v.push_back(spf[n]);
81         n /= spf[n];
82     }
83     if (n >= P) {
84         ll x = pollard_rho(n);
85         v = factorize(x);
86         w = factorize(n / x);
87         v.insert(v.end(), w.begin(), w.end());
88     }
89     return v;
90 }
91 }
92 int32_t main() {
93     ios_base::sync_with_stdio(0);
94     cin.tie(0);
95     PollardRho::init();
96     int t; cin >> t;

```

```

p7 while (t--) {
98     ll n; cin >> n;
99     auto f = PollardRho::factorize(n);
100     sort(f.begin(), f.end());
101     cout << f.size() << ' ';
102     for (auto x: f) cout << x << ' '; cout << '\n';
103 }
104 return 0;
105 }
106 // https://judge.yosupo.jp/problem/factorize

```

9.8 PrimeFactorization

```

1 const int N = 10000000;
2 vector<int> lp(N + 1);
3 vector<int> pr;
4
5 void linearSieve() {
6     for (int i = 2; i <= N; ++i) {
7         if (lp[i] == 0) {
8             lp[i] = i;
9             pr.push_back(i);
10        }
11        for (int j = 0; i * pr[j] <= N; ++j) {
12            lp[i * pr[j]] = pr[j];
13            if (pr[j] == lp[i]) {
14                break;
15            }
16        }
17    }
18 }
19
20 vector<int> primeFactors(int n) {
21     vector<int> ret;
22     while (n > 1) {
23         int p = lp[n];
24         ret.push_back(p);
25         while (n % p == 0) n /= p;
26     }
27     return ret;
28 }

```

10 Random

10.1 random_range

```

1 mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());
2
3 int rand(int l, int r) {
4     return l + rng() % (r - l + 1);
5 }

```

11 String Algorithms

11.1 ahoCorasick

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> ans;
5
6 struct ahoCorasick {
7     struct node {

```

```

8     int cnt = 0;
9     int nxt[26]{};
10
11     int &operator[](int c) {
12         return nxt[c];
13     }
14 };
15
16 vector<node> t;
17 vector<int> pi, out_link;
18 vector<vector<int>> leafs, al;
19
20 int new_node() {
21     pi.push_back(0);
22     t.push_back(node());
23     leafs.emplace_back();
24     out_link.push_back(0);
25     return (int) t.size() - 1;
26 }
27
28 ahoCorasick() {
29     t.clear();
30     new_node();
31 }
32
33 void insert(string &s, int idx) {
34     int u = 0;
35     for (auto &c: s) {
36         if (!t[u][c - 'a']) {
37             int id = new_node();
38             t[u][c - 'a'] = id;
39         }
40         u = t[u][c - 'a'];
41     }
42     leafs[u].push_back(idx);
43 }
44
45 void build() {
46     queue<int> q;
47     q.push(0);
48     while (!q.empty()) {
49         auto u = q.front();
50         q.pop();
51         for (int c = 0; c < 26; c++) {
52             int v = t[u][c];
53             if (v) {
54                 pi[v] = (u ? t[pi[u]][c] : 0);
55                 out_link[v] = leafs[pi[v]].empty() ? out_link[pi[v]] : pi[v];
56                 q.push(v);
57             } else {
58                 t[u][c] = t[pi[u]][c];
59             }
60         }
61     }
62
63     al.assign(t.size(), vector<int>());
64     for (int u = 1; u < t.size(); u++) {
65         if (!leafs[u].empty()) {
66             al[out_link[u]].push_back(u);
67         }
68     }
69 }
70
71 void search(string &s) {
72     int u = 0;
73     for (auto &c: s) {
74         u = t[u][c - 'a'];
75         if (leafs[u].empty()) {

```

```

76             t[out_link[u]].cnt++;
77         } else {
78             t[u].cnt++;
79         }
80     }
81 }
82
83 int dfs(int u) {
84     int ret = t[u].cnt;
85     for (auto &v: al[u]) {
86         ret += dfs(v);
87     }
88
89     for (auto &it: leafs[u]) {
90         ans[it] = ret;
91     }
92
93     return ret;
94 }
95 };
96
97 int main() {
98     ios::sync_with_stdio(false);
99     cin.tie(nullptr);
100
101     ahoCorasick aho;
102
103     int n;
104     string s;
105     cin >> s >> n;
106     ans.assign(n + 1, 0);
107     for (int i = 0; i < n; i++) {
108         string t;
109         cin >> t;
110         aho.insert(t, i);
111     }
112
113     aho.build();
114     aho.search(s);
115     aho.dfs(0);
116
117     for (int i = 0; i < n; i++) {
118         cout << ans[i] << '\n';
119     }
120
121     return 0;
122 }

```

11.2 compute_automaton

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> prefix_function(string &s) {
5     int n = s.size();
6     vector<int> pi(n);
7     for (int i = 1, j = 0; i < n; i++) {
8         if (s[i] == s[j]) {
9             pi[i] = ++j;
10            i++;
11        } else if (!j) {
12            i++;
13        } else {
14            j = pi[j - 1];
15        }
16    }
17    return pi;

```

```

18 }
19
20 vector<vector<int> > compute_automaton(string &s) {
21     int n = s.size();
22     auto pi = prefix_function(s);
23     vector<vector<int> > aut(n, vector<int>(26));
24     for (int len = 0; len < n; len++) {
25         for (char c = 'a'; c <= 'z'; c++) {
26             if (s[len] == c) {
27                 aut[len][c - 'a'] = len + 1;
28             } else {
29                 aut[len][c - 'a'] = (len > 0 ? aut[pi[len - 1]][c - 'a'] : 0);
30             }
31         }
32     }
33     return aut;
34 }

```

11.3 Hashing

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 const int N = 1e6 + 5, MODS = 2;
5 const array<int, MODS> mod = {1000003111, 1000003999};
6
7 array<int, MODS> p = {137, 277}, ip;
8 array<int, MODS> pw[N], ipw[N];
9
10 int fp(int b, int p, int mod) {
11     int r = 1;
12     while (p) {
13         if (p & 1) {
14             r = (1LL * r * b) % mod;
15         }
16         b = (1LL * b * b) % mod;
17         p >>= 1;
18     }
19     return r;
20 }
21
22 // TODO DONT MISS CALL IT
23 void preCalc() {
24     for (int j = 0; j < MODS; ++j) {
25         ipw[0][j] = pw[0][j] = 1;
26         ip[j] = fp(p[j], mod[j] - 2, mod[j]);
27     }
28
29     for (int i = 1; i < N; i++) {
30         for (int j = 0; j < MODS; ++j) {
31             pw[i][j] = (1LL * p[j] * pw[i - 1][j]) % mod[j];
32             ipw[i][j] = (1LL * ip[j] * ipw[i - 1][j]) % mod[j];
33         }
34     }
35 }
36
37 struct Hashing {
38     int n;
39     vector<array<int, MODS> > hs, hr;
40
41     Hashing() {
42     }
43
44     Hashing(string &s) {
45         n = (int) s.size();
46         hs.assign(n, array<int, MODS>{});
47         hr.assign(n, array<int, MODS>{});

```

```

48
49     // hash(abcd) = a*p^0 + b*p^1 + c*p^2 + d*p^3
50     for (int i = 0; i < n; i++) {
51         for (int j = 0; j < MODS; j++) {
52             hs[i][j] = ((i ? hs[i - 1][j] : 0) + (1LL * pw[i][j] * s[i]) % mod[j]) % mod[j];
53         }
54     }
55
56     // rev_hash(abcd) = a*p^3 + b*p^2 + c*p^1 + d*p^0
57     for (int i = 0; i < n; i++) {
58         for (int j = 0; j < MODS; j++) {
59             hr[i][j] = ((i ? hr[i - 1][j] : 0) + (1LL * pw[n - i - 1][j] * s[i]) % mod[j]) %
mod[j];
60         }
61     }
62 }
63
64 // l, r 0-Based
65 array<int, MODS> get_hash(int l, int r) {
66     array<int, MODS> ret;
67     for (int j = 0; j < MODS; j++) {
68         ret[j] = ((hs[r][j] - (l ? hs[l - 1][j] : 0)) % mod[j] + mod[j]) % mod[j];
69         ret[j] = (1LL * ret[j] * ipw[l][j]) % mod[j];
70     }
71     return ret;
72 }
73
74 array<int, MODS> get_rev_hash(int l, int r) {
75     array<int, MODS> ret;
76     for (int j = 0; j < MODS; j++) {
77         ret[j] = ((hr[r][j] - (l ? hr[l - 1][j] : 0)) % mod[j] + mod[j]) % mod[j];
78         ret[j] = (1LL * ret[j] * ipw[n - r + 1][j]) % mod[j];
79     }
80     return ret;
81 }
82
83 };
84
85 int main() {
86     ios_base::sync_with_stdio(false);
87     cin.tie(nullptr);
88
89     preCalc();
90
91     return 0;

```

11.4 Hashing2D

```

1 // find the number of occurences of an XxY pattern
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const int N = 3e5 + 9;
7
8 struct Hashing {
9     vector<vector<int> > hs;
10    vector<int> PWX, PWY;
11    int n, m;
12    static const int PX = 3737, PY = 3999, mod = 998244353;
13
14    Hashing() {
15    }
16
17    Hashing(vector<string> &s) {
18        n = (int) s.size(), m = (int) s[0].size();
19        hs.assign(n + 1, vector<int>(m + 1, 0));

```

```

20     PWX.assign(n + 1, 1);
21     PWY.assign(m + 1, 1);
22     for (int i = 0; i < n; i++) PWX[i + 1] = 1LL * PWX[i] * PX % mod;
23     for (int i = 0; i < m; i++) PWY[i + 1] = 1LL * PWY[i] * PY % mod;
24     for (int i = 0; i < n; i++) {
25         for (int j = 0; j < m; j++) {
26             // TODO
27             hs[i + 1][j + 1] = s[i][j] - 'a' + 1;
28         }
29     }
30     for (int i = 0; i <= n; i++) {
31         for (int j = 0; j < m; j++) {
32             hs[i][j + 1] = (hs[i][j + 1] + 1LL * hs[i][j] * PY % mod) % mod;
33         }
34     }
35     for (int i = 0; i < n; i++) {
36         for (int j = 0; j <= m; j++) {
37             hs[i + 1][j] = (hs[i + 1][j] + 1LL * hs[i][j] * PX % mod) % mod;
38         }
39     }
40 }
41
42 int get_hash(int x1, int y1, int x2, int y2) {
43     // 1-indexed
44     assert(1 <= x1 && x1 <= x2 && x2 <= n);
45     assert(1 <= y1 && y1 <= y2 && y2 <= m);
46     x1--;
47     y1--;
48     int dx = x2 - x1, dy = y2 - y1;
49     return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1] * PWY[dy] % mod + mod) % mod -
50           1LL * (hs[x1][y2] - 1LL * hs[x1][y1] * PWY[dy] % mod + mod) % mod * PWX[dx] %
51           mod + mod) % mod;
52 }
53
54 int get_hash() {
55     return get_hash(1, 1, n, m);
56 }
57
58 };
59
60 int main() {
61     ios::sync_with_stdio(false);
62     cin.tie(nullptr);
63
64     int n, m;
65     cin >> n >> m;
66     vector<string> grid(n);
67     for (int i = 0; i < n; i++) {
68         cin >> grid[i];
69     }
70
71     Hashing H(grid);
72
73     int x, y;
74     cin >> x >> y;
75     vector<string> g(x);
76     for (int i = 0; i < x; i++) {
77         cin >> g[i];
78     }
79
80     auto ret = Hashing(g).get_hash();
81
82     long long ans = 0;
83     for (int i = 1; i + x - 1 <= n; i++) {
84         for (int j = 1; j + y - 1 <= m; j++) {
85             ans += (ret == H.get_hash(i, j, i + x - 1, j + y - 1));
86         }
87     }

```

```

87     cout << ans;
88
89     return 0;
90 }

```

11.5 KMP for integer

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> prefix_function(vector<int> &v) {
5      int n = (int) v.size();
6      vector<int> pi(n);
7      for (int i = 1, j = 0; i < n; i++) {
8          if (v[i] == v[j]) {
9              pi[i] = ++j;
10             i++;
11         } else if (!j) {
12             i++;
13         } else {
14             j = pi[j - 1];
15         }
16     }
17     return pi;
18 }
19
20 int kmp(vector<int> &v, vector<int> &p) {
21     int n = v.size(), m = p.size(), cnt = 0;
22     if (m > n) return 0;
23
24     vector<int> pi = prefix_function(p);
25
26     for (int i = 0, j = 0; i < n; i++) {
27         if (v[i] == p[j]) {
28             i++, j++;
29             if (j == m) {
30                 cnt++;
31                 j = pi[j - 1];
32             }
33         } else if (!j) {
34             i++;
35         } else {
36             j = pi[j - 1];
37         }
38     }
39     return cnt;
40 }

```

11.6 KMP

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // pi[i] is the length of the longest proper prefix of the substring s[0...i] which is also a
5  // suffix of this substring.
6  // A proper prefix of a string is a prefix that is not equal to the string itself
7
8  // we can get all porder for string by
9
10 // int k = lps.back();
11 // while (k > 0) {
12 //     porders.push_back(k);
13 //     k = pi[k - 1];
14 // }
15
16 // if len of string is len n, and porder is len k, the period is n - k

```

```

16
17 vector<int> prefix_function(string &s) {
18     int n = (int) s.size();
19     vector<int> pi(n);
20     for (int i = 1, j = 0; i < n; i++) {
21         if (s[i] == s[j]) {
22             pi[i] = ++j;
23             i++;
24         } else if (!j) {
25             i++;
26         } else {
27             j = pi[j - 1];
28         }
29     }
30     return pi;
31 }
32
33 int kmp(string &s, string &p) {
34     int n = s.size(), m = p.size(), cnt = 0;
35     if (m > n) return 0;
36     vector<int> pi = prefix_function(p);
37     for (int i = 0, j = 0; i < n; i++) {
38         if (s[i] == p[j]) {
39             i++, j++;
40             if (j == m) {
41                 cnt++;
42                 j = pi[j - 1];
43             }
44         } else if (!j) {
45             i++;
46         } else {
47             j = pi[j - 1];
48         }
49     }
50     return cnt;
51 }

```

11.7 SuffixArrayWithSparseTable

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SuffixArray {
5     vector<int> sa; // sa[i] starting index of i-th smallest suffix
6     vector<int> lcp; // lcp[i] LCP of suffixArray[i] and suffixArray[i + 1]
7     vector<int> rank; // rank[i] position of suffix starting at i in suffixArray
8
9     // for sparse table
10    int sz;
11    vector<int> logs;
12    vector<vector<int>> > t;
13
14    SuffixArray() {}
15
16    SuffixArray(string &s) {
17        sz = (int) s.size();
18
19        // TODO
20        s += ' ';
21        sa = build_SuffixArray(s);
22
23        rank.resize(s.size());
24        for (int i = 0; i < s.size(); i++) {
25            rank[sa[i]] = i;
26        }
27    }
28

```

```

29    lcp = build_lcp(s);
30
31    pre_lcp();
32 }
33
34 void count_sort(vector<int> &p, vector<int> &c) {
35     int n = (int) p.size();
36
37     vector<int> cnt(n);
38     for (auto &it: c) {
39         cnt[it]++;
40     }
41
42     vector<int> pos(n);
43     for (int i = 1; i < n; i++) {
44         pos[i] = pos[i - 1] + cnt[i - 1];
45     }
46
47     vector<int> np(n);
48     for (auto &it: p) {
49         int i = c[it];
50         np[pos[i]++] = it;
51     }
52     swap(p, np);
53 }
54
55 vector<int> build_SuffixArray(string &s) {
56     int n = (int) s.size();
57
58     vector<pair<char, int>> a(n);
59     for (int i = 0; i < n; i++) {
60         a[i] = make_pair(s[i], i);
61     }
62     sort(a.begin(), a.end());
63
64     vector<int> p(n); // p[i] starting index of i-th smallest suffix
65     for (int i = 0; i < n; i++) {
66         p[i] = a[i].second;
67     }
68
69     vector<int> c(n); // c[i] class of suffix starting at index i
70     for (int i = 1; i < n; i++) {
71         c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
72     }
73
74     int k = 0;
75     while ((1 << k) < n) {
76         for (int i = 0; i < n; i++) {
77             p[i] = (p[i] - (1 << k) + n) % n;
78         }
79
80         count_sort(p, c);
81
82         vector<int> nc(n);
83         for (int i = 1; i < n; i++) {
84             pair<int, int> prev = make_pair(c[p[i - 1]], c[(p[i - 1] + (1 << k)) % n]);
85             pair<int, int> cur = make_pair(c[p[i]], c[(p[i] + (1 << k)) % n]);
86             nc[p[i]] = nc[p[i - 1]] + (cur != prev);
87         }
88         swap(nc, c);
89
90         k++;
91     }
92     return p;
93 }
94
95 vector<int> build_lcp(string &s) {
96     int n = (int) s.size(), k = 0;

```

```

97     lcp.resize(n - 1);
98     for (int i = 0; i < n; i++) {
99         if (rank[i] == n - 1) {
100             k = 0;
101             continue;
102         }
103
104         int j = sa[rank[i] + 1];
105         while (i + k < n && j + k < n && s[i + k] == s[j + k]) {
106             k++;
107         }
108
109         lcp[rank[i]] = k;
110         k = max(0, k - 1);
111     }
112     return lcp;
113 }
114
115 void pre_lcp() {
116     int n = sz + 1;
117     logs.resize(n + 1);
118     for (int i = 2; i <= n; i++) {
119         logs[i] = 1 + logs[i / 2];
120     }
121
122     t.resize(logs[n] + 1, vector<int>(n));
123     for (int i = 0; i < n; i++) {
124         // TODO 0-based
125         t[0][i] = lcp[i];
126     }
127     for (int p = 1; p <= logs[n]; p++) {
128         for (int i = 0; i + (1 << p) <= n; i++) {
129             t[p][i] = min(t[p - 1][i], t[p - 1][i + (1 << (p - 1))]);
130         }
131     }
132 }
133
134 // TODO l, r 0-based
135 // pass r - 1 because lcp sa[i] and sa[i + 1]
136 int query_lcp(int l, int r) {
137     if (l == r) {
138         return lcp[l];
139     }
140
141     // TODO
142     // l = rank[l], r = rank[r];
143     if (l > r) {
144         swap(l, r);
145     }
146
147     int k = logs[r - l + 1];
148     return min(t[k][l], t[k][r - (1 << k) + 1]);
149 }
150 }
151 };

```

11.8 SuffixArrayWithVectorInt

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct sparseTable {
5     int sz;
6     vector<int> logs;
7     vector<vector<int>> > t;
8
9     sparseTable() {

```

```

10     }
11
12     sparseTable(const vector<int> &v) {
13         sz = v.size();
14
15         logs.resize(sz + 1);
16         for (int i = 2; i <= sz; i++) {
17             logs[i] = 1 + logs[i / 2];
18         }
19
20         t.assign(logs[sz] + 1, vector<int>(sz));
21         for (int i = 0; i < sz; i++) {
22             t[0][i] = v[i];
23         }
24         for (int p = 1; p <= logs[sz]; p++) {
25             for (int i = 0; i + (1 << p) <= sz; i++) {
26                 t[p][i] = min(t[p - 1][i], t[p - 1][i + (1 << (p - 1))]);
27             }
28         }
29     }
30
31     int query(int l, int r) const {
32         int k = logs[r - l + 1];
33         //TODO
34         return min(t[k][l], t[k][r - (1 << k) + 1]);
35     }
36 };
37
38 struct SuffixArray {
39     vector<int> sa; // sa[i] is the starting index of the i-th smallest suffix
40
41     SuffixArray(const vector<int> &v) {
42         vector<int> s = v;
43
44         //TODO
45         s.push_back(-1);
46
47         sa = build_SA(s);
48     }
49
50     void radix_sort(vector<pair<pair<int, int>, int> > &a) {
51         int n = a.size(); {
52             vector<int> cnt(n);
53             for (auto &it: a) {
54                 cnt[it.first.second]++;
55             }
56
57             vector<int> pos(n);
58             for (int i = 1; i < n; i++) {
59                 pos[i] = pos[i - 1] + cnt[i - 1];
60             }
61
62             vector<pair<pair<int, int>, int> > na(n);
63             for (auto &it: a) {
64                 na[pos[it.first.second]++] = it;
65             }
66             a = na;
67         } {
68             vector<int> cnt(n);
69             for (auto &it: a) {
70                 cnt[it.first.first]++;
71             }
72
73             vector<int> pos(n);
74             for (int i = 1; i < n; i++) {
75                 pos[i] = pos[i - 1] + cnt[i - 1];
76             }
77         }

```

```

78     vector<pair<pair<int, int>, int> > na(n);
79     for (auto &it: a) {
80         na[pos[it.first.first]++] = it;
81     }
82     a = na;
83 }
84
85 vector<int> build_SA(vector<int> &s) {
86     int n = s.size();
87
88     vector<pair<int, int> > a(n);
89     for (int i = 0; i < n; i++) {
90         a[i] = make_pair(s[i], i);
91     }
92     sort(a.begin(), a.end());
93
94     vector<int> p(n), c(n);
95     for (int i = 0; i < n; i++) {
96         p[i] = a[i].second;
97     }
98
99     c[p[0]] = 0;
100    for (int i = 1; i < n; i++) {
101        c[p[i]] = c[p[i - 1]] + (a[i].first != a[i - 1].first);
102    }
103
104    int k = 0;
105    vector<pair<pair<int, int>, int> > na(n);
106    while ((1 << k) < n) {
107        for (int i = 0; i < n; i++) {
108            na[i] = make_pair(make_pair(c[i], c[(i + (1 << k)) % n]), i);
109        }
110
111        radix_sort(na);
112
113        for (int i = 0; i < n; i++) {
114            p[i] = na[i].second;
115        }
116
117        c[p[0]] = 0;
118        for (int i = 1; i < n; i++) {
119            c[p[i]] = c[p[i - 1]] + (na[i].first != na[i - 1].first);
120        }
121        k++;
122    }
123    return p;
124 }
125 };
126
127 struct LCP {
128     int n;
129     vector<int> sa, rank, lcp;
130     sparseTable st;
131
132     LCP(vector<int> &v) {
133         SuffixArray suf(v);
134
135         sa = suf.sa;
136
137         n = sa.size();
138
139         rank.resize(n);
140         for (int i = 0; i < n; i++) {
141             rank[sa[i]] = i;
142         }
143
144         build_lcp(v);
145

```

```

146         st = sparseTable(lcp);
147     }
148
149 void build_lcp(vector<int> &v) {
150     lcp.assign(n - 1, 0);
151
152     int k = 0;
153     for (int i = 0; i < n; i++) {
154         if (rank[i] == n - 1) {
155             k = 0;
156             continue;
157         }
158
159         int j = sa[rank[i] + 1];
160         while (i + k < n - 1 && j + k < n - 1 && v[i + k] == v[j + k]) k++;
161
162         lcp[rank[i]] = k;
163         k = max(0, k - 1);
164     }
165 }
166
167 // lcp of suffixes starting at i and j
168 int query(int i, int j) {
169     if (i == j) return n - 1 - i;
170     int ri = rank[i], rj = rank[j];
171     if (ri > rj) swap(ri, rj);
172     return st.query(ri, rj - 1);
173 }
174 };
175

```

11.9 Suffix_Array

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SuffixArray {
5     vector<int> sa; // sa[i] starting index of i-th smallest suffix
6     vector<int> lcp; // lcp[i] LCP of suffixArray[i] and suffixArray[i + 1]
7     vector<int> rank; // rank[i] position of suffix starting at i in suffixArray
8
9     SuffixArray() {}
10
11     SuffixArray(string &s) {
12         // TODO
13         s += '$';
14         sa = build_SuffixArray(s);
15
16         rank.resize(s.size());
17         for (int i = 0; i < s.size(); i++) {
18             rank[sa[i]] = i;
19         }
20
21         lcp = build_lcp(s);
22     }
23
24 void count_sort(vector<int> &p, vector<int> &c) {
25     int n = (int) p.size();
26
27     vector<int> cnt(n);
28     for (auto &it: c) {
29         cnt[it]++;
30     }
31
32     vector<int> pos(n);
33     for (int i = 1; i < n; i++) {
34

```

```

35     pos[i] = pos[i - 1] + cnt[i - 1];
36 }
37
38 vector<int> np(n);
39 for (auto &it: p) {
40     int i = c[it];
41     np[pos[i]++] = it;
42 }
43 swap(p, np);
44 }
45
46 vector<int> build_SuffixArray(string &s) {
47     int n = (int) s.size();
48
49     vector<pair<char, int> > a(n);
50     for (int i = 0; i < n; i++) {
51         a[i] = make_pair(s[i], i);
52     }
53     sort(a.begin(), a.end());
54
55     vector<int> p(n); // p[i] starting index of i-th smallest suffix
56     for (int i = 0; i < n; i++) {
57         p[i] = a[i].second;
58     }
59
60     vector<int> c(n); // c[i] class of suffix starting at index i
61     for (int i = 1; i < n; i++) {
62         c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
63     }
64
65     int k = 0;
66     while ((1 << k) < n) {
67         for (int i = 0; i < n; i++) {
68             p[i] = (p[i] - (1 << k) + n) % n;
69         }
70
71         count_sort(p, c);
72
73         vector<int> nc(n);
74         for (int i = 1; i < n; i++) {
75             pair<int, int> prev = make_pair(c[p[i - 1]], c[(p[i - 1] + (1 << k)) % n]);
76             pair<int, int> cur = make_pair(c[p[i]], c[(p[i] + (1 << k)) % n]);
77             nc[p[i]] = nc[p[i - 1]] + (cur != prev);
78         }
79         swap(nc, c);
80
81         k++;
82     }
83     return p;
84 }
85
86 vector<int> build_lcp(string &s) {
87     int n = (int) s.size(), k = 0;
88
89     lcp.resize(n - 1);
90     for (int i = 0; i < n; i++) {
91         if (rank[i] == n - 1) {
92             k = 0;
93             continue;
94         }
95
96         int j = sa[rank[i] + 1];
97         while (i + k < n && j + k < n && s[i + k] == s[j + k]) {
98             k++;
99         }
100
101         lcp[rank[i]] = k;
102         k = max(0, k - 1);

```

```

103     }
104     return lcp;
105 }
106 } suf;
107
108 int lower_bound(vector<int> &sa, string &s, string &pat) {
109     int n = (int) s.size(), l = 0, r = n - 1, ret = n;
110     while (l <= r) {
111         int md = (l + r) / 2;
112         if (s.substr(sa[md], min((int) pat.size(), n - sa[md])) >= pat) {
113             ret = md;
114             r = md - 1;
115         } else {
116             l = md + 1;
117         }
118     }
119     return ret;
120 }
121
122 int upper_bound(vector<int> &sa, string &s, string &pat) {
123     int n = (int) s.size(), l = 0, r = n - 1, ret = n;
124     while (l <= r) {
125         int md = (l + r) / 2;
126         if (s.substr(sa[md], min((int) pat.size(), n - sa[md])) > pat) {
127             ret = md;
128             r = md - 1;
129         } else {
130             l = md + 1;
131         }
132     }
133     return ret;
134 }
135
136 // count the number of distinct substrings that appear in a string s
137 // substring is prefix of some suffix
138 long long cnt_distinct(string &s, vector<int> &lcp) {
139     int n = (int) s.size() - 1;
140
141     long long total = (1LL * n * (n + 1)) / 2, bad = 0;
142     for (int i = 0; i < lcp.size(); i++) {
143         bad += lcp[i];
144     }
145     return total - bad;
146 }
147
148 string longest_common_substring(string &s, string &t) {
149     int n = s.size();
150
151     string comb = s + '#' + t;
152     int sz = comb.size();
153
154     SuffixArray suf(comb);
155     auto pos = suf.sa, lcp = suf.lcp;
156
157     int mx = 0, idx = 0;
158     for (int i = 0; i < sz; i++) {
159         if ((pos[i] < n == pos[i + 1] < n)) continue; // same class
160         if (lcp[i] > mx) {
161             mx = lcp[i];
162             idx = pos[i];
163         }
164     }
165     return comb.substr(idx, mx);
166 }

```

11.10 Trie

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct node {
5     map<char, int> nxt;
6
7     int &operator[](char c) {
8         return nxt[c];
9     }
10 };
11
12 struct trie {
13     vector<node> t;
14
15     int new_node() {
16         t.push_back(node());
17         return t.size() - 1;
18     }
19
20     trie() {
21         t.clear();
22         new_node();
23     }
24
25     void insert(string &s) {
26         int u = 0;
27         for (auto &c: s) {
28             if (!t[u].nxt.count(c)) { // to avoid memory limit
29                 int id = new_node();
30                 t[u][c] = id;
31             }
32             u = t[u][c];
33         }
34     }
35 };

```

11.11 Trie01

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct node {
5     int nxt[2]{}, sz = 0;
6
7     int &operator[](int x) {
8         return nxt[x];
9     }
10 };
11
12 struct trie {
13     const int bits = 31;
14     vector<node> t;
15
16     int new_node() {
17         t.push_back(node());
18         return t.size() - 1;
19     }
20
21     trie() {
22         t.clear();
23         new_node();
24     }
25
26     int sz(int u) {
27         return t[u].sz;
28     }

```

```

29
30 void update(int x, int op) {
31     int u = 0;
32     for (int bit = bits - 1; bit >= 0; bit--) {
33         int b = x >> bit & 1;
34         if (!t[u][b]) {
35             int id = new_node();
36             t[u][b] = id;
37         }
38         u = t[u][b];
39         t[u].sz += op;
40     }
41 }
42
43 // find the number in the trie that gives the maximum XOR when XOR-ed with x.
44 int query_max_xor(int x) {
45     int u = 0, ret = 0;
46     for (int bit = bits - 1; bit >= 0; bit--) {
47         int b = (x >> bit) & 1;
48         // Prefer the opposite bit for maximum XOR
49         if (t[u][b ^ 1] && sz(t[u][b ^ 1])) {
50             ret |= (1 << bit); // this bit contributes a 1 to XOR
51             u = t[u][b ^ 1];
52         } else {
53             u = t[u][b];
54         }
55     }
56     return ret;
57 }
58
59 // find the number in the trie that gives the minimum XOR when XOR-ed with a given number x
60 int query_min_xor(int x) {
61     int u = 0, ret = 0;
62     for (int bit = bits - 1; bit >= 0; bit--) {
63         int b = (x >> bit) & 1;
64         // go the same bit to minimize XOR
65         if (t[u][b] && sz(t[u][b])) {
66             u = t[u][b];
67         } else {
68             ret |= (1 << bit); // XOR bit is 1 here
69             u = t[u][b ^ 1];
70         }
71     }
72     return ret;
73 }
74
75 // count of numbers in the trie such that: (x ^ num) < k
76 int query(int x, int k) {
77     int u = 0, ret = 0;
78     for (int bit = bits - 1; bit >= 0; bit--) {
79         int b_x = x >> bit & 1, b_k = k >> bit & 1;
80         if (b_k) {
81             ret += sz(t[u][b_x]);
82         }
83         u = t[u][b_x ^ b_k];
84         if (!t[u].sz) {
85             break;
86         }
87     }
88     return ret + t[u].sz;
89 }
90 };

```

12 Trees

12.1 Centroid

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int k;
5 long long ans;
6 vector<int> sz, frq;
7 vector<bool> removed;
8 vector<vector<int>> > al;
9
10 int get_sz(int u, int p) {
11     sz[u] = 1;
12     for (auto &v: al[u]) {
13         if (v == p || removed[v]) continue;
14         sz[u] += get_sz(v, u);
15     }
16     return sz[u];
17 }
18
19 int get_centroid(int u, int p, int cur_sz) {
20     for (auto &v: al[u]) {
21         if (v == p || removed[v]) continue;
22         if (sz[v] > cur_sz / 2) {
23             return get_centroid(v, u, cur_sz);
24         }
25     }
26     return u;
27 }
28
29 void get_answer(int u, int p, int d) {
30     ans += (k >= d ? frq[k - d] : 0);
31     for (auto &v: al[u]) {
32         if (v == p || removed[v]) continue;
33         get_answer(v, u, d + 1);
34     }
35 }
36
37 void update(int u, int p, int d, int op) {
38     frq[d] += op;
39     for (auto &v: al[u]) {
40         if (v == p || removed[v]) continue;
41         update(v, u, d + 1, op);
42     }
43 }
44
45 void decomp(int u) {
46     int cur_sz = get_sz(u, -1);
47     int centroid = get_centroid(u, -1, cur_sz);
48
49     frq[0] = 1;
50
51     for (auto &v: al[centroid]) {
52         if (removed[v]) continue;
53         get_answer(v, centroid, 1);
54         update(v, centroid, 1, 1);
55     }
56
57     for (auto &v: al[centroid]) {
58         if (removed[v]) continue;
59         update(v, centroid, 1, -1);
60     }
61
62     removed[centroid] = true;
63
64     for (auto &v: al[centroid]) {
65         if (removed[v]) continue;
66         decomp(v);
67     }

```

```

68 }
69
70 int main() {
71     ios::sync_with_stdio(false);
72     cin.tie(nullptr);
73
74     int n;
75     cin >> n >> k;
76     sz.assign(n + 1, 0);
77     frq.assign(n + 5, 0);
78     removed.assign(n + 1, false);
79     al.assign(n + 1, vector<int>());
80     for (int i = 1; i < n; i++) {
81         int u, v;
82         cin >> u >> v;
83         al[u].push_back(v);
84         al[v].push_back(u);
85     }
86
87     decomp(1);
88
89     cout << ans;
90
91     return 0;
92 }

```

12.2 diameter

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> par;
5 vector<vector<int>> > al;
6 int diameter, farthest, start, endd;
7
8 void dfs(int u, int p, int d) {
9     par[u] = p;
10
11     if (d > diameter) {
12         farthest = u;
13         diameter = d;
14     } else if (d == diameter) {
15         farthest = max(farthest, u);
16     }
17
18     for (auto v: al[u]) {
19         if (v == p) continue;
20         dfs(v, u, d + 1);
21     }
22 }
23
24 void find_diameter(int u) {
25     diameter = -1;
26     farthest = u;
27     dfs(farthest, -1, 0);
28     start = farthest;
29
30     diameter = -1;
31     farthest = start;
32     dfs(farthest, -1, 0);
33     endd = farthest;
34 }

```

12.3 euler_tour

```

1 #include <bits/stdc++.h>

```

```

2 using namespace std;
3
4 int timer = 1;
5 vector<vector<int>> al;
6 vector<int> dep, tin, tout, flatten;
7
8 void euler_tour(int u, int p) {
9     tin[u] = timer;
10    flatten[timer++] = u;
11    for (auto &v: al[u]) {
12        if (v == p) continue;
13        dep[v] = dep[u] + 1;
14        euler_tour(v, u);
15    }
16    tout[u] = timer;
17    flatten[timer++] = u;
18 }

```

12.4 HLD

```

1 // https://codeforces.com/gym/106063/problem/L
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const int mod = 1e9 + 7;
7
8 template<typename T>
9 struct segtree {
10     int sz = 1;
11
12     vector<T> t, lazy;
13
14     segtree(int n) {
15         while (sz < n) sz += sz;
16         t.resize(2 * sz, 1);
17         lazy.resize(2 * sz, 1);
18     }
19
20     long long combine(long long a, long long b) {
21         return (a + b) % mod;
22     }
23
24     void pull(int x) {
25         t[x] = combine(t[2 * x], t[2 * x + 1]);
26     }
27
28     void push(int x, int lx, int rx) {
29         if (lx > rx || rx < 1) return;
30         //TODO
31         t[x] = (t[x] * lazy[x]) % mod;
32         if (lx != rx) {
33             lazy[2 * x] = (lazy[2 * x] * lazy[x]) % mod;
34             lazy[2 * x + 1] = (lazy[2 * x + 1] * lazy[x]) % mod;
35         }
36         lazy[x] = 1;
37     }
38
39     void update(int x, int lx, int rx, int l, int r, T val) {
40         push(x, lx, rx);
41         if (lx > r || rx < l) return;
42         if (lx >= l && rx <= r) {
43             lazy[x] = (lazy[x] * val) % mod;
44             push(x, lx, rx);
45             return;
46         }
47         int m = (lx + rx) >> 1;

```

```

48         update(2 * x, lx, m, l, r, val);
49         update(2 * x + 1, m + 1, rx, l, r, val);
50         pull(x);
51     }
52
53     void update(int l, int r, T val) {
54         update(1, 1, sz, l, r, val);
55     }
56
57     T query(int x, int lx, int rx, int l, int r) {
58         push(x, lx, rx);
59         if (rx < l || lx > r) return 0;
60         if (lx >= l && rx <= r) return t[x];
61         int m = (lx + rx) >> 1;
62         return combine(query(2 * x, lx, m, l, r), query(2 * x + 1, m + 1, rx, l, r));
63     }
64
65     T query(int l, int r) {
66         return query(1, 1, sz, l, r);
67     }
68 };
69
70 struct HLD {
71     int timer = 1;
72     segtree<long long> seg;
73     vector<vector<pair<int, int>> > al;
74     vector<int> sz, d, par, id, heavy, head, weight;
75
76     HLD(int n, vector<vector<pair<int, int>> > &_al) : seg(n), al(_al), sz(n + 1), heavy(n + 1),
77         , par(n + 1), id(n + 1), d(n + 1), head(n
78         + 1), weight(n + 1) {
79         dfs_sz(1);
80         head[1] = 1;
81         dfs_hld(1);
82         for (int u = 1; u <= n; u++) {
83             if (par[u]) seg.update(id[u], id[u], weight[u]);
84         }
85     }
86
87     void dfs_sz(int u) {
88         sz[u] = 1;
89         for (auto &[v, w]: al[u]) {
90             if (v == par[u]) continue;
91             weight[v] = w;
92             par[v] = u;
93             d[v] = d[u] + 1;
94             dfs_sz(v);
95             sz[u] += sz[v];
96             if (!heavy[u] || sz[v] > sz[heavy[u]]) {
97                 heavy[u] = v;
98             }
99         }
100     }
101
102     void dfs_hld(int u) {
103         id[u] = timer++;
104         if (heavy[u]) {
105             head[heavy[u]] = head[u];
106             dfs_hld(heavy[u]);
107         }
108         for (auto &[v, w]: al[u]) {
109             if (v == par[u] || v == heavy[u]) continue;
110             head[v] = v;
111             dfs_hld(v);
112         }
113     }

```

```

114 vector<pair<int, int> > process(int u, int v, bool edges) {
115     vector<pair<int, int> > path;
116     while (head[u] != head[v]) {
117         if (d[head[u]] > d[head[v]]) swap(u, v);
118         path.push_back(make_pair(id[head[v]], id[v]));
119         v = par[head[v]];
120     }
121     if (d[u] > d[v]) swap(u, v);
122     path.push_back(make_pair(id[u] + edges, id[v]));
123     return path;
124 }
125
126 void update(int u, int v, long long x) {
127     auto path = process(u, v, true);
128     for (auto &[l, r]: path) {
129         seg.update(l, r, x);
130     }
131 }
132
133 long long query(int u, int v) {
134     long long ans = 0;
135     auto path = process(u, v, true);
136     for (auto &[l, r]: path) {
137         ans += seg.query(l, r);
138         ans %= mod;
139     }
140     return ans;
141 }
142 };
143
144 int main() {
145     ios::sync_with_stdio(false);
146     cin.tie(nullptr);
147
148     int n, q;
149     cin >> n >> q;
150     vector<vector<pair<int, int> > > al(n + 1);
151     for (int i = 1; i < n; i++) {
152         int u, v, w;
153         cin >> u >> v >> w;
154         al[u].push_back(make_pair(v, w));
155         al[v].push_back(make_pair(u, w));
156     }
157
158     HLD hld(n, al);
159
160     while (q--) {
161         int u, v, x;
162         cin >> u >> v >> x;
163         hld.update(u, v, x);
164         cout << hld.query(u, v) << '\n';
165     }
166
167     return 0;
168 }

```

12.5 LCA with Sparse Tables

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template<typename T>
5 class SparseTable {
6 private:
7     int n, log2dist;
8     vector<vector<T> > st;
9

```

```

10 public:
11     SparseTable(const vector<T> &v) {
12         n = (int) v.size();
13         log2dist = 1 + (int) log2(n);
14         st.resize(log2dist);
15         st[0] = v;
16         for (int i = 1; i < log2dist; i++) {
17             st[i].resize(n - (1 << i) + 1);
18             for (int j = 0; j + (1 << i) <= n; j++) {
19                 st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
20             }
21         }
22     }
23
24     T query(int l, int r) {
25         int i = (int) log2(r - l + 1);
26         return min(st[i][l], st[i][r - (1 << i) + 1]);
27     }
28 };
29
30 class LCA {
31 private:
32     const int n;
33     const vector<vector<int> > &adj;
34     SparseTable<pair<int, int> > rmq;
35     vector<int> tin, et, dep;
36     int timer = 0;
37
38     void dfs(int u, int p) {
39         tin[u] = timer;
40         et[timer++] = u;
41         for (int v: adj[u]) {
42             if (v == p) { continue; }
43             dep[v] = dep[u] + 1;
44             dfs(v, u);
45             et[timer++] = u;
46         }
47     }
48
49 public:
50     // make sure the adjacency list is 0 indexed
51     LCA(vector<vector<int> > &adj)
52         : n((int) _adj.size()), adj(_adj), tin(n), et(2 * n), dep(n),
53           rmq(vector<pair<int, int> >(1)) {
54             dfs(0, -1);
55             vector<pair<int, int> > arr(2 * n);
56             for (int i = 0; i < 2 * n; i++) {
57                 arr[i] = {dep[et[i]], et[i]};
58             }
59             rmq = SparseTable<pair<int, int> >(arr);
60         }
61
62     // LCA of nodes a and b
63     int query(int a, int b) {
64         if (tin[a] > tin[b]) {
65             swap(a, b);
66         }
67         return rmq.query(tin[a], tin[b]).second;
68     }
69 };

```

12.6 lca

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> d;

```

```

5 vector<vector<int> > al, up;
6
7 void dfs_lca(int u, int p) {
8     up[u][0] = p;
9     for (int bit = 1; bit < 18; bit++) {
10         up[u][bit] = (up[u][bit - 1] < 0 ? -1 : up[up[u][bit - 1]][bit - 1]);
11     }
12     for (int v: al[u]) {
13         if (v == p) continue;
14         d[v] = d[u] + 1;
15         dfs_lca(v, u);
16     }
17 }
18
19 int kth(int u, int k) {
20     for (int bit = 0; bit < 18; bit++) {
21         if (k >> bit & 1) u = up[u][bit];
22     }
23     return u;
24 }
25
26 int lca(int u, int v) {
27     if (d[u] > d[v]) swap(u, v);
28     v = kth(v, d[v] - d[u]);
29     if (u == v) return u;
30     for (int bit = 17; bit >= 0; bit--) {
31         if (up[u][bit] != up[v][bit]) {
32             u = up[u][bit];
33             v = up[v][bit];
34         }
35     }
36     return up[u][0];
37 }
38
39 int dist(int u, int v) {
40     return d[u] + d[v] - 2 * d[lca(u, v)];
41 }
42
43 // return node on path u->v at distance k from u
44 int kth_on_path(int u, int v, int k) {
45     int L = lca(u, v), dis = dist(u, v);
46     if (k <= d[u] - d[L]) {
47         return kth(u, k);
48     }
49     return kth(v, dis - k);
50 }

```

12.7 Max Query by LCA

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> d;
5 vector<vector<int> > up, mx;
6 vector<vector<pair<int, int> > > al;
7
8 void dfs_lca(int u, int p) {
9     for (auto &[v, w]: al[u]) {
10         if (v == p) continue;
11
12         d[v] = d[u] + 1;
13         up[v][0] = u, mx[v][0] = w;
14         for (int bit = 1; bit < 18; bit++) {
15             up[v][bit] = up[up[v][bit - 1]][bit - 1];
16             mx[v][bit] = max(mx[v][bit - 1], mx[up[v][bit - 1]][bit - 1]);
17         }
18     }

```

```

19         dfs_lca(v, u);
20     }
21 }
22
23 int queryMx(int u, int v) {
24     if (d[u] > d[v]) {
25         swap(u, v);
26     }
27
28     int ans = 0, k = d[v] - d[u];
29     for (int bit = 0; bit < 18; bit++) {
30         if (k >> bit & 1) {
31             ans = max(ans, mx[v][bit]);
32             v = up[v][bit];
33         }
34     }
35
36     if (u == v) return ans;
37
38     for (int bit = 17; bit >= 0; bit--) {
39         if (up[u][bit] != up[v][bit]) {
40             ans = max(ans, max(mx[u][bit], mx[v][bit]));
41             u = up[u][bit];
42             v = up[v][bit];
43         }
44     }
45     ans = max(ans, max(mx[u][0], mx[v][0]));
46
47     return ans;
48 }

```

12.8 MO on Tree

```

1 // https://www.spoj.com/problems/COT2/
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 void compression(vector<int> &a, int &n) {
7     vector<int> comp(a.begin() + 1, a.end());
8
9     sort(comp.begin(), comp.end());
10    comp.erase(unique(comp.begin(), comp.end()), comp.end());
11
12    for (int i = 1; i <= n; i++) {
13        a[i] = lower_bound(comp.begin(), comp.end(), a[i]) - comp.begin() + 1;
14    }
15 }
16
17 const int N = 40005;
18
19 bool seen[N];
20 vector<int> a, al[N];
21 int tin[N], tout[N], flatten[2 * N], timer, d[N], up[N][20], cur, frq[N];
22
23 void euler_tour(int u, int p) {
24     tin[u] = ++timer;
25     flatten[timer] = u;
26     for (int &v: al[u]) {
27         if (v == p) continue;
28         euler_tour(v, u);
29     }
30     tout[u] = ++timer;
31     flatten[timer] = u;
32 }
33
34 void dfs_lca(int u, int p) {

```

```

35     up[u][0] = p;
36     for (int bit = 1; bit < 18; bit++) {
37         up[u][bit] = (up[u][bit - 1] < 0 ? -1 : up[up[u][bit - 1]][bit - 1]);
38     }
39     for (int v: al[u]) {
40         if (v == p) continue;
41         d[v] = d[u] + 1;
42         dfs_lca(v, u);
43     }
44 }
45
46 int kth(int u, int k) {
47     for (int bit = 0; bit < 18; bit++) {
48         if (k >> bit & 1) u = up[u][bit];
49     }
50     return u;
51 }
52
53 int lca(int u, int v) {
54     if (d[u] > d[v]) swap(u, v);
55     v = kth(v, d[v] - d[u]);
56     if (u == v) return u;
57     for (int bit = 17; bit >= 0; bit--) {
58         if (up[u][bit] != up[v][bit]) {
59             u = up[u][bit];
60             v = up[v][bit];
61         }
62     }
63     return up[u][0];
64 }
65
66 struct Query {
67     int l, r, idx, lca;
68 };
69
70 void add(int u) {
71     frq[a[u]]++;
72     cur += (frq[a[u]] == 1);
73 }
74
75 void remove(int u) {
76     cur -= (frq[a[u]] == 1);
77     frq[a[u]]--;
78 }
79
80 void toggle(int u) {
81     if (seen[u]) {
82         remove(u);
83     } else {
84         add(u);
85     }
86     seen[u] = !seen[u];
87 }
88
89 int main() {
90     ios::sync_with_stdio(false);
91     cin.tie(nullptr);
92
93     int n, m;
94     cin >> n >> m;
95     int block_size = ceil(sqrt(n));
96
97     a.assign(n + 1, 0);
98     for (int u = 1; u <= n; u++) {
99         cin >> a[u];
100     }
101
102     compression(a, n);

```

```

103
104     for (int i = 1; i < n; i++) {
105         int u, v;
106         cin >> u >> v;
107         al[u].push_back(v);
108         al[v].push_back(u);
109     }
110
111     euler_tour(1, 0);
112
113     dfs_lca(1, 0);
114
115     vector<Query> queries;
116     for (int i = 0; i < m; i++) {
117         int u, v;
118         cin >> u >> v;
119         if (tin[v] < tin[u]) swap(u, v);
120
121         int lc = lca(u, v);
122         if (lc == u) {
123             queries.push_back({tin[u], tin[v], i, -1});
124         } else {
125             queries.push_back({tout[u], tin[v], i, lc});
126         }
127     }
128
129     sort(queries.begin(), queries.end(), [&](const Query &a, const Query &b) {
130         return make_pair(a.l / block_size, ((a.l / block_size) & 1) ? -a.r : +a.r) < make_pair(b
131             .l / block_size, ((b.l / block_size) & 1) ? -b.r : +b.r);
132     });
133
134     vector<int> ans(m);
135     int mo_left = 1, mo_right = 0;
136     for (const auto &[l, r, idx, lca]: queries) {
137         while (mo_left > l) toggle(flatten[--mo_left]);
138         while (mo_right < r) toggle(flatten[++mo_right]);
139         while (mo_left < l) toggle(flatten[mo_left++]);
140         while (mo_right > r) toggle(flatten[mo_right--]);
141
142         if (~lca) add(lca);
143         ans[idx] = cur;
144         if (~lca) remove(lca);
145     }
146
147     for (int i = 0; i < m; i++) {
148         cout << ans[i] << '\n';
149     }
150     return 0;
151 }

```